

```
In [16]: import os
import sys
import glob
import matplotlib.pyplot as plt
import numpy as np
import scipy.constants as const
import h5py
from importlib import reload
import m3dc1_units
reload(m3dc1_units)
from matplotlib import cm
REpath = './RE_Solver'
import sys
sys.path.append(REpath)
import RE as RE
```

```
In [17]: %cd "/global/homes/r/rdatta/m3dc1_vizualization/"
/global/u1/r/rdatta/m3dc1_vizualization
```

```
In [18]: # Uncomment the next line Install these libraries once
# %pip install termcolor pint matplotlib-label-lines h5py scipy
```

```
In [232... # Helper functions

def print_h5_hierarchy(name, obj):
    """Callback function to print the name of each object in the file."""
    if isinstance(obj, h5py.Dataset):
        print(f"Dataset: {name}")
    elif isinstance(obj, h5py.Group):
        print(f"Group: {name}")

# To view hdf5 file keys
filename = inDir + sim_id + 'C1.h5'
with h5py.File(filename, 'r') as f:
    print(f"--- Keys in {filename} ---")
    # To see just the top-level keys:
    print("Top-level keys:", list(f.keys()))

    print("\n--- Full Hierarchy ---")
    # This visits every group and dataset recursively
    f.visititems(print_h5_hierarchy)

def get_h5_dataset(dataset_path, filename=None):
    """
    Returns the data from a specific path in C1.h5 as a numpy array.
    """
    if filename is None:
        # Uses variables already defined in your notebook
        filename = inDir + sim_id + 'C1.h5'
```

```

try:
    with h5py.File(filename, 'r') as f:
        if dataset_path in f:
            # [()] loads the entire dataset into a numpy array
            data = f[dataset_path][()]
            return data
        else:
            print(f"Error: Path '{dataset_path}' not found in {filename}")
            return None
except Exception as e:
    print(f"An error occurred: {e}")
    return None

# Mask
th = np.linspace(-0.2*np.pi, 180)
xx = 1.85 + 0.58 * np.cos(th + 0.5 * np.sin(th))
zz = 0.0 + 1.21 * np.sin(th)

def load_jre(tid, angle=0):
    tid = '-' + str(tid)
    jre = np.loadtxt(inDir + sim_id + '/idl_output/' + 'nre' + tid + '-' + s
    R = np.loadtxt(inDir + sim_id + '/idl_output/' + 'R.txt')
    Z = np.loadtxt(inDir + sim_id + '/idl_output/' + 'Z.txt')

    R = R.flatten()
    Z = Z.flatten()
    jre = jre.reshape(R.shape[0], Z.shape[0])
    jre = -1 * m3dc1_units.unit['j0'] * jre # A/m^2
    return R, Z, jre

def load_data(tid, field, phi=0):
    tid = '-' + str(tid)
    out = np.loadtxt(inDir + sim_id + '/idl_output/' + field + tid + '-' + s
    R = np.loadtxt(inDir + sim_id + '/idl_output/' + 'R.txt')
    Z = np.loadtxt(inDir + sim_id + '/idl_output/' + 'Z.txt')
    lcfs = np.loadtxt(inDir + sim_id + '/idl_output/' + 'lcfs' + tid + '.txt')

    R = R.flatten()
    Z = Z.flatten()
    out = out.reshape(R.shape[0], Z.shape[0])
    return R, Z, out, lcfs

def get_time(tid):
    data_time = np.loadtxt(inDir + sim_id + '/times_values.csv', delimiter='
    from scipy.interpolate import interp1d
    f = interp1d(data_time[:, 0], data_time[:, 1])
    return f(tid)

from scipy.interpolate import RegularGridInterpolator

def find_contour_intersections(jre, R, Z, xx, zz, min_val=1e-4): # Extracts
    """
    Evaluates the 2D jre field along a contour defined by (xx, zz)

```

```
and returns the coordinates where the field exceeds min_val.
''''
# 1. Create an interpolator for the jre data on the R, Z grid.
# (Note: RegularGridInterpolator expects R and Z to be strictly ascending
# which is standard for the M3D-C1 IDL output grids).
interp = RegularGridInterpolator((R, Z), jre, bounds_error=False, fill_v

# 2. Stack the contour coordinates into an array of (R, Z) points
contour_points = np.column_stack((xx, zz))

# 3. Evaluate the jre field at every point along the contour
jre_along_contour = interp(contour_points)

# 4. Find the indices where the evaluated jre is greater than the thresh
valid_indices = np.where(jre_along_contour > min_val)[0]

# 5. Extract the R, Z locations and the specific jre values at those str
intersect_R = xx[valid_indices]
intersect_Z = zz[valid_indices]
intersect_vals = jre_along_contour[valid_indices]

return intersect_R, intersect_Z, intersect_vals
```

```

--- Keys in /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/C1.h5 ---
Top-level keys: ['bharmonics', 'equilibrium', 'keharmonics', 'kspits', 'scal
ars', 'time_000', 'time_001', 'time_002', 'time_003', 'time_004', 'time_00
5', 'time_006', 'time_007', 'time_008', 'time_009', 'time_010', 'time_011',
'time_012', 'time_013', 'time_014', 'time_015', 'time_016', 'time_017', 'tim
e_018', 'time_019', 'time_020', 'time_021', 'time_022', 'time_023', 'time_02
4', 'time_025', 'time_026', 'time_027', 'time_028', 'time_029', 'time_030',
'time_031', 'time_032', 'time_033', 'time_034', 'time_035', 'time_036', 'tim
e_037', 'time_038', 'time_039', 'time_040', 'time_041', 'time_042', 'time_04
3', 'time_044', 'time_045', 'time_046', 'time_047', 'time_048', 'time_049',
'time_050', 'time_051', 'time_052', 'time_053', 'time_054', 'time_055', 'tim
e_056', 'time_057', 'time_058', 'time_059', 'time_060', 'time_061', 'time_06
2', 'time_063', 'time_064', 'time_065', 'time_066', 'time_067', 'time_068',
'time_069', 'time_070', 'time_071', 'time_072', 'time_073', 'time_074', 'tim
e_075', 'time_076', 'time_077', 'time_078', 'time_079', 'time_080', 'time_08
1', 'time_082', 'time_083', 'time_084', 'time_085', 'time_086', 'time_087',
'time_088', 'time_089', 'time_090', 'time_091', 'time_092', 'time_093', 'tim
e_094', 'time_095', 'time_096', 'time_097', 'time_098', 'time_099', 'time_10
0', 'time_101', 'time_102', 'time_103', 'time_104', 'time_105', 'time_106',
'time_107', 'time_108', 'time_109', 'time_110', 'time_111', 'time_112', 'tim
e_113', 'time_114', 'time_115', 'time_116', 'time_117', 'time_118', 'time_11
9', 'time_120', 'time_121', 'time_122', 'time_123', 'time_124', 'time_125',
'time_126', 'time_127', 'time_128', 'time_129', 'time_130', 'time_131', 'tim
e_132', 'time_133', 'time_134', 'time_135', 'time_136', 'time_137', 'time_13
8', 'time_139', 'time_140', 'time_141', 'time_142', 'time_143', 'time_144',
'time_145', 'time_146', 'time_147', 'time_148', 'time_149', 'time_150', 'tim
e_151', 'time_152', 'time_153', 'time_154', 'time_155', 'time_156', 'time_15
7', 'time_158', 'time_159', 'time_160', 'time_161', 'time_162', 'time_163',
'time_164', 'time_165', 'time_166', 'time_167', 'time_168', 'time_169', 'tim
e_170', 'time_171', 'time_172', 'time_173', 'time_174', 'time_175', 'time_17
6', 'time_177', 'time_178', 'time_179', 'time_180', 'time_181', 'time_182',
'time_183', 'time_184', 'time_185', 'time_186', 'time_187', 'time_188', 'tim
e_189', 'time_190', 'time_191', 'time_192', 'time_193', 'time_194', 'time_19
5', 'time_196', 'time_197', 'time_198', 'time_199', 'time_200', 'time_201',
'time_202', 'time_203', 'time_204', 'time_205', 'time_206', 'time_207', 'tim
e_208', 'time_209', 'time_210', 'time_211', 'time_212', 'time_213', 'time_21
4', 'time_215', 'time_216', 'time_217', 'time_218', 'time_219', 'time_220',
'time_221', 'time_222', 'time_223', 'time_224', 'time_225', 'time_226', 'tim
e_227', 'time_228', 'time_229', 'time_230', 'time_231', 'time_232', 'time_23
3', 'time_234', 'time_235', 'time_236', 'time_237', 'time_238', 'time_239',
'time_240', 'time_241', 'time_242', 'time_243', 'time_244', 'time_245', 'tim
e_246', 'time_247', 'time_248', 'time_249', 'time_250', 'time_251', 'time_25
2', 'time_253', 'time_254', 'time_255', 'time_256', 'time_257', 'time_258',
'time_259', 'timings']

```

```

--- Full Hierarchy ---

```

```

Group:    bharmonics
Dataset:  bharmonics/bharmonics
Group:    keharmonics
Dataset:  keharmonics/keharmonics
Group:    kspits
Dataset:  kspits/kspits
Group:    scalars
Dataset:  scalars/Ave_P
Dataset:  scalars/E_K3
Dataset:  scalars/E_K3D

```


Dataset: scalars/E_K3H
Dataset: scalars/E_KP
Dataset: scalars/E_KPD
Dataset: scalars/E_KPH
Dataset: scalars/E_KT
Dataset: scalars/E_KTD
Dataset: scalars/E_KTH
Dataset: scalars/E_MP
Dataset: scalars/E_MPC
Dataset: scalars/E_MPD
Dataset: scalars/E_MPH
Dataset: scalars/E_MPV
Dataset: scalars/E_MT
Dataset: scalars/E_MTC
Dataset: scalars/E_MTD
Dataset: scalars/E_MTH
Dataset: scalars/E_MTV
Dataset: scalars/E_P
Dataset: scalars/E_PD
Dataset: scalars/E_PE
Dataset: scalars/E_PH
Dataset: scalars/E_grav
Dataset: scalars/Flux_kinetic
Dataset: scalars/Flux_poynting
Dataset: scalars/Flux_pressure
Dataset: scalars/Flux_thermal
Dataset: scalars/IP_co
Dataset: scalars/IP_sn
Dataset: scalars/M_IZ
Dataset: scalars/M_IZ_co
Dataset: scalars/M_IZ_sn
Dataset: scalars/Parallel_viscous_heating
Dataset: scalars/Particle_Flux_convective
Dataset: scalars/Particle_Flux_diffusive
Dataset: scalars/Particle_source
Dataset: scalars/Torque_com
Dataset: scalars/Torque_em
Dataset: scalars/Torque_gyro
Dataset: scalars/Torque_parvisc
Dataset: scalars/Torque_sol
Dataset: scalars/Torque_visc
Dataset: scalars/W_M
Dataset: scalars/W_P
Dataset: scalars/Wall_Force_n0_x
Dataset: scalars/Wall_Force_n0_x_halo
Dataset: scalars/Wall_Force_n0_y
Dataset: scalars/Wall_Force_n0_z
Dataset: scalars/Wall_Force_n0_z_halo
Dataset: scalars/Wall_Force_n1_x
Dataset: scalars/Wall_Force_n1_y
Dataset: scalars/angular_momentum
Dataset: scalars/angular_momentum_p
Dataset: scalars/area
Dataset: scalars/area_p
Dataset: scalars/brem_rad
Dataset: scalars/circulation

Dataset: scalars/dt
Dataset: scalars/electron_number
Dataset: scalars/helicity
Dataset: scalars/i_control%err_i
Dataset: scalars/i_control%err_p_old
Dataset: scalars/ion_loss
Dataset: scalars/kprad_dt
Dataset: scalars/kprad_n
Dataset: scalars/kprad_n0
Dataset: scalars/line_rad
Dataset: scalars/loop_voltage
Dataset: scalars/n_control%err_i
Dataset: scalars/n_control%err_p_old
Dataset: scalars/particle_number
Dataset: scalars/particle_number_p
Dataset: scalars/power_injected
Dataset: scalars/psi0
Dataset: scalars/psi_lcfs
Dataset: scalars/psimin
Dataset: scalars/radiation
Dataset: scalars/reck_rad
Dataset: scalars/recp_rad
Dataset: scalars/runaways
Dataset: scalars/temax
Dataset: scalars/time
Dataset: scalars/toroidal_current
Dataset: scalars/toroidal_current_p
Dataset: scalars/toroidal_current_w
Dataset: scalars/toroidal_flux
Dataset: scalars/toroidal_flux_p
Dataset: scalars/volume
Dataset: scalars/volume_p
Dataset: scalars/volume_pd
Dataset: scalars/xmag
Dataset: scalars/xnull
Dataset: scalars/xnull2
Dataset: scalars/zmag
Dataset: scalars/znull
Dataset: scalars/znull2
Group: timings
Dataset: timings/t_aux
Dataset: timings/t_kprad
Dataset: timings/t_ludefall
Dataset: timings/t_mvm
Dataset: timings/t_onestep
Dataset: timings/t_output_cgm
Dataset: timings/t_output_hdf5
Dataset: timings/t_output_reset
Dataset: timings/t_smoother
Dataset: timings/t_solve_b
Dataset: timings/t_solve_n
Dataset: timings/t_solve_p
Dataset: timings/t_solve_v
Dataset: timings/t_sources

Directory

```
In [20]: # Directory and ID
inDir = '/pscratch/sd/r/rdata/'
sim_id = "REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/"
```

Current vs. Time

\$I_p\$ vs \$t\$

```
In [21]: ip = get_h5_dataset('scalars/toroidal_current_p',filename=inDir + sim_id + '
t = get_h5_dataset('scalars/time',filename=inDir + sim_id + 'C1.h5')

ip = ip * m3dc1_units.unit["j0"]
t = t * m3dc1_units.unit["tau_A"] * np.sqrt(2)

ip0 = 10.28526e6
i_remc = 350 * (1-ip/ip0) # kA
i_remc[np.where(t<1.0e-3)] = 0.1

fig, axs = plt.subplots(1,1,figsize=(4,2),dpi=400)
print('I_p [MA] = %1.7g'%(ip[-1]/1e6))
plt.plot(t*1e3,ip/1e6,'-') # plot ip
plt.ylim([0,12])

plt.plot(t*1e3,i_remc/100,'--',color='tab:red') # remc current

#### Ire #####
re = get_h5_dataset('scalars/runaways',filename=inDir + sim_id + 'C1.h5')
time = get_h5_dataset('scalars/time',filename=inDir + sim_id + 'C1.h5')

re = re * m3dc1_units.unit["j0"]
time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2)

plt.plot(t*1e3,re/1e6,'-',color='tab:orange') # plot ire

axs.grid(alpha=0.5)
axs.set_xlabel('t [ms]')
axs.set_ylabel('Current [MA]')
axs.set_xlim([0,2.5])
axs.set_yticks(np.arange(0,13,2))

#### Zmag #####

zmag = get_h5_dataset('scalars/zmag',filename=inDir + sim_id + 'C1.h5')
time = get_h5_dataset('scalars/time',filename=inDir + sim_id + 'C1.h5')

time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2)

ax2 = axs.twinx()
```

```

ax2.plot(time*1e3,zmag,'-.',color='purple',alpha=0.7)
ax2.set_ylim([-1.2,1.2])
# --- Add these lines to color the axis elements purple ---
ax2.spines['right'].set_color('purple')           # Colors the vertical spine
ax2.tick_params(axis='y', colors='purple')        # Colors the ticks and numbe
ax2.set_ylabel('$Z_{mag}$ [m]',color='purple')    # Colors the axis label text

ax2.set_yticks(np.arange(-1.2,1.21,0.4))

# 2D

# ip = get_h5_dataset('scalars/toroidal_current_p',filename=inDir + 'REMC/TQ
# t = get_h5_dataset('scalars/time',filename=inDir + 'REMC/TQ/SPRC-m8-Ne-REM

# ip = ip * m3dc1_units.unit["j0"]
# t = t * m3dc1_units.unit["tau_A"] * np.sqrt(2)

# re = get_h5_dataset('scalars/runaways',filename=inDir + 'REMC/TQ/SPRC-m8-M
# re = re * m3dc1_units.unit["j0"]

# axs.plot(t*1e3+1.0,ip/1e6,'--',color='tab:blue')
# axs.plot(t*1e3+1.0,re/1e6,'--',color='tab:orange')

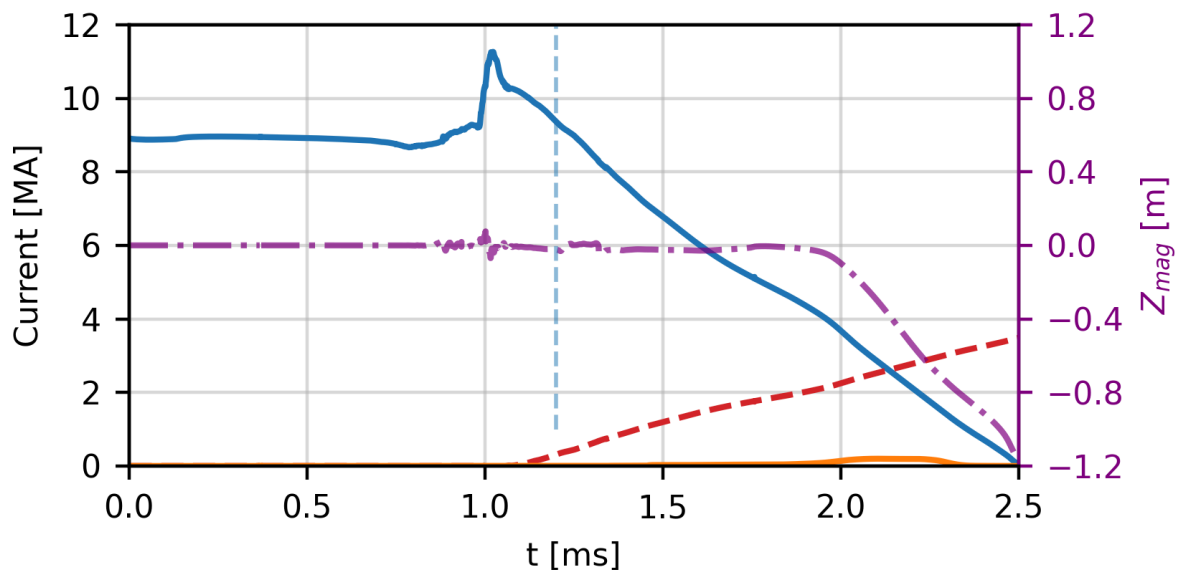
# # extrapolating
# didt = (ip[-10] - ip[-100])/(t[-10] - t[-100])
# tt = np.linspace(t[-10]*1e3,3,100) * 1e-3
# y = didt * (tt - t[-100]) + ip[-100]
# plt.plot(tt*1e3,y/1e6,'b:',alpha=0.5)

# Lines
plt.plot([1.2]*2,[-1,12],'-.',alpha=0.5,linewidth=1)
# plt.plot([0,2.5],[8.9]*2,'--',alpha=0.5,linewidth=1)

```

I_p [MA] = 0.03902116

Out[21]: [<matplotlib.lines.Line2D at 0x7f95496eec10>]



I_{RE} vs t

```
In [150... # RE Current vs time

# Load
re = get_h5_dataset('scalars/runaways',filename=inDir + sim_id + 'C1.h5')
time = get_h5_dataset('scalars/time',filename=inDir + sim_id + 'C1.h5')

re = re * m3dc1_units.unit["j0"]
time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2)

print('I_RE [kA] = %1.3g'%(re[-1]/1e3))

# Plot

fig, axs = plt.subplots(1,1,figsize=(4,2),dpi=400)

axs.plot(time*1e3,re/1e3,'-',alpha=1,color='tab:orange')

axs.set_xlabel('t [ms]')
axs.set_ylabel('I$_{RE}$ [kA]')
axs.set_ylim(ymin=1e-3,ymax=250)
axs.set_xlim(xmin=0,xmax=2.5)
axs.set_yscale('log')
axs.grid(which='both',alpha=0.35)

re = get_h5_dataset('scalars/runaways',filename=inDir + 'REMC/TQ/SPRC-m8-Ne-
time = get_h5_dataset('scalars/time',filename=inDir + 'REMC/TQ/SPRC-m8-Ne-RE

re = re * m3dc1_units.unit["j0"]
time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2)
# axs.plot(time*1e3,re/1e3,'--',alpha=0.5,color='tab:orange')

# Temperature

# ax2 = axs
# temax = get_h5_dataset('scalars/temax',filename=inDir + sim_id + 'C1.h5')
# time = get_h5_dataset('scalars/time',filename=inDir + sim_id + 'C1.h5')

# temax = temax * m3dc1_units.unit["T0"] - 7.5
# time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2)

# temax = np.hstack([2e4,temax])
# time = np.hstack([0,time])

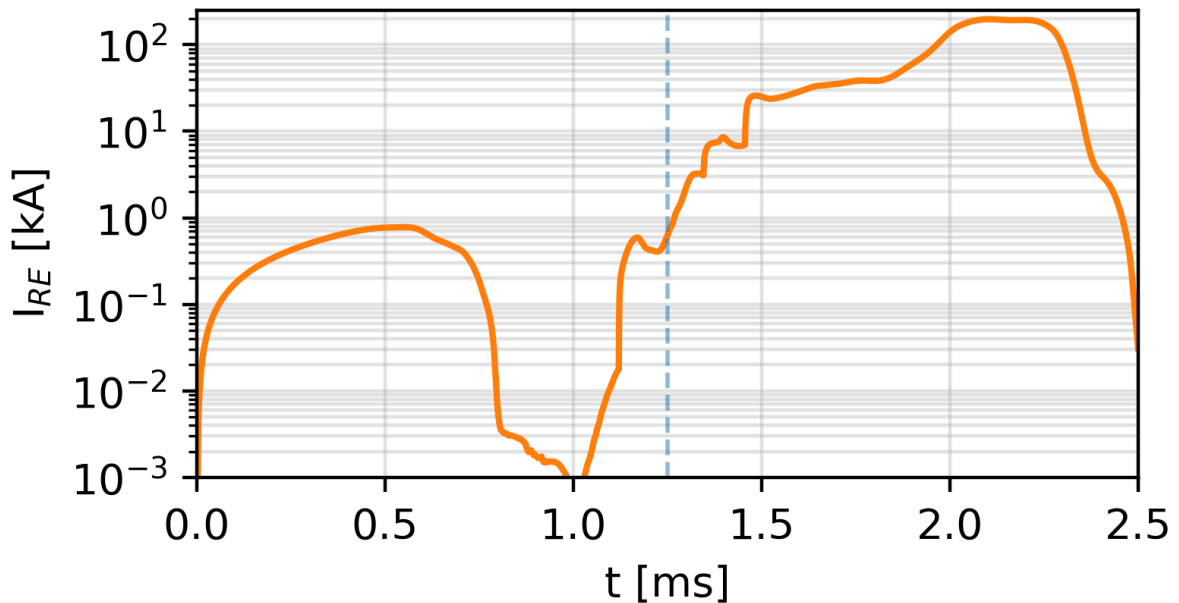
# ax2.plot(time*1e3,temax,color='tab:red')
# ax2.set_yscale('log')
# ax2.set_ylim([0,2e4])
```

```
# ax2.set_ylabel('$T_{e,max}$ [keV]')

# lines
plt.plot([1.25]*2,[1e-3,1e3],'--',alpha=0.5,linewidth=1)
# plt.plot([2.05]*2,[1e-3,1e3],'--',alpha=0.5,linewidth=1)
```

I_RE [kA] = 0.0217

Out[150... [<matplotlib.lines.Line2D at 0x7f2088892350>]



Max. Temp vs time

```
In [326... # Max Temp. vs time

temax = get_h5_dataset('scalars/temax',filename=inDir + sim_id + 'C1.h5')
time = get_h5_dataset('scalars/time',filename=inDir + sim_id + 'C1.h5')

temax = temax * m3dc1_units.unit["T0"] - 7.5
time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2)

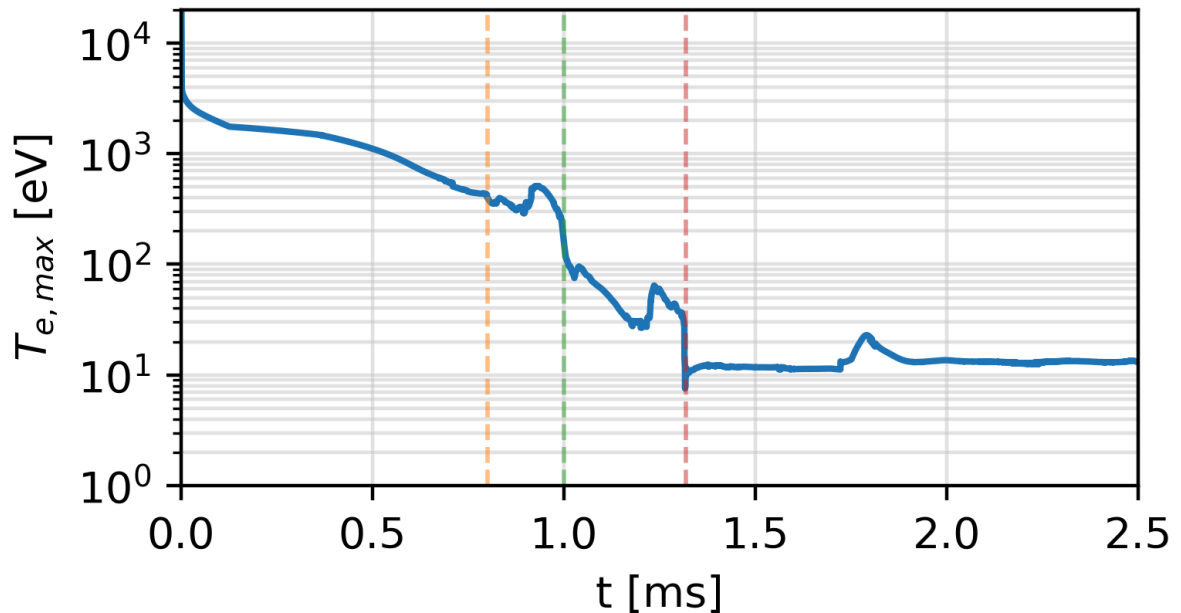
temax = np.hstack([20e4,temax])
time = np.hstack([0,time])

fig, axs = plt.subplots(1,1,figsize=(4,2),dpi=400)
plt.plot(time*1e3,temax)
plt.xlabel('t [ms]')
plt.ylabel('$T_{e,max}$ [eV]')
plt.yscale('log')
plt.ylim([1,20e3])
axs.set_xlim(xmin=0.0,xmax=2.5)
plt.grid(which='both',alpha=0.33)

# Lines
plt.plot([0.8]*2,[1,2e4],'--',alpha=0.5,linewidth=1)
```

```
plt.plot([1.0]*2,[1,2e4], '--',alpha=0.5,linewidth=1)
plt.plot([1.32]*2,[1,2e4], '--',alpha=0.5,linewidth=1)
```

Out[326... [<matplotlib.lines.Line2D at 0x7f3c30f97890>]



Magnetic axis position vs time

```
In [327... # Zmag. vs time

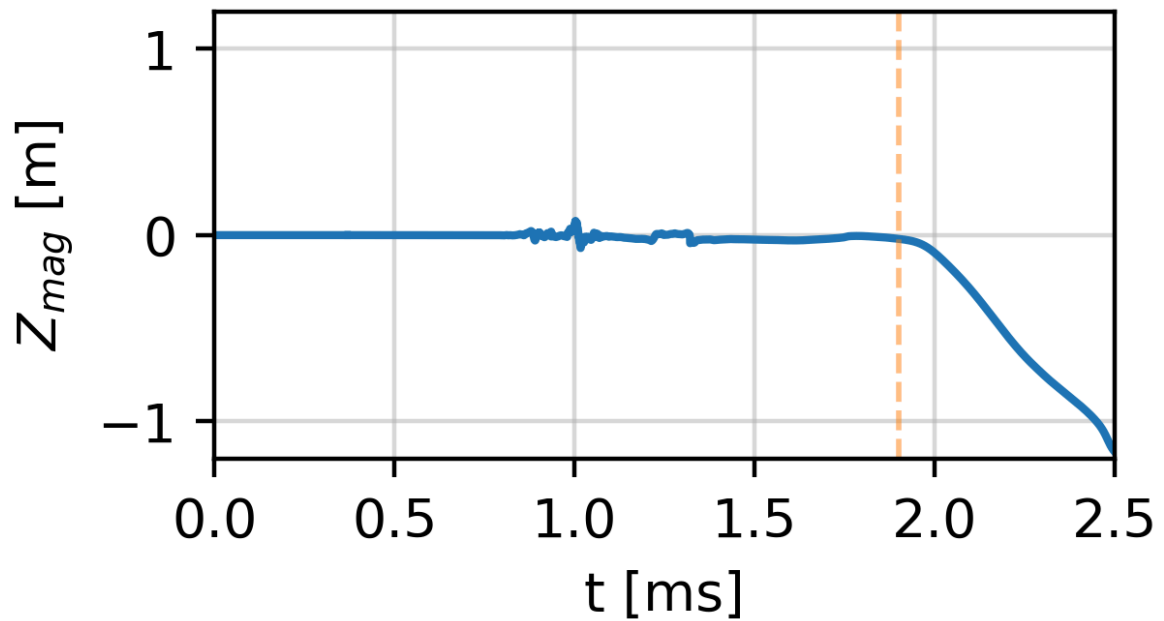
zmag = get_h5_dataset('scalars/zmag',filename=inDir + sim_id + 'C1.h5')
time = get_h5_dataset('scalars/time',filename=inDir + sim_id + 'C1.h5')

time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2)

fig, axs = plt.subplots(1,1,figsize=(3,1.5),dpi=400)
plt.plot(time*1e3,zmag)
plt.xlabel('t [ms]')
plt.ylabel('$Z_{mag}$ [m]')
# plt.yscale('log')
plt.ylim([-1.2,1.2])
plt.xlim([0,2.5])
plt.grid(alpha=0.5)

plt.plot([1.9]*2,[-1.2,1.2], '--',alpha=0.5,linewidth=1)
```

Out[327... [<matplotlib.lines.Line2D at 0x7f3c30f0e710>]



Inductances

Inductance and current profile

```
In [271... # Poloidal Magnetic Energy

time = get_h5_dataset('scalars/time',filename=inDir + sim_id + 'C1.h5')
time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2) # time, s

W_P = get_h5_dataset('scalars/E_MP',filename=inDir + sim_id + 'C1.h5') # mag
W_MPV = get_h5_dataset('scalars/E_MPV',filename=inDir + sim_id + 'C1.h5') #
W_MPC = get_h5_dataset('scalars/E_MPC',filename=inDir + sim_id + 'C1.h5') #

# Unit conversion

W_P = W_P * m3dc1_units.unit['B0']**2/const.mu_0 # J
W_MPV = W_MPV * m3dc1_units.unit['B0']**2/const.mu_0 # J
W_MPC = W_MPC * m3dc1_units.unit['B0']**2/const.mu_0 # J

W_tot = 70e6 # MJ (This is the expected value)
W_ext = W_tot - W_P

print('W_P [MJ] = %1.3g'%(W_P[0]/1e6))
print('W_ext [MJ] = %1.3g'%(W_ext[0]/1e6))
# print('W_MPV [MJ] = %1.3g'%(W_MPV[0]/1e6))
# print('W_MPC [MJ] = %1.3g'%(W_MPC[0]/1e6))

# print(' ')
# print('At the end of sim:')
# print('W_P [MJ] = %1.3g'%(W_P[-1]/1e6))
# print('W_MPV [MJ] = %1.3g'%(W_MPV[-1]/1e6))
# print('W_MPC [MJ] = %1.3g'%(W_MPC[-1]/1e6))
# print(' ')
```



```

# Inductance calculations

ip = get_h5_dataset('scalars/toroidal_current_p', filename=inDir + sim_id + '
ip = ip * m3dc1_units.unit["j0"] # A, plasma current

Li = 2 * W_P / ip**2 # internal inductance
Lext = 2 * W_ext[0] / ip**2 # external inductance
Ltot = Lext[0] + Li

print('Li [uH] = %1.3g'%(Li[0]*1e6))
print('Lext [uH] = %1.3g'%(Lext[0]*1e6))
print('Ltot [uH] = %1.3g'%(Ltot[0]*1e6))

fig, axs = plt.subplots(2,1,dpi=300,figsize=(3,3),sharex=True)
axs[0].plot(time*1e3,W_P/1e6,label='Plasma')
axs[0].legend()
axs[0].set_ylim(ymin=0)
axs[0].grid()
axs[0].set_ylabel('Mag. Energy [MJ]')

axs[1].plot(time*1e3,Li*1e6,label='$L_p$')
axs[1].plot(time*1e3,Ltot*1e6,label='$L_{iv}$')
axs[1].plot(time*1e3,Ltot*1e6*150/70,label='$L_{tot}$')
axs[1].legend()
axs[1].set_ylim(ymin=0)
axs[-1].set_xlabel('t [ms]')
axs[1].grid()
axs[1].set_ylabel('Inductance [uH]')
axs[-1].set_ylim([0,5])
axs[-1].set_xlim([0,2])

# Lines
# axs[-1].plot([0,2],[1.1]*2,'--k',alpha=0.5)
# axs[-1].plot([0,2],[1.8]*2,'--k',alpha=0.5)
# axs[-1].plot([0,2],[1.24]*2,'--r',alpha=0.5)
# axs[-1].plot([0,2],[0.62]*2,'--r',alpha=0.5)

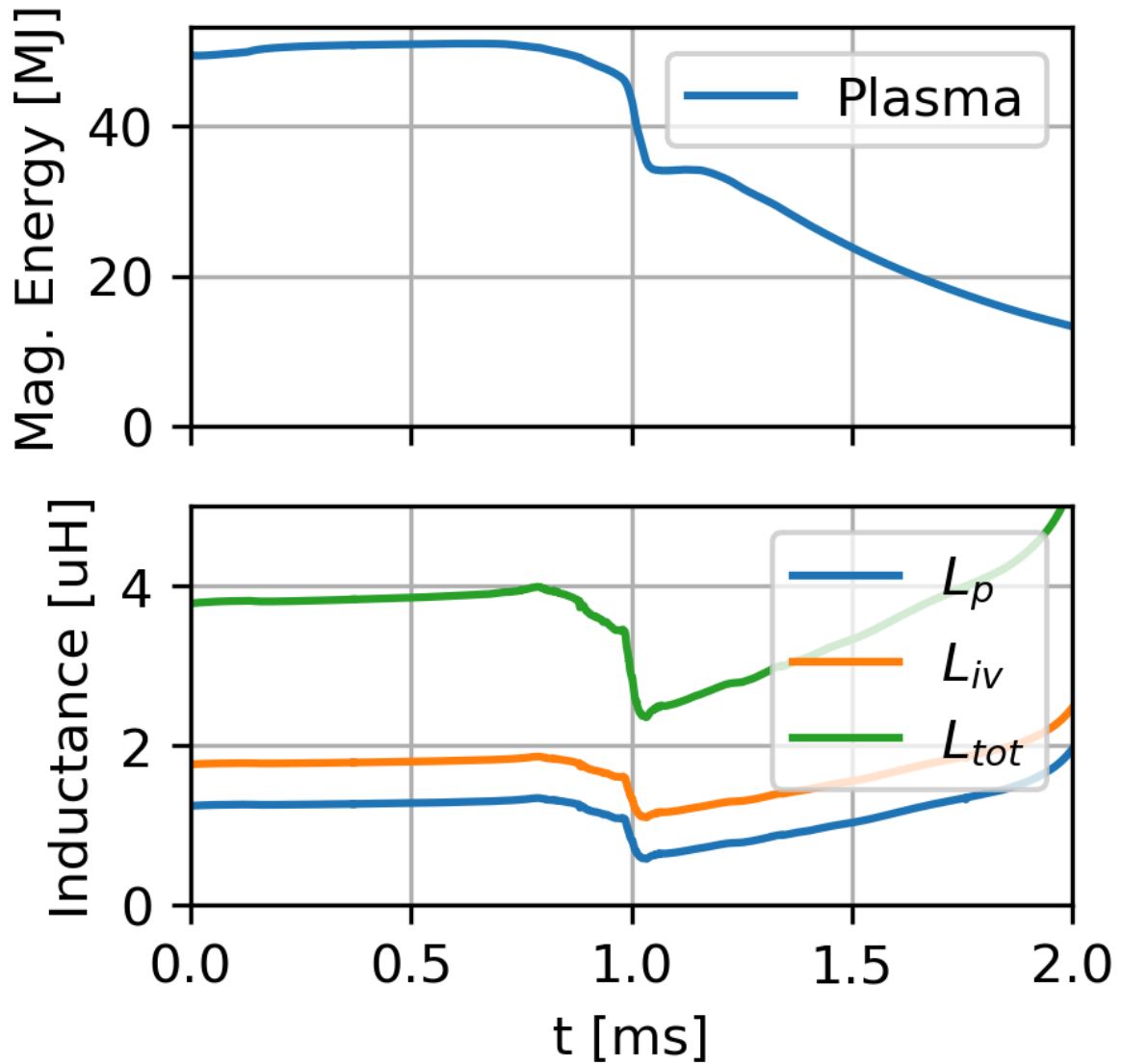
```

```

W_P [MJ] = 49.4
W_ext [MJ] = 20.6
Li [uH] = 1.24
Lext [uH] = 0.519
Ltot [uH] = 1.76

```

```
Out[271...] (0.0, 2.0)
```



Current profile flattening

```
In [261... # Current profile flattening

tids = np.array([47,100])

fig, axs = plt.subplots(1,1,dpi=300,figsize=(2,1))

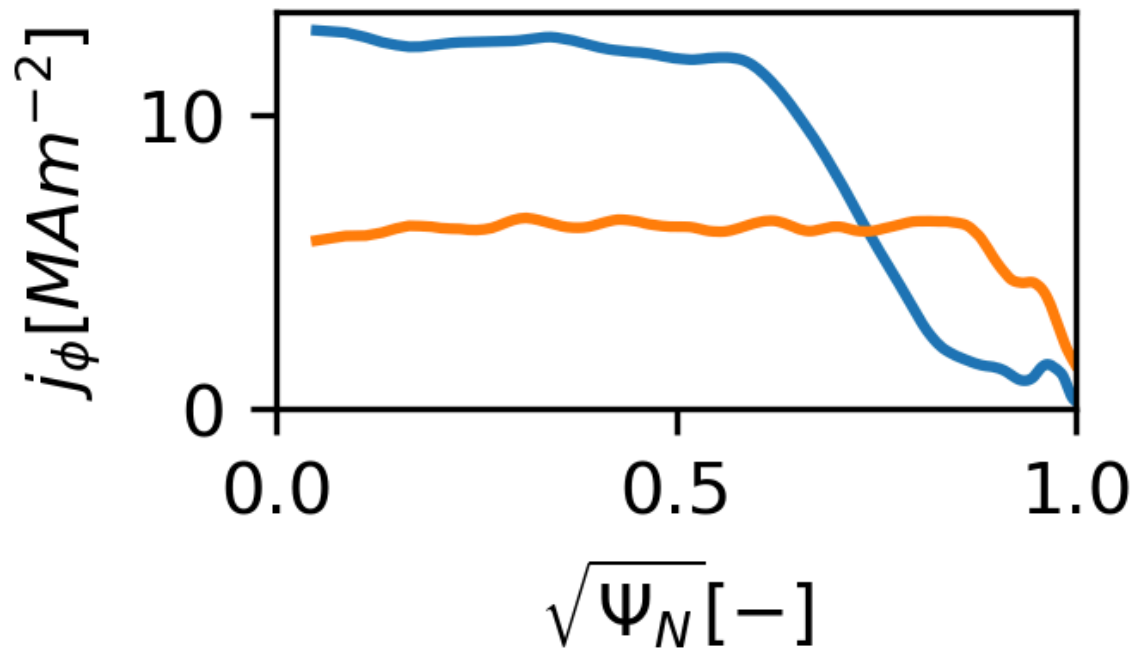
for ii in range(tids.shape[0]):
    data = np.loadtxt(inDir + sim_id + 't_' + str(tids[ii]) + '_jy_plasma_vs
    psi = data[:,0]
    jy = data[:,1]

    axs.plot(np.sqrt(psi),jy/1e6)
    axs.set_xlabel(r'$\sqrt{\Psi_N}$ [-]$')
    axs.set_ylabel(r'$j_\phi$ [MAm-2]$')

# axs.grid()
axs.set_ylim(ymin=0)
```

```
axs.set_xlim([0,1])
```

Out [261... (0.0, 1.0)



```
In [261... # Current profile flattening

tids = np.array([47,100])

fig, axs = plt.subplots(1,1,dpi=300,figsize=(2,1))

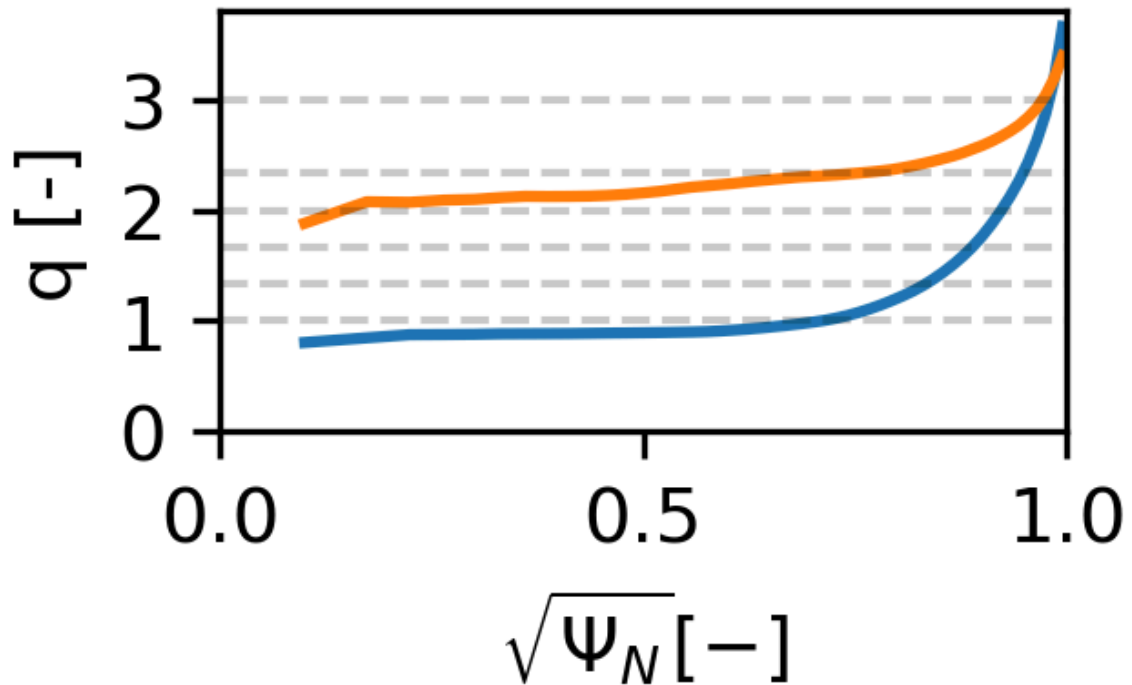
for ii in range(tids.shape[0]):
    data = np.loadtxt(inDir + sim_id + '/q_profiles/q-' + str(tids[ii]) + '.
    psi = data[:,0]
    q = data[:,1]

    axs.plot(np.sqrt(psi),q)
    axs.set_xlabel(r'$\sqrt{\Psi_N}$ [-]$')
    axs.set_ylabel(r'q [-]')

# axs.grid()
axs.set_ylim(ymin=0)
axs.set_xlim([0,1])
axs.set_yticks([0,1,2,3])

axs.plot([0,1],[1]*2,'k--',alpha=0.2,linewidth=1)
axs.plot([0,1],[4/3]*2,'k--',alpha=0.2,linewidth=1)
axs.plot([0,1],[5/3]*2,'k--',alpha=0.2,linewidth=1)
axs.plot([0,1],[2]*2,'k--',alpha=0.2,linewidth=1)
axs.plot([0,1],[7/3]*2,'k--',alpha=0.2,linewidth=1)
axs.plot([0,1],[3]*2,'k--',alpha=0.2,linewidth=1)
```

Out[261... [<matplotlib.lines.Line2D at 0x7f3c43a02850>]



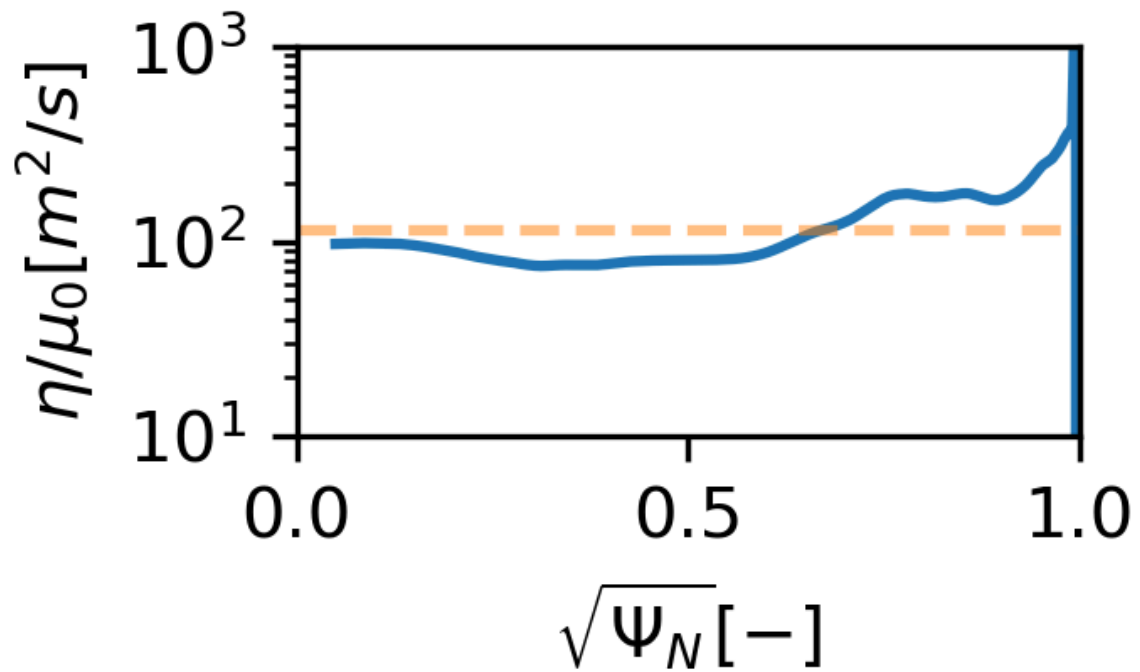
L/R Time

```
In [275... data = np.loadtxt(inDir + sim_id + 't_' + str(178) + '_eta_vs_x.txt')
psi = data[:,0]
eta = data[:,1]*2.74101/3.04983e-10 # Ohm-m

fig, axs = plt.subplots(1,1,dpi=300,figsize=(2,1))
plt.plot(np.sqrt(psi),eta/const.mu_0)
plt.yscale('log')
plt.ylabel(r'$\eta/\mu_0$ [m2/s]$')
axs.set_xlabel(r'$\sqrt{\Psi_N}$ [-]$')
plt.ylim([10,1000])
axs.set_xlim([0,1])

# L/R Time
eta_mean = np.mean(eta[0:-50])
plt.plot([0,1],[eta_mean/const.mu_0]*2,'--',alpha=0.5)
R = eta_mean * 2 * np.pi * 1.85 / (np.pi * 0.57**2) # Ohm
print('R [Ohm] = %1.3g'%R)
```

R [Ohm] = 0.00164



```
In [279... # L = 0.5*(1.1 + 2) * 1e-6 # Total inductance [H]
L = 1.8e-6
tau_LR = L/R
print('L/R [ms] = %1.3g'%(tau_LR*1e3))

fig, axs = plt.subplots(1,1,dpi=300,figsize=(2,1))
plt.plot(time*1e3,ip/1e6)

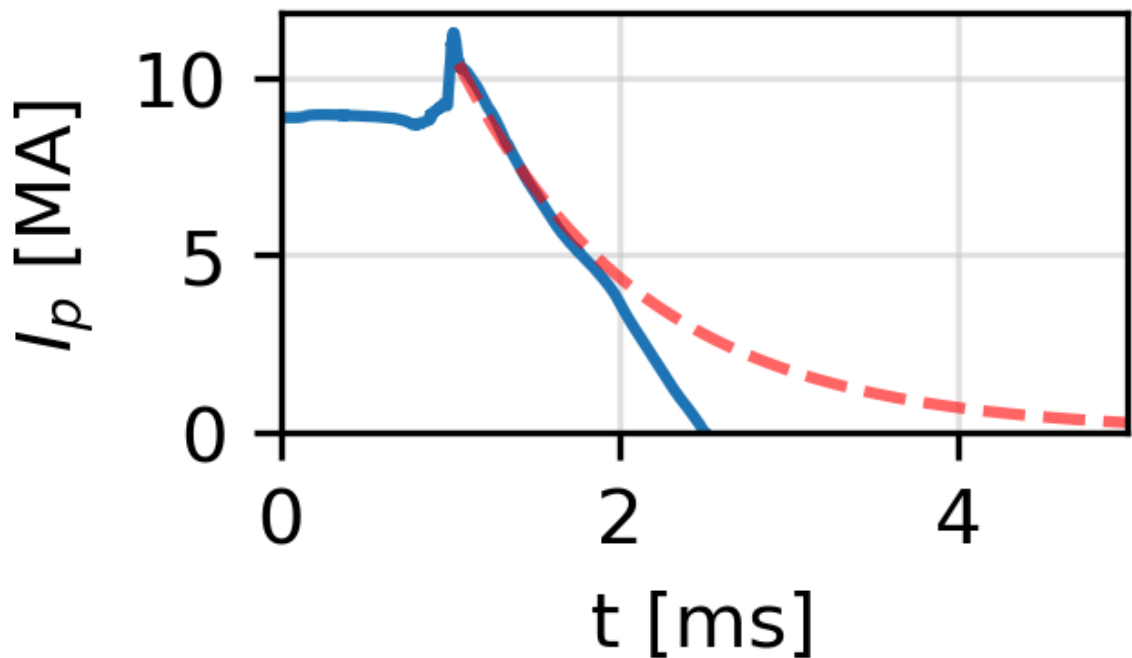
tt = np.linspace(0,5e-3,100)
ip[np.isnan(ip)] = 0
ii = 10.4e6 * np.exp(-tt/tau_LR)

plt.plot(tt*1e3 + 1.05,ii/1e6,'r--',alpha=0.6)
axs.set_xlim([0,5])
axs.set_ylim(ymin=0)

plt.ylabel('$I_p$ [MA]')
plt.xlabel('t [ms]')
axs.set_yticks([0,5,10])
axs.grid(alpha=0.33)
axs.set_xlabel('t [ms]')
```

L/R [ms] = 1.1

Out[279... Text(0.5, 0, 't [ms]')



In []:

MHD Activity

Toroidal spectrum of MHD Modes

```
In [201... filename = inDir + sim_id + 'C1.h5'
with h5py.File(filename, 'r') as f:
    # Loading both datasets
    data_matrix = f['keharmonics']['keharmonics'][:]
    data_matrix_me = f['bharmonics']['bharmonics'][:]

t = get_h5_dataset('scalars/time', filename = filename)
t_ms = t * m3dc1_units.unit["tau_A"] * np.sqrt(2) * 1e3

fig, ax_ke = plt.subplots(1, 1, figsize=(4, 2), dpi=400) # Half the height t
colors = cm.inferno(np.linspace(0.1, 0.9, 9))

for i in range(1, data_matrix.shape[1]-1):
    ls = '-' if (i % 2 == 1) else '--'
    if (i < 5):
        ax_ke.plot(t_ms[:data_matrix.shape[0]], data_matrix[:, i], label=f'r

# Formatting Top Plot (Kinetic Energy)
ax_ke.set_ylabel('Kinetic Energy [a.u.]', fontsize=9)
# ax_ke.legend(loc='upper right', fontsize=6, ncol=2)
ax_ke.grid(True, alpha=0.5)
```

```

ax_ke.set_ylim(ymin=1e-7, ymax=0.2)
ax_ke.set_yscale('log')
ax_ke.set_xlabel('time [ms]')
ax_ke.set_xlim(xmin=0, xmax=1.2)

# Lines
# ax_ke.plot([0.8]*2, [1e-8, 1e-1], '--', alpha=0.5)

# qmin
qmin, q95, t = [], [], []
tid = range(0, 259, 1)

data_time = np.loadtxt(inDir + sim_id + '/times_values.csv', delimiter=',')
from scipy.interpolate import interp1d
f = interp1d(data_time[:, 0], data_time[:, 1])

# tid = np.hstack((tid, 81))
for ii in tid:
    try:
        data = np.loadtxt(inDir + sim_id + '/q_profiles/q-' + str(ii) + '.txt')
        qmin.append(np.min(data[:, 1]))
        t.append(f(ii))

        fn = interp1d(data[:, 0], data[:, 1])
        q95.append(fn(0.950)) # q value at the 0.95 \psi_N

    except:
        continue

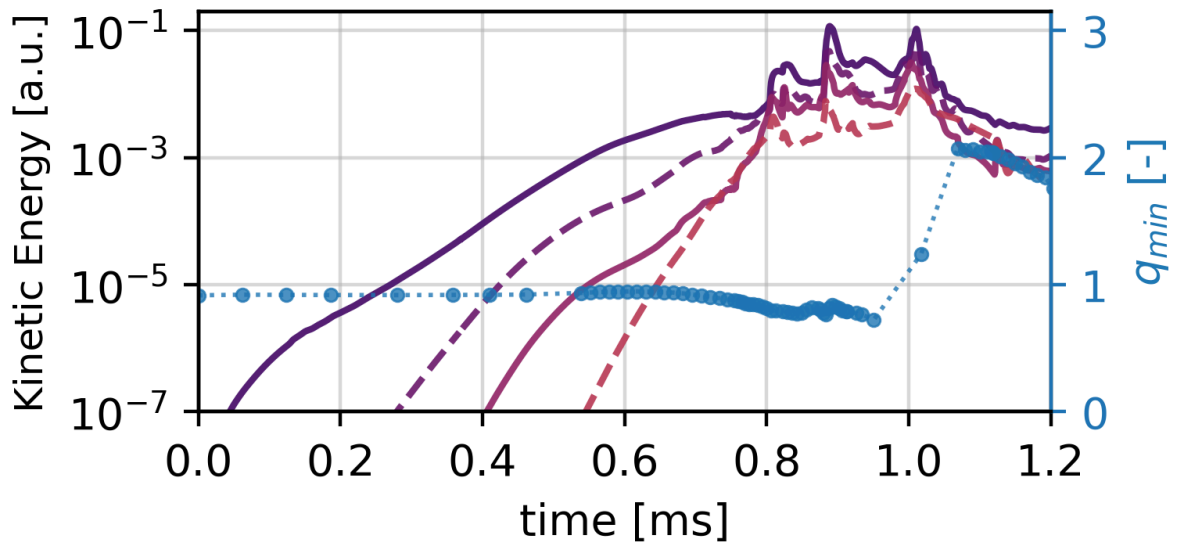
qmin = np.asarray(qmin)
q95 = np.asarray(q95)

ax2 = ax_ke.twinx()
ax2.plot(t, qmin, ':o', label='$q_{min}$', markersize=3, markeredgewidth=0.5, mark
ax2.set_ylim([0, 3.15])
ax2.set_yticks([0, 1, 2, 3])
ax2.set_ylabel('$q_{min}$ [-]')

ax2.spines['right'].set_color('tab:blue') # Colors the vertical spine
ax2.tick_params(axis='y', colors='tab:blue') # Colors the ticks and nu
ax2.set_ylabel('$q_{min}$ [-]', color='tab:blue') # Colors the axis label t

plt.tight_layout()
plt.show()

```



```
In [ ]: plotdelB = False

filename = inDir + sim_id + 'C1.h5'
with h5py.File(filename, 'r') as f:
    # Loading both datasets
    data_matrix = f['keharmonics']['keharmonics'][:]
    data_matrix_me = f['bharmonics']['bharmonics'][:]

t = get_h5_dataset('scalars/time', filename = filename)
t_ms = t * m3dc1_units.unit["tau_A"] * np.sqrt(2) * 1e3

# Setup figures conditionally based on plotdelB
if plotdelB:
    fig, axs = plt.subplots(2, 1, figsize=(3, 4), dpi=400, sharex=True)
    ax_ke = axs[0]
    ax_delB = axs[1]
else:
    fig, ax_ke = plt.subplots(1, 1, figsize=(3, 2), dpi=400) # Half the height

colors = cm.viridis(np.linspace(0.1, 0.9, 9))

for i in range(1, data_matrix.shape[1]-1):
    ls = '-' if (i % 2 == 1) else '--'

    # Kinetic Energy (Always plotted)
    if (i < 5):
        ax_ke.plot(t_ms[:data_matrix.shape[0]], data_matrix[:, i], label=f'r{i}',
                   color=colors[i], linestyle=ls)

    # Magnetic Harmonics (Only plotted if plotdelB is True)
    if plotdelB:
        delB = np.sqrt(data_matrix_me[:, i] / data_matrix_me[:, 0])
        ax_delB.plot(t_ms[:delB.shape[0]], delB, label=f'n = {i}', linestyle=ls,
                     color=colors[i])

# Formatting Top Plot (Kinetic Energy)
ax_ke.set_ylabel('Kinetic Energy [a.u.], fontsize=9)
# ax_ke.legend(loc='upper right', fontsize=6, ncol=2)
ax_ke.grid(True, alpha=0.5)
```



```

ax_ke.set_ylim(ymin=1e-7, ymax=0.2)
ax_ke.set_yscale('log')

# Formatting Bottom Plot & Shared Elements
if plotdelB:
    ax_delB.set_ylabel('$\\delta B/B_0$ [-]')
    ax_delB.set_xlabel('Time [ms]')
    ax_delB.grid(True, alpha=0.5)
    ax_delB.set_yscale('log')
    ax_delB.set_ylim([1e-10, 2e-1])
    ax_delB.set_xlim(xmin=0, xmax=1.2)
else:
    ax_ke.set_xlabel('time [ms]')
    ax_ke.set_xlim(xmin=0, xmax=1.2)

# Lines
# ax_ke.plot([0.8]*2, [1e-8, 1e-1], '--', alpha=0.5)

plt.tight_layout()
plt.show()

```

CQ MHD

```

In [52]: filename = inDir + sim_id + 'C1.h5'
with h5py.File(filename, 'r') as f:
    # Loading both datasets
    data_matrix = f['keharmonics']['keharmonics'][:]
    data_matrix_me = f['bharmonics']['bharmonics'][:]

t = get_h5_dataset('scalars/time', filename = filename)
t_ms = t * m3dc1_units.unit["tau_A"] * np.sqrt(2) * 1e3

fig, axs = plt.subplots(2, 1, figsize=(3, 4), dpi=400, sharex=True)

colors = cm.viridis(np.linspace(0.1, 0.9, 9))

for i in range(1, data_matrix.shape[1]-1):
    ls = '-' if (i % 2 == 1) else '--'

    # Row 1: Kinetic Energy
    axs[0].plot(t_ms[:data_matrix.shape[0]], data_matrix[:, i], label=f'n = {i}',
                linestyle=ls, color=colors[i])

    # Row 2: Magnetic Harmonics (bharmonics)
    delB = np.sqrt(data_matrix_me[:, i] / data_matrix_me[:, 0])
    axs[1].plot(t_ms[:delB.shape[0]], delB, label=f'n = {i}', linestyle=ls,
                color=colors[i])

# Formatting Top Plot
axs[0].set_ylabel('Kinetic Energy [a.u.]')
axs[0].legend(loc='upper right', fontsize=6, ncol=2)
axs[0].grid(True, alpha=0.5)
axs[0].set_ylim(ymin=1e-7, ymax=0.2)
axs[0].set_yscale('log')

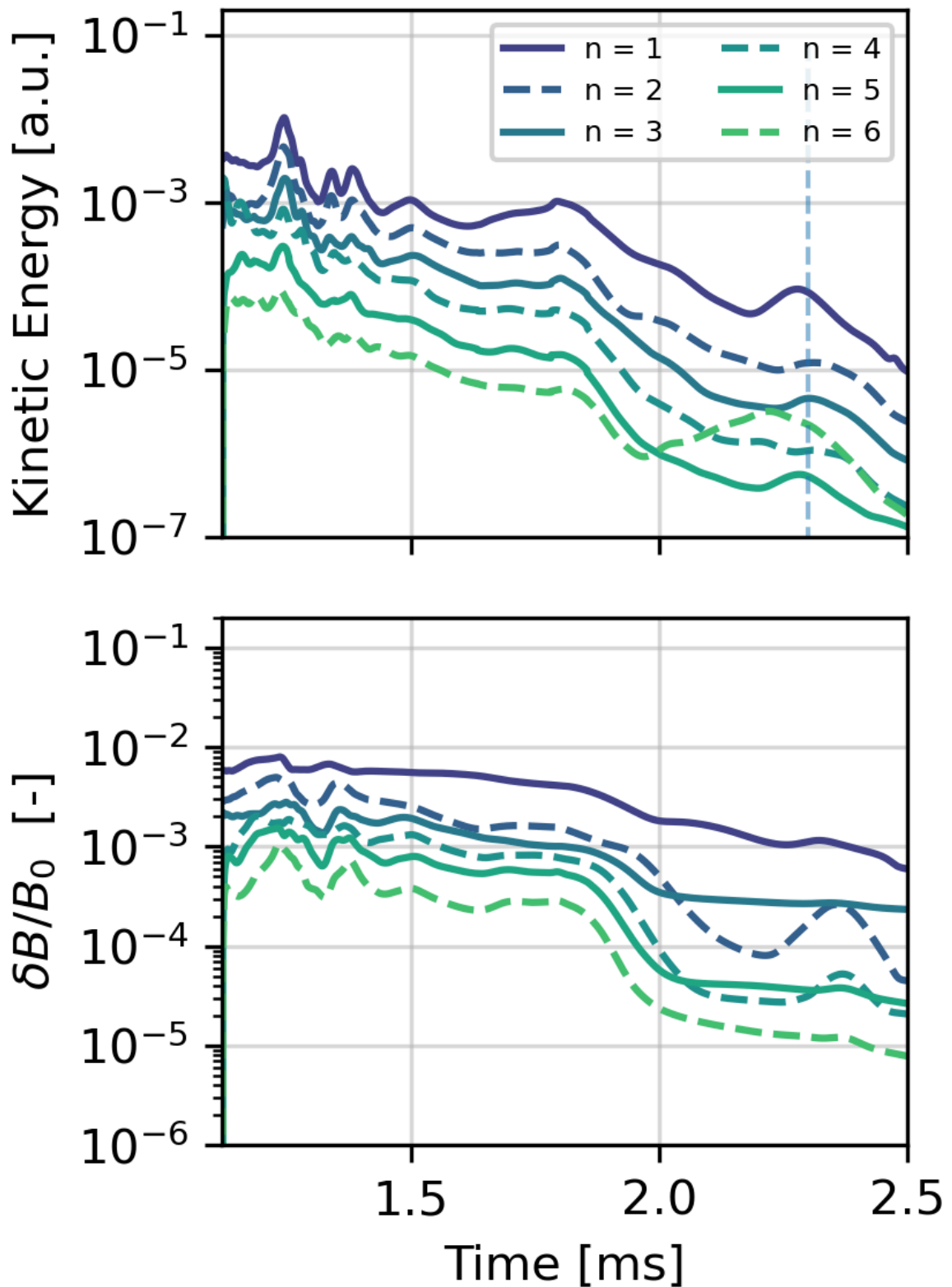
```

```
# Formatting Bottom Plot
axs[1].set_ylabel('$\\delta B/B_0$ [-]')
axs[1].set_xlabel('Time [ms]')
axs[1].grid(True, alpha=0.5)
axs[1].set_yscale('log')
axs[1].set_ylim([1e-6, 2e-1])

axs[-1].set_xlim(xmin=1.121, xmax=2.5)

axs[0].plot([2.3]*2, [1e-8, 1e-1], '--', alpha=0.5, linewidth=1)

plt.tight_layout()
plt.show()
```



```
In [143... filename = inDir + sim_id + 'C1.h5'
with h5py.File(filename, 'r') as f:
    # Loading both datasets
    data_matrix = f['keharmonics']['keharmonics'][(0)]
    data_matrix_me = f['bharmonics']['bharmonics'][(0)]

t = get_h5_dataset('scalars/time', filename = filename)
```

```

t_ms = t * m3dc1_units.unit["tau_A"] * np.sqrt(2) * 1e3

fig, ax_ke = plt.subplots(1, 1, figsize=(4, 2), dpi=400)

fac = fac = m3dc1_units.unit['B0']**2 / const.mu_0 * 1**3 # [J]

colors = cm.inferno(np.linspace(0.1, 0.9, 9))

for i in range(1, data_matrix.shape[1]-1):
    ls = '-' if (i % 2 == 1) else '--'

    ax_ke.plot(t_ms[:data_matrix.shape[0]], data_matrix[:, i]*fac/1e3, label=

# Formatting Top Plot (Kinetic Energy)
ax_ke.set_ylabel('Kinetic Energy [kJ]',fontsize=9)
# ax_ke.legend(loc='upper right', fontsize=6, ncol=2)
ax_ke.grid(True, alpha=0.5)
ax_ke.set_ylim(ymin=1e-4, ymax=2e2)
ax_ke.set_yscale('log')
ax_ke.set_xlabel('time [ms]')
ax_ke.set_xlim(xmin=0, xmax=2.5)
# ax_ke.set_xticks(np.arange(1.2,2.51,0.2))

# Lines
# ax_ke.plot([0.8]*2, [1e-8, 1e-1], '--',alpha=0.5)

# qmin
qmin, q95, t = [], [], []
tid = range(0,259,1)

data_time = np.loadtxt(inDir + sim_id + '/times_values.csv', delimiter=',')
from scipy.interpolate import interp1d
f = interp1d(data_time[:,0],data_time[:,1])

# tid = np.hstack((tid,81))
for ii in tid:
    try:
        data = np.loadtxt(inDir + sim_id + '/q_profiles/q-' + str(ii) + '.txt')
        qmin.append(np.min(data[2:,1]))
        t.append(f(ii))

        fn = interp1d(data[:,0],data[:,1])
        q95.append(fn(0.950)) # q value at the 0.95 |psi_N

    except:
        continue

qmin = np.asarray(qmin)
q95 = np.asarray(q95)

```

```

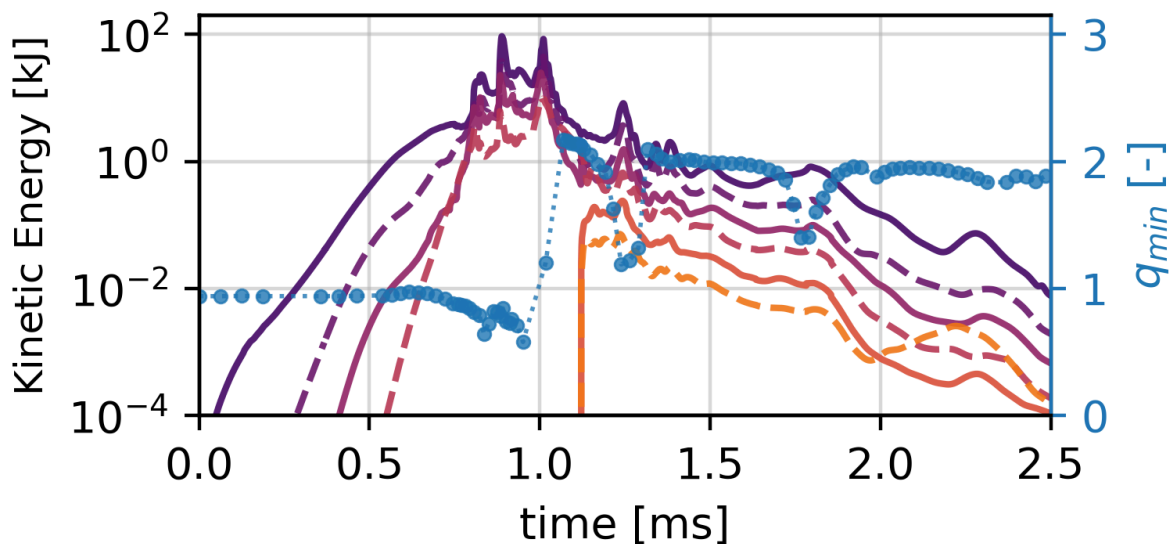
ax2 = ax_ke.twinx()
qmin[np.where(qmin>2.5)] = np.nan
ax2.plot(t,qmin,':o',label='$q_{min}$',markersize=3,markeredgewidth=0.5,mark
ax2.set_ylim([0,3.15])
ax2.set_yticks([0,1,2,3])
ax2.set_ylabel('$q_{min}$ [-]')

ax2.spines['right'].set_color('tab:blue')           # Colors the vertical spi
ax2.tick_params(axis='y', colors='tab:blue')       # Colors the ticks and nu
ax2.set_ylabel('$q_{min}$ [-]', color='tab:blue')  # Colors the axis label t

# lines
# plt.plot([1.12]*2,[0,3], '--k',alpha=0.5)

plt.tight_layout()
plt.show()

```



Change in safety Factor

```

In [66]: fig, axs = plt.subplots(1,1,figsize=(4,2.25),dpi=400)

data_time = np.loadtxt(inDir + sim_id + '/times_values.csv', delimiter=',')
from scipy.interpolate import interp1d
f = interp1d(data_time[:,0],data_time[:,1])

tid = np.array([152,158,168])

clrs = plt.cm.viridis(np.linspace(0.1,0.9,5))

for ii in range(tid.shape[0]):

    data = np.loadtxt(inDir + sim_id + '/q_profiles/q-' + str(tid[ii]) +
    psi = data[:,0]
    q = data[:,1]

```

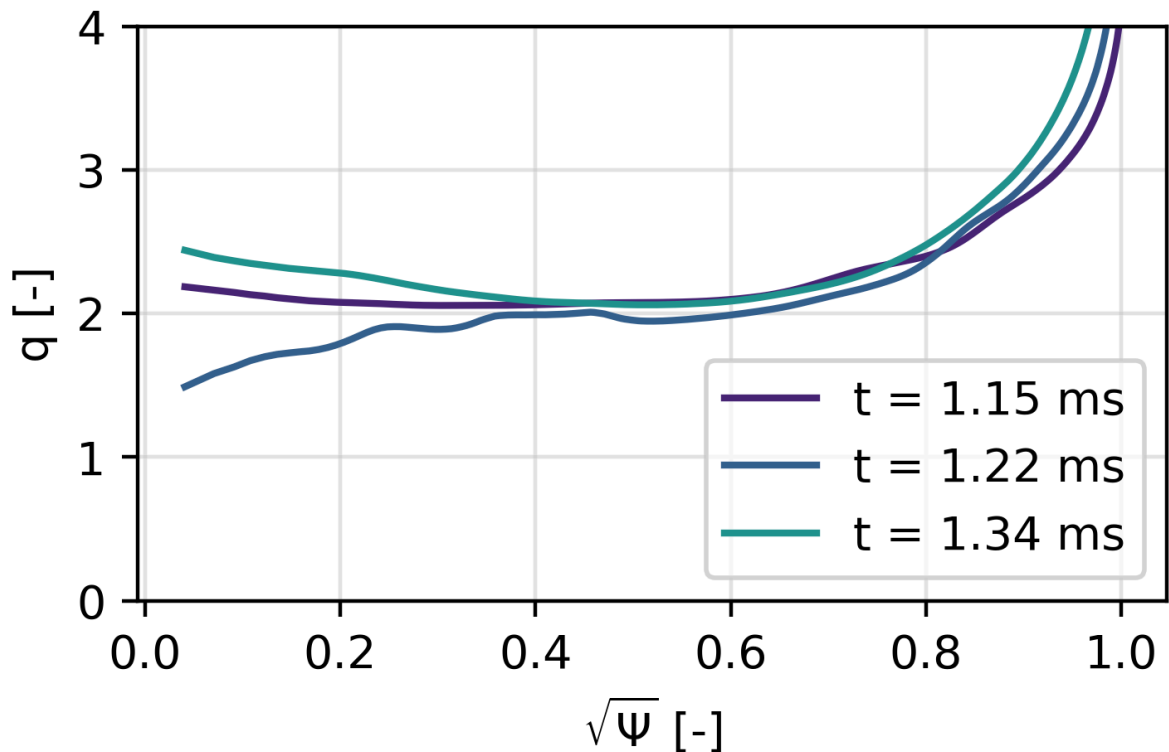
```

        axs.plot(np.sqrt(psi), q, '-',
                  color=clrs[ii],
                  label='t = ' + str(np.round(f(tid[ii]),2)) + ' ms')

axs.set_xlabel(r'$\sqrt{\Psi}$ [-]')
axs.set_ylabel('q [-]')
axs.legend()
axs.grid(alpha=0.33)
axs.set_ylim([0,4])

```

Out[66]: (0.0, 4.0)



Current Density and RE Density

```

In [461]: tids = np.array([152,160])

fig, axs = plt.subplots(1,1,dpi=300,figsize=(3,2))

for ii in range(tids.shape[0]):
    data = np.loadtxt(inDir + sim_id + 't_' + str(tids[ii]) + '_jy_plasma_vs_psi')
    psi = data[:,0]
    jy = data[:,1]

    axs.plot(np.sqrt(psi), jy/1e6, color=clrs[ii],
              label='t = ' + str(np.round(f(tid[ii]),2)) + ' ms')

    data = np.loadtxt(inDir + sim_id + 't_' + str(tids[ii]) + '_nre_vs_x.txt')
    psi = data[:,0]
    jre = data[:,1] * -1 * m3dc1_units.unit['j0']

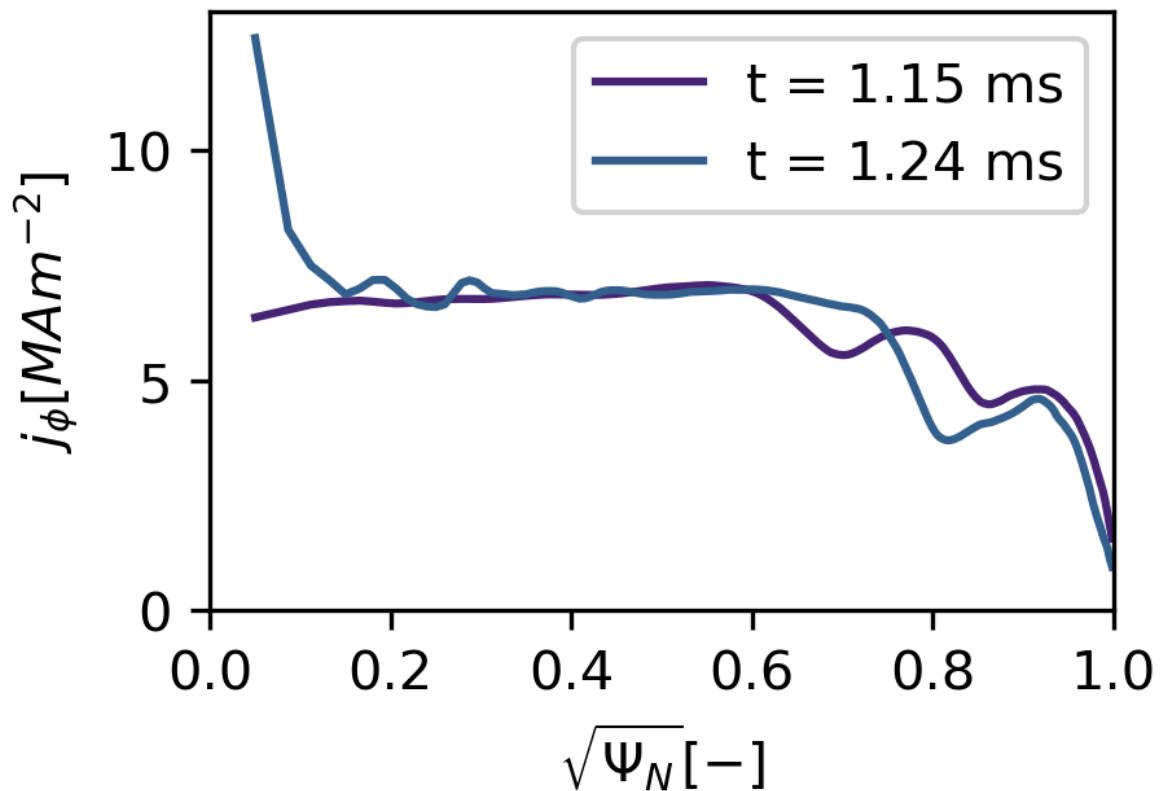
```

```
# axs.plot(np.sqrt(psi),jre/1e6,'--',color=clrs[ii])

axs.set_xlabel(r'\sqrt{\Psi_N} [-]$')
axs.set_ylabel(r'$j_{\phi}$ [MAm-2]$')

# axs.grid()
axs.set_ylim(ymin=0)
axs.set_xlim([0,1])
axs.legend()
```

Out[461]... <matplotlib.legend.Legend at 0x7f3c4ae90a50>



```
In [452]... tids = np.array([152,160])

fig, axs = plt.subplots(1,1,dpi=300,figsize=(3,2))

for ii in range(tids.shape[0]):

    data = np.loadtxt(inDir + sim_id + 't_' + str(tids[ii]) + '_nre_vs_x.txt')
    psi = data[:,0]
    jre = data[:,1] * -1 * m3dc1_units.unit['j0']

    axs.plot(np.sqrt(psi),jre/1e6,'--',color=clrs[ii])

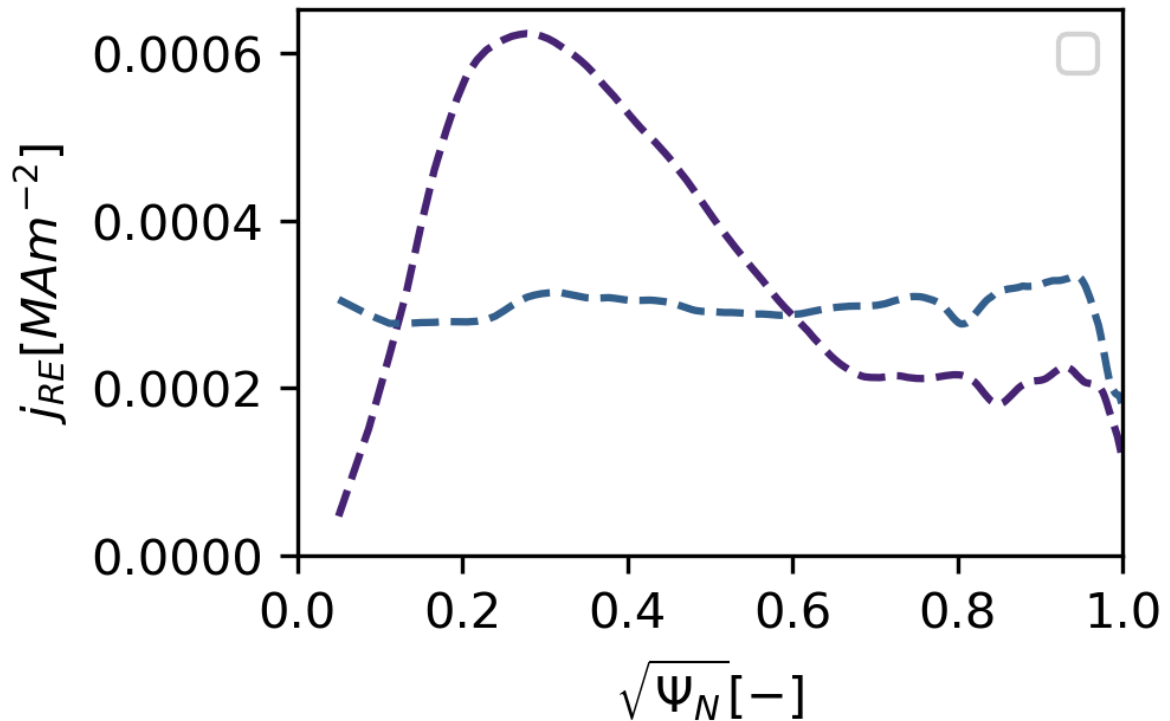
    axs.set_xlabel(r'\sqrt{\Psi_N} [-]$')
    axs.set_ylabel(r'$j_{\phi}$ [MAm-2]$')
```

```
# axs.grid()
axs.set_ylim(ymin=0)
axs.set_xlim([0,1])
axs.legend()
```

/tmp/ipykernel_437005/2144408007.py:20: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

```
axs.legend()
```

Out[452... <matplotlib.legend.Legend at 0x7f3c32244b90>



```
In [457... tids = np.array([152,160])

fig, axs = plt.subplots(1,1,dpi=300,figsize=(3,2))

for ii in range(tids.shape[0]):
    data = np.loadtxt(inDir + sim_id + 't_' + str(tids[ii]) + '_eta_vs_x.txt')
    psi = data[:,0]
    eta = data[:,1]

    axs.plot(np.sqrt(psi),eta,color=clrs[ii],
             label='t = ' + str(np.round(f(tid[ii]),2)) + ' ms')

    axs.set_xlabel(r'$\sqrt{\Psi_N}$ [-]$')
    axs.set_ylabel(r'$\eta$')

# axs.grid()
# axs.set_ylim(ymin=0)
axs.set_yscale('log')
```

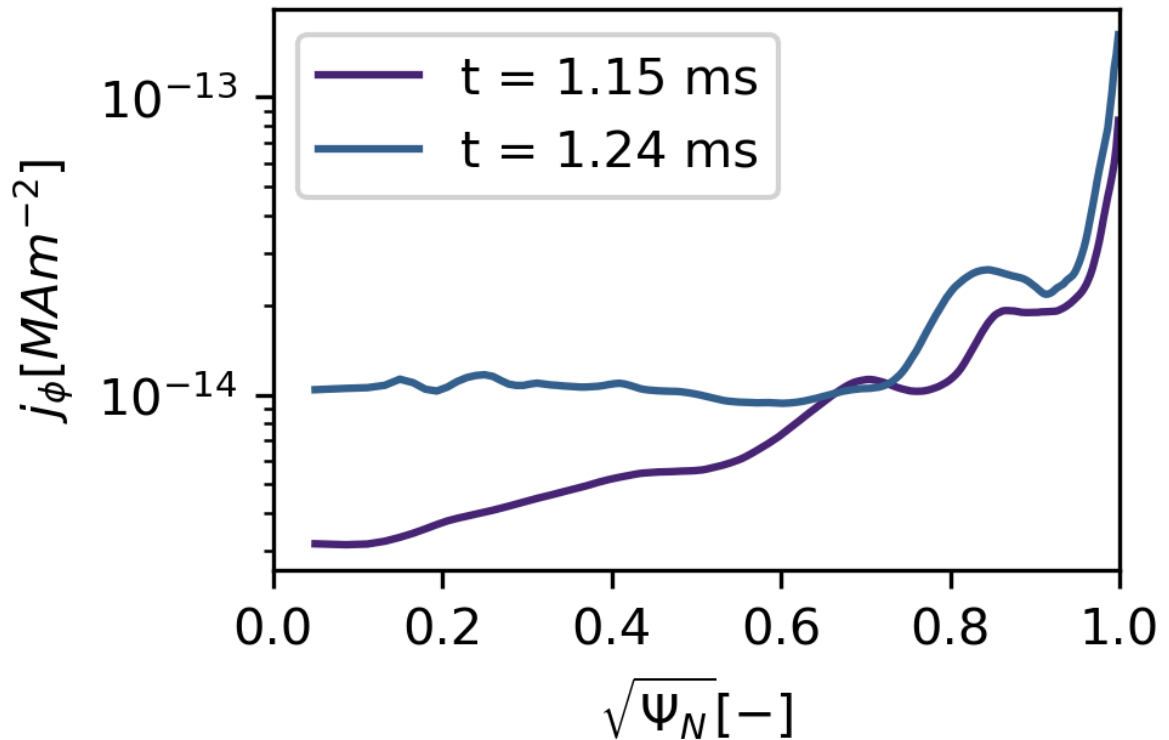


```

axs.set_xlim([0,1])
axs.legend()

```

Out[457... <matplotlib.legend.Legend at 0x7f3c43eb7b10>



In [454...]

Out[454... 0.5511798647017148

Safety Factor vs. Time

```

In [50]: fig, axs = plt.subplots(1,1,figsize=(4,2.25),dpi=400)
         clr = plt.cm.viridis(np.linspace(0.1,0.9,70))
         qmin, q95, t = [], [], []
         tid = range(0,259,1)

         data_time = np.loadtxt(inDir + sim_id + '/times_values.csv', delimiter=',')
         from scipy.interpolate import interp1d
         f = interp1d(data_time[:,0],data_time[:,1])

         # tid = np.hstack((tid,81))
         for ii in tid:
             try:
                 data = np.loadtxt(inDir + sim_id + '/q_profiles/q-' + str(ii) + '.txt')
                 qmin.append(np.min(data[2:,1]))
                 t.append(f(ii))

                 fn = interp1d(data[:,0],data[:,1])
                 q95.append(fn(0.950)) # q value at the 0.95 \psi_N

```

```

except:
    continue

qmin = np.asarray(qmin)
q95 = np.asarray(q95)

plt.plot(t, qmin, ':o', label='$q_{min}$', markersize=3, markeredgewidth=0.5, marker
plt.plot(t, q95, ':s', label='$q_{95}$', markersize=3, markeredgewidth=0.5, marker
plt.ylim([0.5, 10])
plt.xlim(xmin=0)
# plt.yticks(np.arange(0.8, 1.3, 0.1))
plt.grid(alpha=0.2)
plt.xlabel('t [ms]')
plt.ylabel('$q_{min}$ [-]')
plt.legend(loc='lower left')

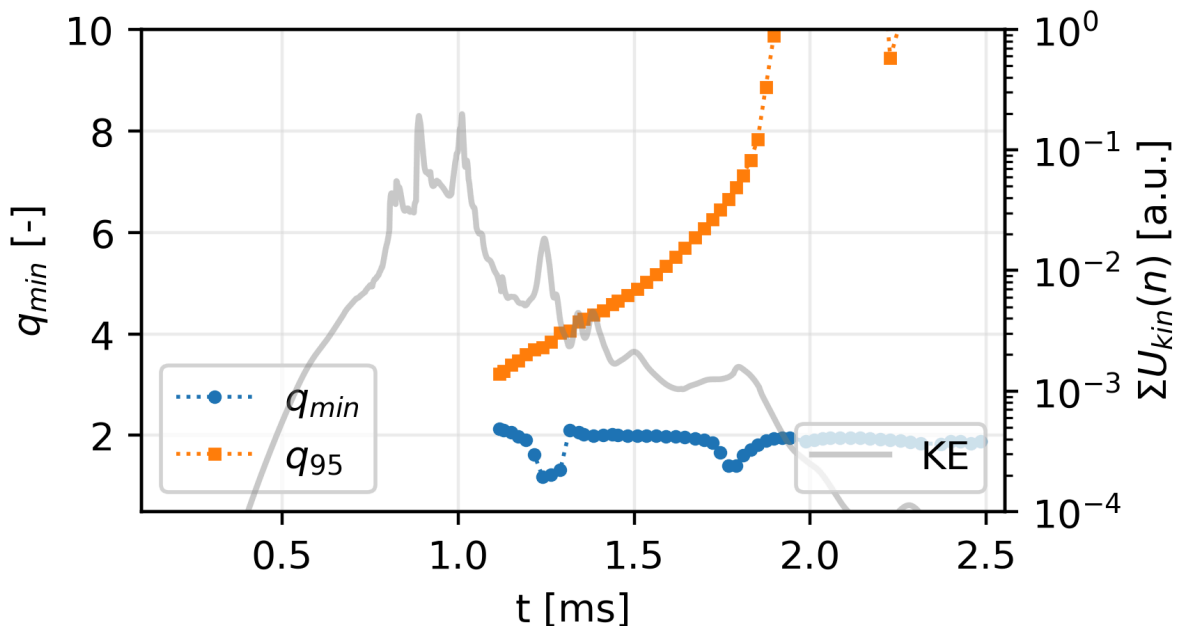
ax2 = plt.twinx()

ke = np.sum(data_matrix[:, 1:], 1)

ax2.plot(t_ms, ke, label='KE', color='tab:grey', alpha=0.4)
ax2.set_ylabel('$\\Sigma U_{kin}(n)$ [a.u.]')
ax2.set_ylim([1e-4, 1])
ax2.set_yscale('log')
ax2.legend(loc='lower right')
ax2.set_xlim(xmin=0.1)

```

Out[50]: (0.1, 2.55293)



Safety factor during re-healing

```

In [115... fig, axs = plt.subplots(1,1,figsize=(3.5,2.5),dpi=400)

data_time = np.loadtxt(inDir + sim_id + '/times_values.csv', delimiter=',')
from scipy.interpolate import interp1d
f = interp1d(data_time[:,0],data_time[:,1])

tid = np.array([152,158,166,228,238,254])

lines = ['-','--',':','-',':', '--']

clrs = plt.cm.inferno(np.linspace(0.1,0.9,6))

for ii in range(tid.shape[0]):

    data = np.loadtxt(inDir + sim_id + '/q_profiles/q-' + str(tid[ii]) +
    psi = data[:,0]
    q = data[:,1]
    axs.plot(np.sqrt(psi[1:]),q[1:],lines[ii],
              color=clrs[ii],
              label='t = ' + str(np.round(f(tid[ii]),2)) + ' ms')

axs.set_xlabel(r'$\sqrt{\Psi}$ [-]')
axs.set_ylabel('q [-]')
axs.legend(fontsize=8,ncol=2)
# axs.grid(alpha=0.33)
axs.set_ylim([0,4])
axs.set_xlim([0,1])

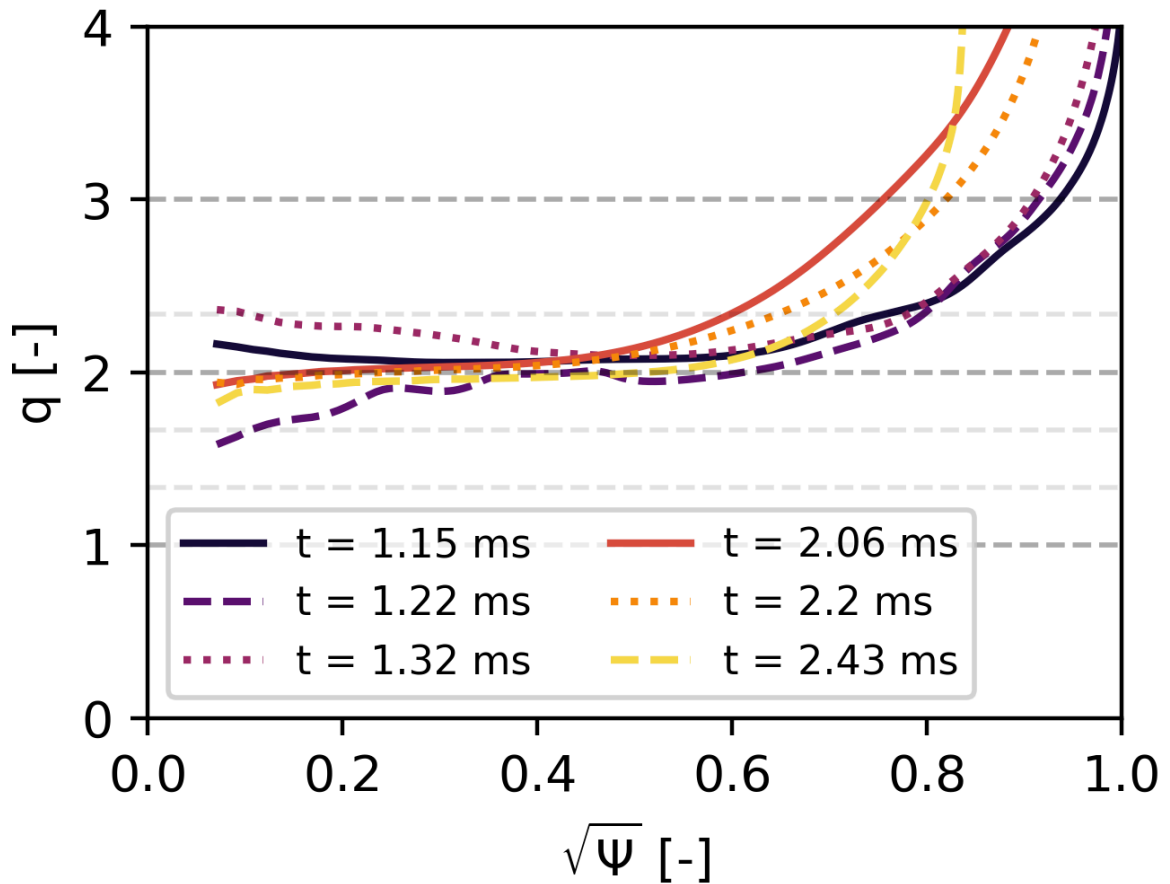
axs.plot([0,1],[1]*2,'k--',alpha=0.33,linewidth=1)
axs.plot([0,1],[4/3]*2,'k--',alpha=0.1,linewidth=1)
axs.plot([0,1],[5/3]*2,'k--',alpha=0.1,linewidth=1)
axs.plot([0,1],[2]*2,'k--',alpha=0.33,linewidth=1)
axs.plot([0,1],[7/3]*2,'k--',alpha=0.1,linewidth=1)
axs.plot([0,1],[3]*2,'k--',alpha=0.33,linewidth=1)

```

```

Out[115... [<matplotlib.lines.Line2D at 0x7f209ccfd6d0>]

```



```
In [36]: q.shape
```

```
Out[36]: (50,)
```

```
In [ ]:
```

```
# 2-D
```

```
In [348]: ip = get_h5_dataset('scalars/toroidal_current_p', filename=inDir + 'REMC/TQ/SPRC-m8-Ne-REMC-3D-01-Copy1.h5')
t = get_h5_dataset('scalars/time', filename=inDir + 'REMC/TQ/SPRC-m8-Ne-REMC-3D-01-Copy1.h5')

ip = ip * m3dc1_units.unit["j0"]
t = t * m3dc1_units.unit["tau_A"] * np.sqrt(2)

fig, axs = plt.subplots(1, 1, figsize=(2, 1.5), dpi=400)
print('I_p [MA] = %1.7g'%(ip[-1]/1e6))
plt.plot(t*1e3, ip/1e6, '-', linewidth=1.5)
plt.ylim([0, 2])

re = get_h5_dataset('scalars/runaways', filename=inDir + 'REMC/TQ/SPRC-m8-Ne-REMC-3D-01-Copy1.h5')
time = get_h5_dataset('scalars/time', filename=inDir + 'REMC/TQ/SPRC-m8-Ne-REMC-3D-01-Copy1.h5')

re = re * m3dc1_units.unit["j0"]
time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2)
```

```

axs.plot(time*1e3,re/1e6,'-',alpha=1,linewidth=1.5)

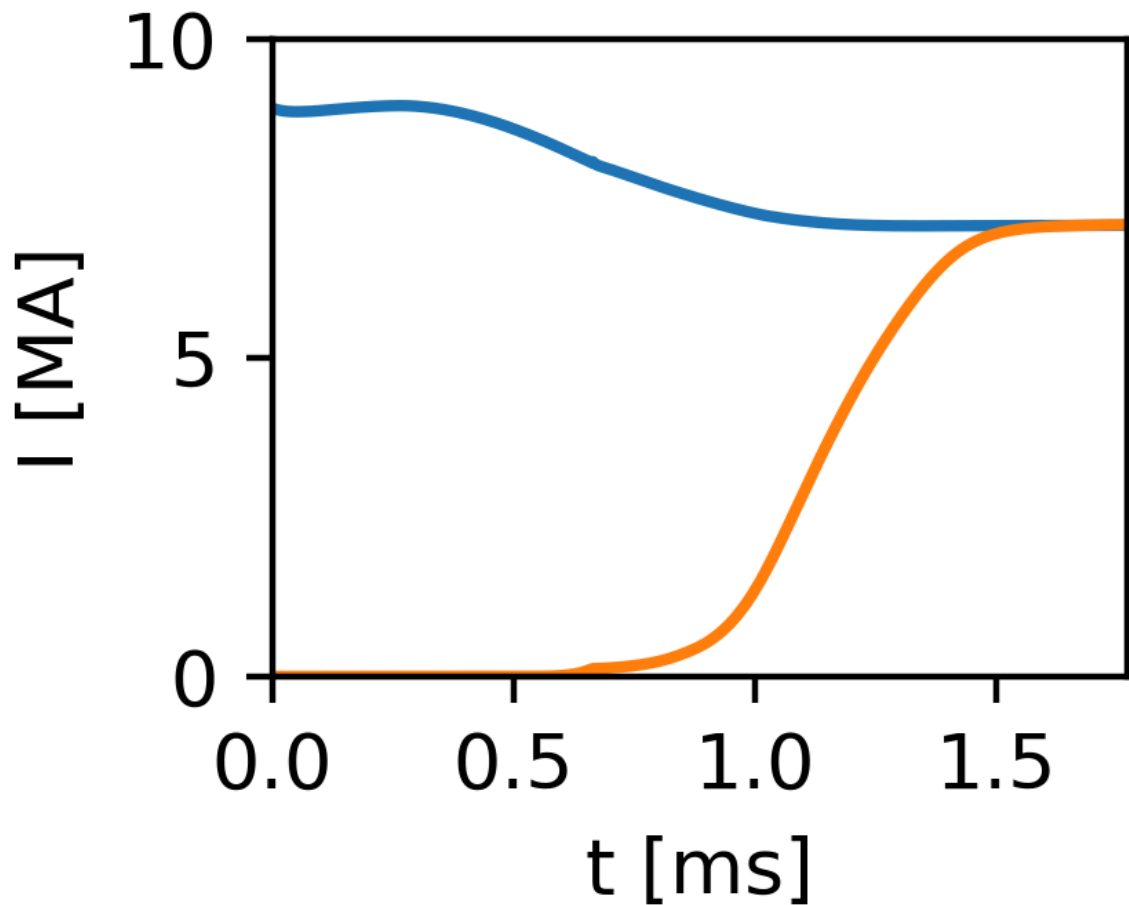
# axs.grid(alpha=0.5)
axs.set_xlabel('t [ms]')
axs.set_ylabel('I [MA]')
axs.set_xlim([0,1.77])
axs.set_yticks([0,5,10])
axs.set_xticks([0,0.5,1,1.5])
# axs.set_xticks(np.arange(1,2.5,0.1))

# axs.plot([0,2],[7]*2,'--',alpha=0.5)

fig.savefig('current.eps', format='eps', transparent=True, facecolor='none',

```

I_p [MA] = 7.076956



In [284... *# Max Temp. vs time*

```

temax = get_h5_dataset('scalars/temax',filename=inDir + 'REMC/TQ/SPRC-m8-Ne-
time = get_h5_dataset('scalars/time',filename=inDir + 'REMC/TQ/SPRC-m8-Ne-RE

temax = temax * m3dc1_units.unit["T0"] - 7.5
time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2)

```

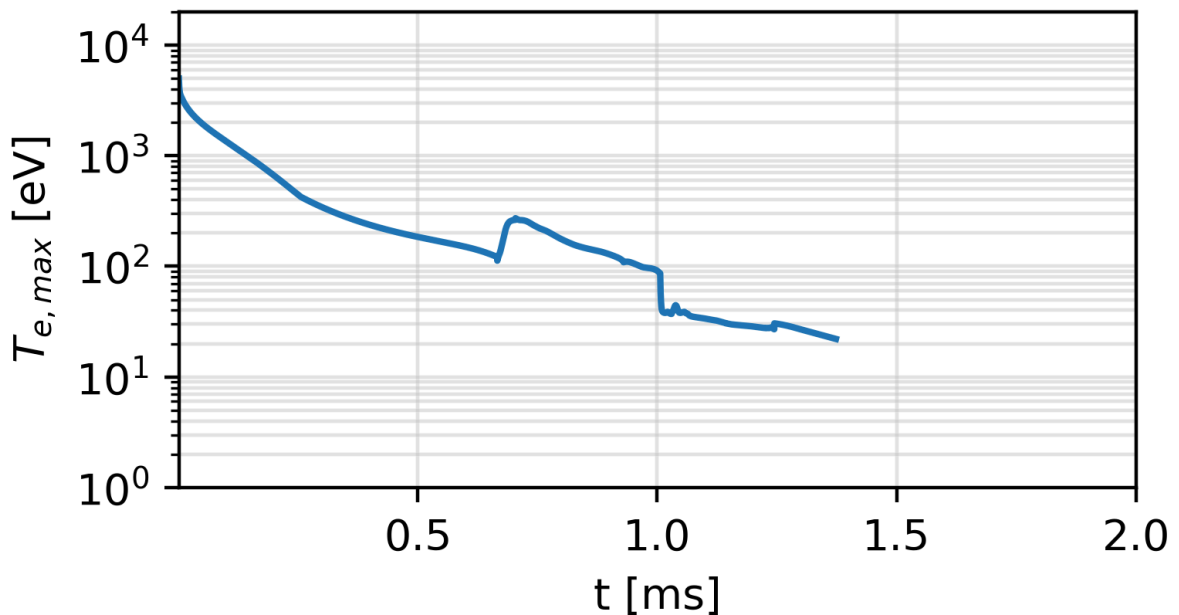
```

fig, axs = plt.subplots(1,1,figsize=(4,2),dpi=400)
plt.plot(time*1e3,temax)
plt.xlabel('t [ms]')
plt.ylabel('$T_{e,max}$ [eV]')
plt.yscale('log')
# plt.xscale('log')
plt.ylim([1,20e3])
# axs.set_xlim(xmin=0.0,xmax=2.5)
plt.grid(which='both',alpha=0.33)
axs.set_xlim([0.001,2])

# Lines
# plt.plot([0.8]*2,[1,1e4])

```

Out[284... (0.001, 2.0)



Fusion-IO

```

In [7]: # Fusion-IO
# 1. BIND THE MISSING SCIPY FUNCTION TO PREVENT THE IMPORT ERROR
import scipy.integrate
if not hasattr(scipy.integrate, 'trapz'):
    scipy.integrate.trapz = scipy.integrate.trapezoid
if not hasattr(scipy.integrate, 'cumtrapz'):
    scipy.integrate.cumtrapz = scipy.integrate.cumulative_trapezoid

fio_lib = '/global/homes/r/rdatta/fusion-io/install/lib/'
os.environ['LD_LIBRARY_PATH'] = f"{fio_lib}:{os.environ.get('LD_LIBRARY_PATH', '')}"

sys.path.append('/global/homes/r/rdatta/M3DC1/unstructured/python/')

# This is the line that needed to change:

```

```
sys.path.append(fio_lib)
```

```
import m3dc1
import fpy
```

Warning: Cannot import plot_vector_field. This is probably because mayavi is not installed. But do not worry, plot_vector_field is not required for the rest of the m3dc1 package.

```
In [8]: from m3dc1.get_time_of_slice import get_time_of_slice

def poin_plot(sim=None,time=None,
              Rlim=[None,None],Zlim=[None,None],
              ax=None,inDir=None,sim_id=None):

    # Set font sizes and plot style parameters
    if False:
        axlblfs = 20
        ticklblfs = 18
        titlefs = 18
        linewidth = 1
    else:
        axlblfs = 12
        ticklblfs = 12
        titlefs = None
        linewidth = 1

    #Change to correct directory if time is specified
    cwd = os.getcwd()
    change_dir=False
    if time is not None:
        os.chdir(inDir + sim_id + 'TRACE/TS'+str(time).zfill(3))
    outfiles = sorted(glob.glob(os.getcwd()+"/out*"))

    print('Reading ' + os.getcwd())

    for of in outfiles:
        # Skip the image file if it gets picked up
        if of.endswith('.png'):
            continue
        if os.path.getsize(of) > 0:
            x,y = np.loadtxt(of, usecols=(1,2), unpack=True)
            # ax.plot(x,y,lw=0,marker='.',ms=0.8)
            ax.scatter(x,y,s=0.1,marker=',')
        #else:
        #     print(of + 'is empty')

    ax.grid(True,alpha=0.4)
    ax.set_aspect('equal',adjustable='box')
    ax.set_xlim(Rlim[0],Rlim[1])
    ax.set_ylim(Zlim[0],Zlim[1])

    unitlabel = 'ms'
    figtitle = 'time='+"{0:.2f}".format(get_time_of_slice(int(time),filename
```

```
ax.set_title(figtitle,fontsize=titlefs)
fig.tight_layout()

os.chdir(cwd)
return fig
```

```
In [9]: # FIO Load (Run once)
sim_id = 'REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/'
time_slice = 0
sim = fpy.sim_data(inDir + sim_id + 'C1.h5',time=time_slice)
print('Fields Available:')
print(sim.available_fields)

# Load the mesh
sim, time, mesh_ob, R, phi_list, Z, R_mesh, Z_mesh, R_ave, Z_ave, field1_ave
```

Fields Available:

```
{'j': ('current density', 'vector', None, 'simple'), 'ni': ('density', 'scalar', 'main ion', 'simple'), 'ne': ('density', 'scalar', 'electron', 'simple'), 'v': ('fluid velocity', 'vector', None, 'simple'), 'B': ('magnetic field', 'vector', None, 'simple'), 'p': ('total pressure', 'scalar', None, 'simple'), 'pi': ('pressure', 'scalar', 'main ion', 'simple'), 'pe': ('pressure', 'scalar', 'electron', 'simple'), 'alpha': ('alpha', 'scalar', None, 'simple'), 'ti': ('temperature', 'scalar', 'main ion', 'simple'), 'te': ('temperature', 'scalar', 'electron', 'simple'), 'A': ('vector potential', 'vector', None, 'simple'), 'gradA': ('grad vector potential', 'tensor', None, 'simple'), 'E': ('electric field', 'vector', None, 'simple'), 'psi': ('psi', 'scalar', None, 'composite'), 'kprad_rad': ('total radiation', 'scalar', None, 'simple')}
```

Output version: 45

Mesh shape: (148764, 10)

Evaluating field 'psi'...

period = 6.28319

nperiods = 1

Reading adjacency data.

Max neighbors = 5

[DONE]

Poincare Plots

```
In [82]: tid = 172

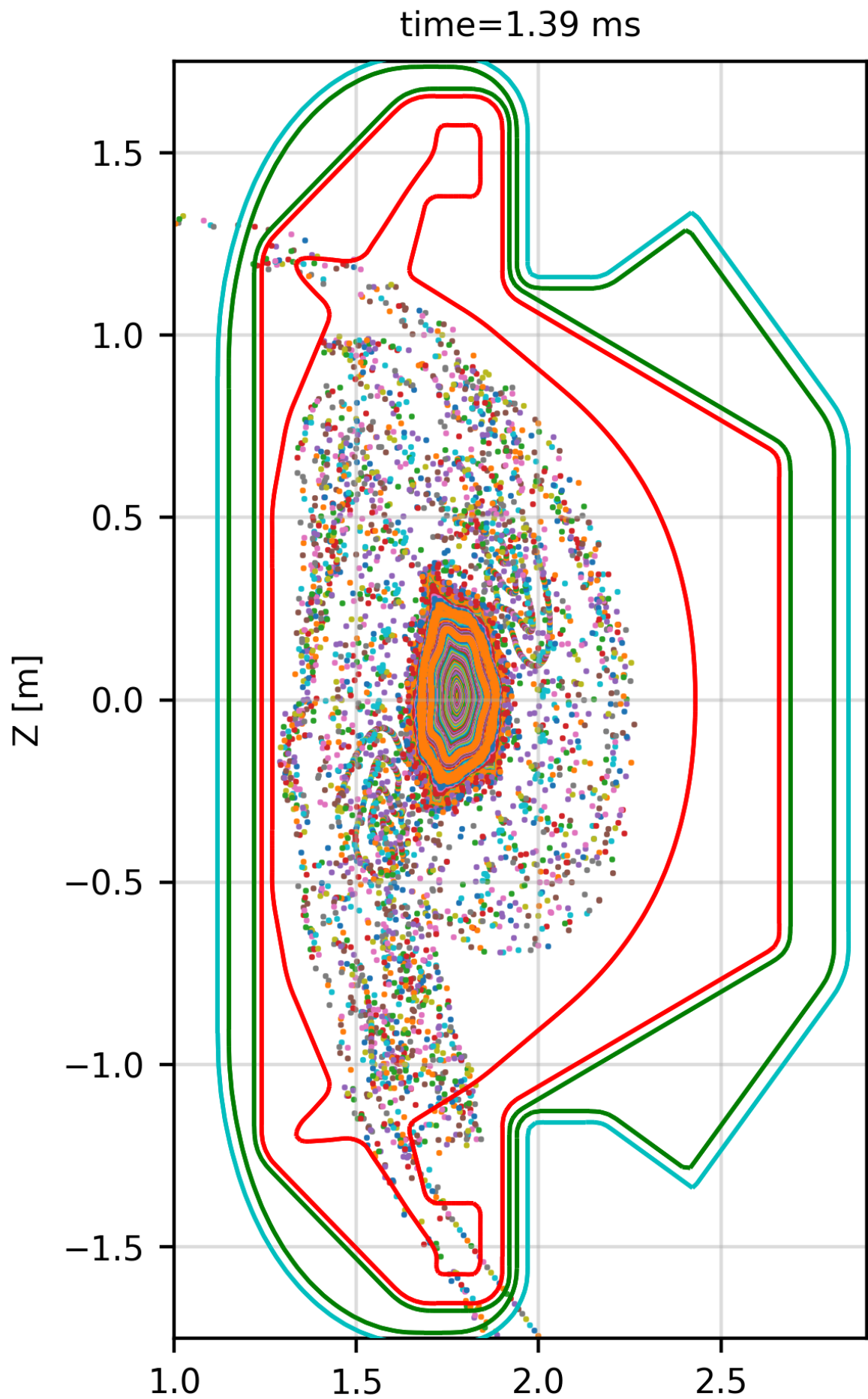
fig, axs = plt.subplots(1,1,dpi=400)

fig = poin_plot(sim=sim,time=tid,ax=axs,inDir=inDir,sim_id=sim_id) # Poincaré
# Boundary
```



```
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS172
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
```

https://jupyter.nersc.gov/user/rdatta/perlmutter-login-node-base/lab/tree/global/homes/r/rdatta/m3dc1_vizualization/REMC-TQ-CQ-SPRC-m8-Ne-REMC-3D-01-C... 37/86



R [m]

```

In [41]: # tids = np.array([13,40,50,52,
#                          55,63,100,147]) # TQ

tids = np.array([155,172,177,195,
                203,210,217,228,
                238,242,250,254]) # CQ

plot_bound = True
plot_flux = False
levels = [-3.2,-1.6,-0.8,-0.2,0,0.1,0.2,0.4]

# fig, axs = plt.subplots(1,tids.shape[0],dpi=400,figsize=(12,6),sharey=True)
fig, axs = plt.subplots(3,int(tids.shape[0]/3),dpi=600,figsize=(6,7.5),share
axs = axs.flatten()

for ii in range(tids.shape[0]):
    poin_plot(sim=sim,time=tids[ii],ax=axs[ii],inDir=inDir,sim_id=sim_id)
    axs[ii].set_xlim([1,3])
    axs[ii].set_ylim([-1.8,1.8])
    axs[ii].text(2.2,-1.7,str(tids[ii]),color='k',alpha=0.33)
    if (ii / 4) >= 2:
        axs[ii].set_xlabel('R [m]')
    if (ii % 4) == 0:
        axs[ii].set_ylabel('Z [m]')

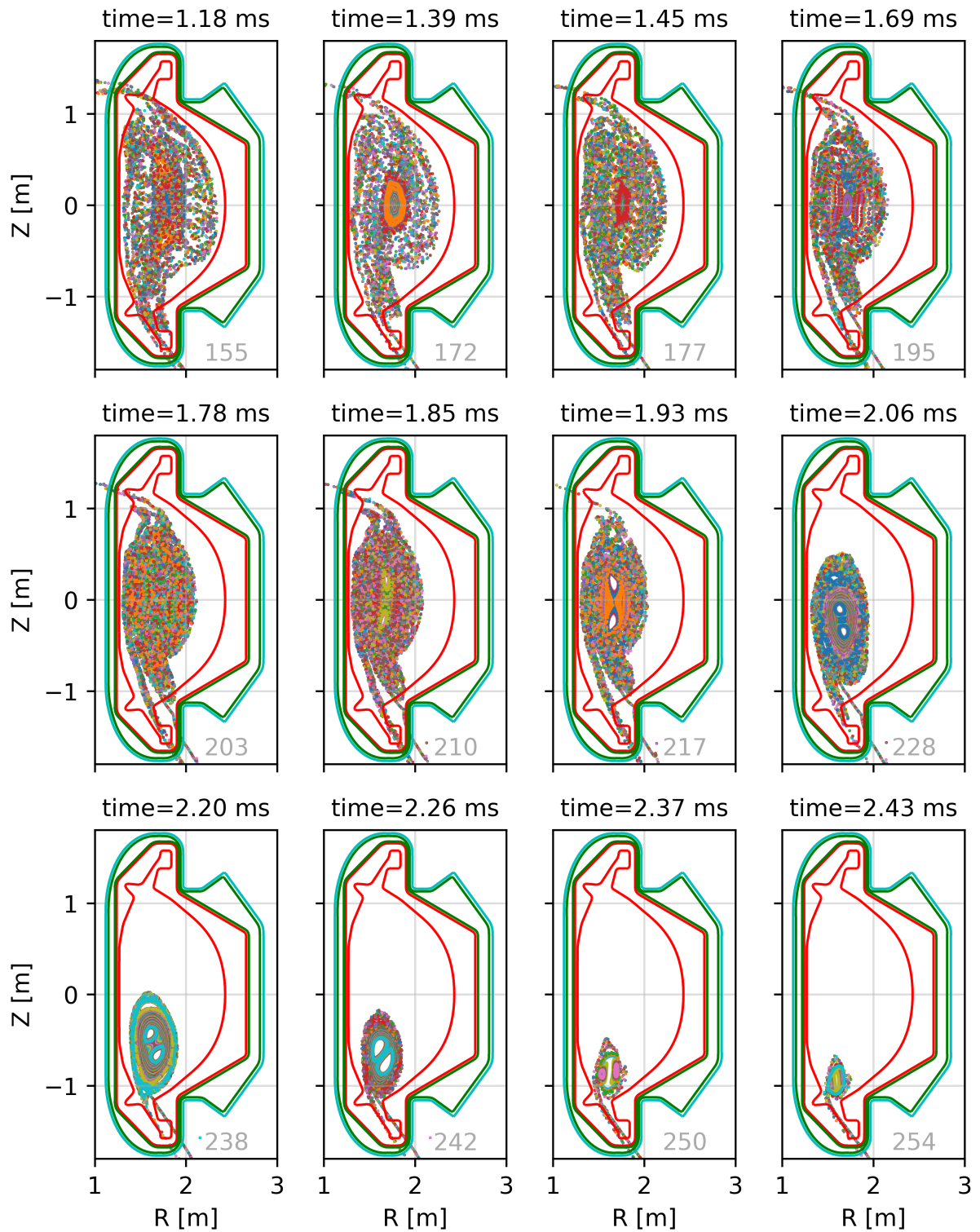
    if plot_bound:
        m3dc1.plot_mesh(mesh_ob,boundary=True,ax=axs[ii],meshcol='C1',filena

    if plot_flux:
        sim, time, mesh_ob, R, phi_list, Z, R_mesh, Z_mesh, R_ave, Z_ave, fi

    axs[ii].contour(R_ave,Z_ave,np.squeeze(field1_ave), levels=levels, c

```

Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS155
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS172
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS177
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS195
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS203
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS210
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS217
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS228
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS238
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS242
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS250
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS254
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields



```
In [374... tids = np.array([155,
                        172,203])

# tids = np.array([228,
#                  238,250])

plot_bound = True
```

```

plot_flux = True
levels = [-3.2,-1.6,-0.8,-0.2,0,0.1,0.2,0.4]

fig, axs = plt.subplots(1,tids.shape[0],dpi=400,figsize=(6.5,4),sharey=True)
# fig, axs = plt.subplots(2,int(tids.shape[0]/2),dpi=600,figsize=(6,6),share
axs = axs.flatten()

for ii in range(tids.shape[0]):
    poin_plot(sim=sim,time=tids[ii],ax=axs[ii],inDir=inDir,sim_id=sim_id)
    axs[ii].set_xlim([1,3])
    axs[ii].set_xticks([1,2,3])
    axs[ii].set_ylim([-1.8,1.8])
    # Text
    axs[ii].text(2.5,-1.7,str(tids[ii]),color='k',alpha=0.2)
    tval = np.round(m3dc1.get_time_of_slice(int(tids[ii]),filename=inDir+sim
    axs[ii].text(2.1,1.5,str(tval) + ' ms',color='k',fontsize=12)

    # axs[ii].set_title('t = ' + str(tval) + ' ms')

    axs[ii].set_xlabel('R [m]')
    if (ii % 5) == 0:
        axs[ii].set_ylabel('Z [m]')

    if plot_bound:
        m3dc1.plot_mesh(mesh_ob,boundary=True,ax=axs[ii],meshcol='C1',filena

    if plot_flux:
        sim, time, mesh_ob, R, phi_list, Z, R_mesh, Z_mesh, R_ave, Z_ave, fi

        axs[ii].contour(R_ave,Z_ave,np.squeeze(field1_ave), levels=levels, c

fig.subplots_adjust(wspace=0.055) # Adjust the float value to your liking

```

```

Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS155
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Output version: 45
Mesh shape: (260337, 10)
Evaluating field 'psi'...

```

```

period = 6.28319
nperiods = 1
Reading adjacency data.
Max neighbors = 5

```

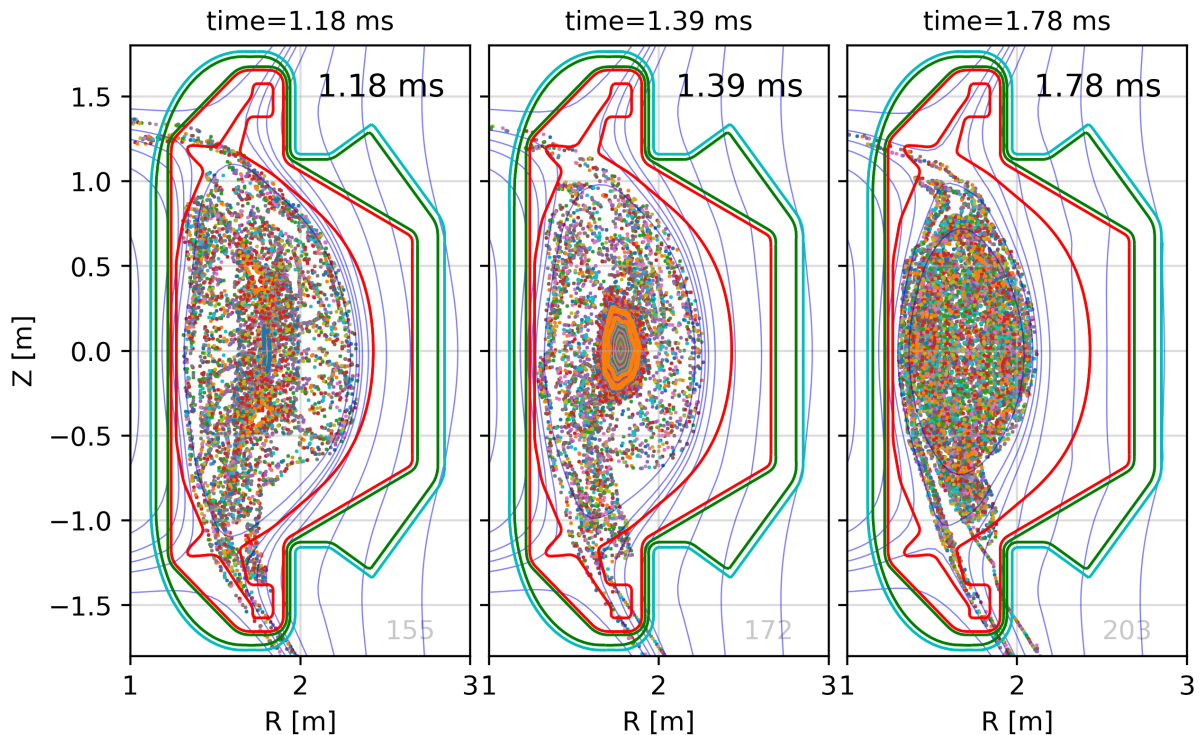
```
[DONE]
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS172
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Output version: 45
Mesh shape: (260337, 10)
Evaluating field 'psi'...
```

```
period = 6.28319
nperiods = 1
Reading adjacency data.
Max neighbors = 5
```

```
[DONE]
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS203
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Output version: 45
Mesh shape: (260337, 10)
Evaluating field 'psi'...
```

```
period = 6.28319
nperiods = 1
Reading adjacency data.
Max neighbors = 5
```

```
[DONE]
deleting simulation object and closing 0 fields
```



VDE


```

period = 6.28319
nperiods = 1
Reading adjacency data.
Max neighbors = 5
[DONE]
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS238
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Output version: 45
Mesh shape: (260337, 10)
Evaluating field 'psi'...

```

```

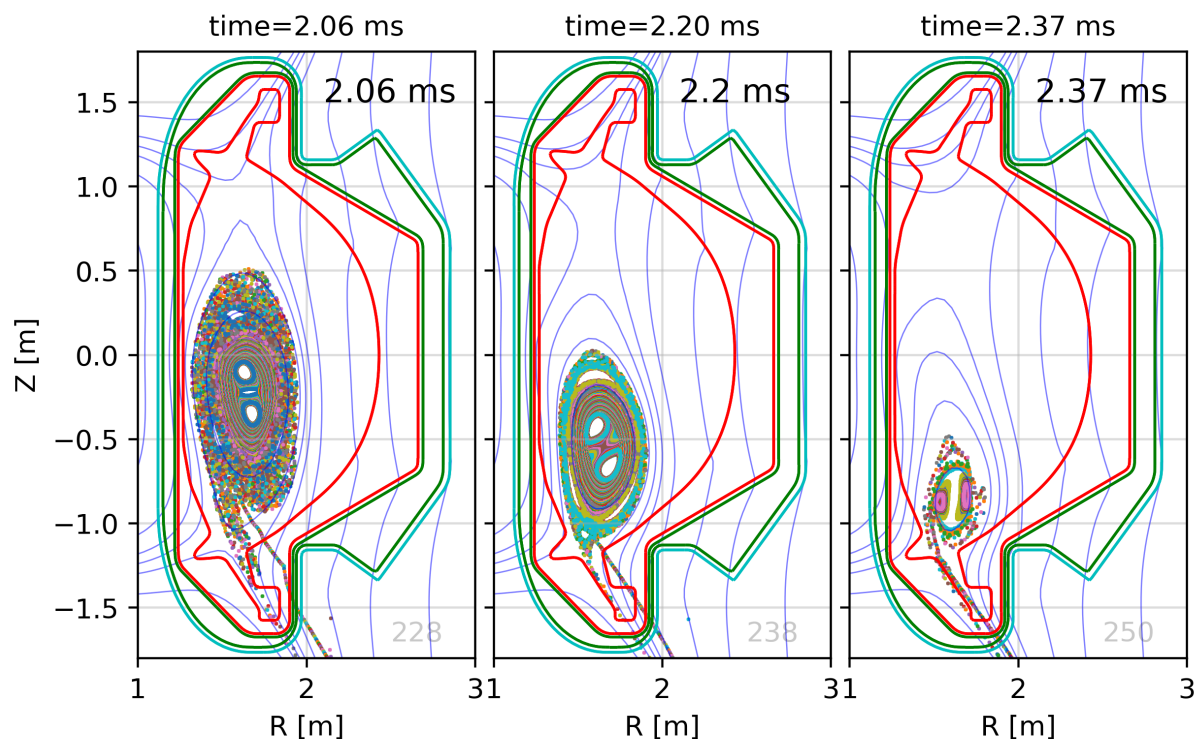
period = 6.28319
nperiods = 1
Reading adjacency data.
Max neighbors = 5
[DONE]
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS250
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Output version: 45
Mesh shape: (260337, 10)
Evaluating field 'psi'...

```

```

period = 6.28319
nperiods = 1
Reading adjacency data.
Max neighbors = 5
[DONE]
deleting simulation object and closing 0 fields

```



Save as GIF

```
In [392... # Save as GIF

from PIL import Image
import imageio
import os

tids_TQ = np.array([13,40,50,52,
                    55,63,100,147]) # TQ

tids_CQ = np.array([155,172,177,195,
                    203,210,217,228,
                    238,240,242,247,250,254,258]) # CQ

tids = np.hstack([tids_TQ,tids_CQ])

plot_bound = True
plot_flux = False
levels = [-3.2,-1.6,-0.8,-0.2,0,0.1,0.2,0.4]

# Temp folder for frames
frame_dir = inDir + sim_id + '/frames_tmp/'
os.makedirs(frame_dir, exist_ok=True)
frames = []

for ii, tid in enumerate(tids):
    t = tids[ii]

    try:
        fig, axs = plt.subplots(1, 1, dpi=400)

        if tids[ii] <= 147:
            sim_id_now = "REMC/TQ/SPRC-m8-Ne-REMC-3D-01/"
        else:
            sim_id_now = "REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/"

        poin_plot(sim=sim,time=tids[ii],ax=axs,inDir=inDir,sim_id=sim_id_now)
        axs.set_xlim([1,2.9])
        axs.set_ylim([-1.8,1.8])
        axs.set_xlabel('R [m]')
        axs.set_ylabel('Z [m]')
        axs.text(2.5,-1.7,str(tids[ii]),color='k',alpha=0.33)

        if plot_bound:
            m3dc1.plot_mesh(mesh_ob,boundary=True,ax=axs,meshcol='C1',filena

        frame_path = frame_dir + f"frame_{ii:03d}.png"
        fig.savefig(frame_path, bbox_inches="tight")
        plt.close(fig)

        frames.append(frame_path)
    except:
```

```
        continue

frames_images = [Image.open(f) for f in frames]
gif_path = os.getcwd() + "/GIFs/" + sim_id.replace("/", "-") + "poincare.gif"

frames_images[0].save(
    gif_path,
    format='GIF',
    append_images=frames_images[1:],
    save_all=True,
    duration=750, # duration in milliseconds
    loop=0
)

print(f"GIF saved to {gif_path}")
```

Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-m8-Ne-REMC-3D-01/TRACE/TS013
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-m8-Ne-REMC-3D-01/TRACE/TS040
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-m8-Ne-REMC-3D-01/TRACE/TS050
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-m8-Ne-REMC-3D-01/TRACE/TS052
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-m8-Ne-REMC-3D-01/TRACE/TS055
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-m8-Ne-REMC-3D-01/TRACE/TS063
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-m8-Ne-REMC-3D-01/TRACE/TS100
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-m8-Ne-REMC-3D-01/TRACE/TS147
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS155
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS172
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS177
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS195
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS203
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS210
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS217
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS228
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS238
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS240
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS242
deleting simulation object and closing 0 fields

```

deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS247
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS250
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS254
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS258
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
GIF saved to /global/u1/r/rdatta/m3dc1_vizualization/GIFs/REMC-TQ-SPRC-CQ-m8-Ne-REMC-3D-01-poincare.gif

```

RE Strike Locations

```

In [11]: # tids = np.array([228,238,242,250,254])
tids = np.array([172,238])

fig, axs = plt.subplots(1,tids.shape[0],dpi=400,figsize=(4.5,4),sharey=True)

#### Load the limiter ###
data = np.loadtxt('sparc_v3_first_wall_202303-smooth.dat',skiprows=1)
Rlim, Zlim = data[:,0], data[:,1]
# axs.plot(Rlim, Zlim,'--g')

from shapely.geometry import Polygon

# 1. Create a Polygon object from your Rlim, Zlim arrays
contour_coords = np.column_stack((Rlim, Zlim))
poly = Polygon(contour_coords)

# 2. Shrink the polygon by a specific distance (e.g., 0.02 meters)
# Passing a negative value to .buffer() shrinks it inwards
shrunk_poly = poly.buffer(-0.01)

# 3. Extract the new R and Z arrays to use in your interpolation/plotting
Rlim = np.array(shrunk_poly.exterior.coords.xy[0])
Zlim = np.array(shrunk_poly.exterior.coords.xy[1])

#####

for ii in range(tids.shape[0]):

    # Load
    tid = tids[ii]
    R, Z, jre = load_jre(tid)

```

```

im = axs[ii].imshow(jre/1e6, cmap='inferno', origin='lower', extent=[R[0],
                        vmin=1e-5,
                        vmax=1,
                        norm='log'
                    ])

axs[ii].set_xlabel('R [m]')
axs[ii].set_ylim([-1.8, 1.8])
axs[ii].set_xlim([1.0, 3.0])
axs[ii].set_facecolor('k')

# LCFS
lcfs = np.loadtxt(inDir + sim_id + '/idl_output/' + 'lcfs-' + str(tid) +
axs[ii].plot(lcfs[:,0], lcfs[:,1], 'w', linewidth=0.33, alpha=1)

# Bound
m3dc1.plot_mesh(mesh_ob, boundary=True, ax=axs[ii], meshcol='C1', filename=i

# if (ii == tids.shape[0]-1):
#     fig.colorbar(im, ax=axs, fraction=0.025, pad=0.04, label='$j_{RE}$

tval = np.round(get_time_of_slice(int(tid), filename=inDir+sim_id+'C1.h5'
# axs[ii].set_title('t = ' + str(tval) + ' ms')
# axs[ii].grid(color='w', alpha=0.25)
axs[ii].text(2.1, 1.5, str(tval) + ' ms', color='w', alpha=1, zorder=11, fonts

# Strike locations
# axs[ii].plot(Rlim, Zlim, '--g')
intersect_R, intersect_Z, strike_jre = find_contour_intersections(jre.T/
axs[ii].plot(intersect_R, intersect_Z, 's', color='lime', markersize=0.5

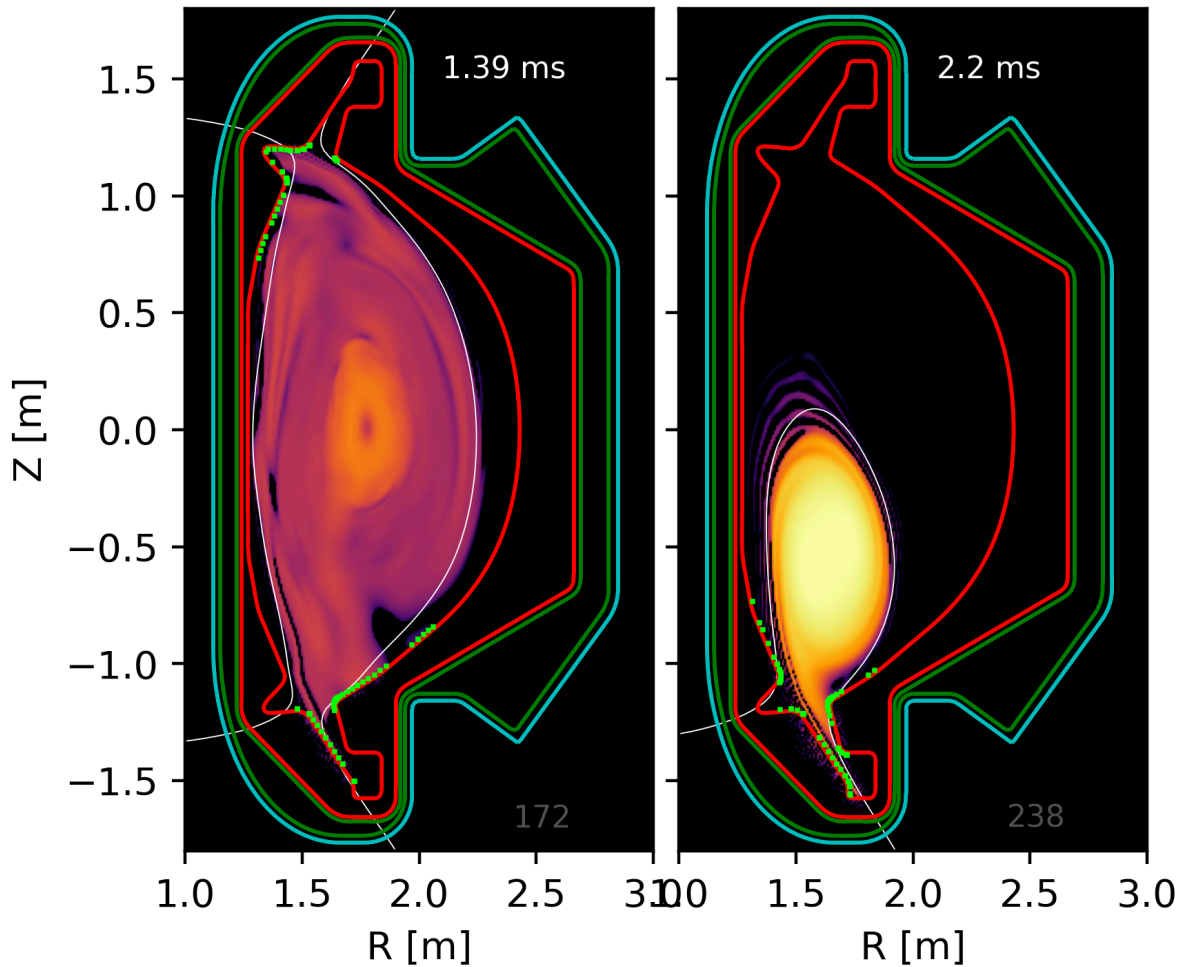
axs[ii].text(2.4, -1.7, str(tids[ii]), color='w', alpha=0.33, zorder=11, fonts

axs[0].set_ylabel('Z [m]')
fig.subplots_adjust(wspace=0.055)

# fig.tight_layout(pad=0.2)

```

deleting simulation object and closing 0 fields
 deleting simulation object and closing 0 fields
 deleting simulation object and closing 0 fields
 deleting simulation object and closing 0 fields



Strikes vs toroidal position

```
In [ ]: tid = 172
angles = np.array([0,60,90,120,180,240,270])

fig, axs = plt.subplots(1,angles.shape[0],dpi=400,figsize=(8.5,4),sharey=True)

#### Load the limiter ###
data = np.loadtxt('sparc_v3_first_wall_202303-smooth.dat',skiprows=1)
Rlim, Zlim = data[:,0], data[:,1]
# axs.plot(Rlim, Zlim,'--g')

from shapely.geometry import Polygon

# 1. Create a Polygon object from your Rlim, Zlim arrays
contour_coords = np.column_stack((Rlim, Zlim))
poly = Polygon(contour_coords)

# 2. Shrink the polygon by a specific distance (e.g., 0.02 meters)
# Passing a negative value to .buffer() shrinks it inwards
shrunk_poly = poly.buffer(-0.01)

# 3. Extract the new R and Z arrays to use in your interpolation/plotting
Rlim = np.array(shrunk_poly.exterior.coords.xy[0])
Zlim = np.array(shrunk_poly.exterior.coords.xy[1])
```

```
#####

for ii in range(angles.shape[0]):
    R, Z, jre = load_jre(tid, angles[ii])
    im = axs[ii].imshow(jre/1e6, cmap='inferno', origin='lower', extent=[R[0],
                                vmin=1e-4,
                                vmax=1,
                                norm='log'])

    axs[ii].set_xlabel('R [m]')
    axs[ii].set_ylim([-1.8, 1.8])
    axs[ii].set_xlim([1.0, 3.0])
    axs[ii].set_facecolor('k')

    # LCFS
    lcfs = np.loadtxt(inDir + sim_id + '/idl_output/' + 'lcfs-' + str(tid) +
    axs[ii].plot(lcfs[:,0], lcfs[:,1], 'w', linewidth=0.33, alpha=1)

    # Bound
    m3dc1.plot_mesh(mesh_ob, boundary=True, ax=axs[ii], meshcol='C1', filename=i

    if (ii == tids.shape[0]-1):
        fig.colorbar(im, ax=axs, fraction=0.015, pad=0.04, label='$j_{RE}$ I

    tval = np.round(get_time_of_slice(int(tid), filename=inDir+sim_id+'C1.h5'
    axs[ii].set_title('$\\phi$ = ' + str(angles[ii]) + '$\\degree$')

    # Strike locations
    intersect_R, intersect_Z, strike_jre = find_contour_intersections(jre.T/
    axs[ii].plot(intersect_R, intersect_Z, 's', color='lime', markersize=0.2

    axs[ii].text(2.4, -1.7, str(tid), color='w', alpha=0.33, zorder=11, fontsize=8

axs[0].set_ylabel('Z [m]')
```

TQ

In [12]: # FIO Load (Run once)

```
time_slice = 0
sim_id = 'REMC/TQ/SPRC-m8-Ne-REMC-3D-01/'
sim = fpy.sim_data(inDir + sim_id + 'C1.h5', time=time_slice)
print('Fields Available:')
print(sim.available_fields)

# Load the mesh
sim, time, mesh_ob, R, phi_list, Z, R_mesh, Z_mesh, R_ave, Z_ave, field1_ave
```


Fields Available:

```
{'j': ('current density', 'vector', None, 'simple'), 'ni': ('density', 'scalar', 'main ion', 'simple'), 'ne': ('density', 'scalar', 'electron', 'simple'), 'v': ('fluid velocity', 'vector', None, 'simple'), 'B': ('magnetic field', 'vector', None, 'simple'), 'p': ('total pressure', 'scalar', None, 'simple'), 'pi': ('pressure', 'scalar', 'main ion', 'simple'), 'pe': ('pressure', 'scalar', 'electron', 'simple'), 'alpha': ('alpha', 'scalar', None, 'simple'), 'ti': ('temperature', 'scalar', 'main ion', 'simple'), 'te': ('temperature', 'scalar', 'electron', 'simple'), 'A': ('vector potential', 'vector', None, 'simple'), 'gradA': ('grad vector potential', 'tensor', None, 'simple'), 'E': ('electric field', 'vector', None, 'simple'), 'psi': ('psi', 'scalar', None, 'composite'), 'kprad_rad': ('total radiation', 'scalar', None, 'simple')}
```

Output version: 45

Mesh shape: (148764, 10)

Evaluating field 'psi'...

```
period = 6.28319
nperiods = 1
Reading adjacency data.
Max neighbors = 5
```

[DONE]

deleting simulation object and closing 0 fields

Poincare Plots (TQ)

```
In [368... # tids = np.array([13,40,50,52,
#                  55,63,100,147]) # TQ

tids = np.array([40,
                55,147]) # TQ

plot_bound = True
plot_flux = True
levels = [-3.2,-1.6,-0.8,-0.2,0,0.1,0.2,0.4]

fig, axs = plt.subplots(1,tids.shape[0],dpi=400,figsize=(6.5,4),sharey=True)
# fig, axs = plt.subplots(2,int(tids.shape[0]/2),dpi=600,figsize=(6,6),share
axs = axs.flatten()

for ii in range(tids.shape[0]):
    poin_plot(sim=sim,time=tids[ii],ax=axs[ii],inDir=inDir,sim_id=sim_id)
    axs[ii].set_xlim([1,3])
    axs[ii].set_xticks([1,2,3])
    axs[ii].set_ylim([-1.8,1.8])
    # Text
    axs[ii].text(2.5,-1.7,str(tids[ii]),color='k',alpha=0.2)
    tval = np.round(m3dc1.get_time_of_slice(int(tids[ii]),filename=inDir+sim
    axs[ii].text(2.1,1.5,str(tval) + ' ms',color='k',fontsize=12)
```

```

# axs[ii].set_title('t = ' + str(tval) + ' ms')

axs[ii].set_xlabel('R [m]')
if (ii % 5) == 0:
    axs[ii].set_ylabel('Z [m]')

if plot_bound:
    m3dc1.plot_mesh(mesh_ob,boundary=True,ax=axs[ii],meshcol='C1',filena

if plot_flux:
    sim, time, mesh_ob, R, phi_list, Z, R_mesh, Z_mesh, R_ave, Z_ave, fi

    axs[ii].contour(R_ave,Z_ave,np.squeeze(field1_ave), levels=levels, c

fig.subplots_adjust(wspace=0.055) # Adjust the float value to your liking

```

Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-m8-Ne-REMC-3D-01/TRACE/TS040

deleting simulation object and closing 0 fields

deleting simulation object and closing 0 fields

deleting simulation object and closing 0 fields

Output version: 45

Mesh shape: (148764, 10)

Evaluating field 'psi'...

period = 6.28319

nperiods = 1

Reading adjacency data.

Max neighbors = 5

[DONE]

deleting simulation object and closing 0 fields

Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-m8-Ne-REMC-3D-01/TRACE/TS055

deleting simulation object and closing 0 fields

deleting simulation object and closing 0 fields

deleting simulation object and closing 0 fields

Output version: 45

Mesh shape: (148764, 10)

Evaluating field 'psi'...

period = 6.28319

nperiods = 1

Reading adjacency data.

Max neighbors = 5

[DONE]

deleting simulation object and closing 0 fields

Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-m8-Ne-REMC-3D-01/TRACE/TS147

deleting simulation object and closing 0 fields

deleting simulation object and closing 0 fields

deleting simulation object and closing 0 fields

Output version: 45

Mesh shape: (148764, 10)

Evaluating field 'psi'...

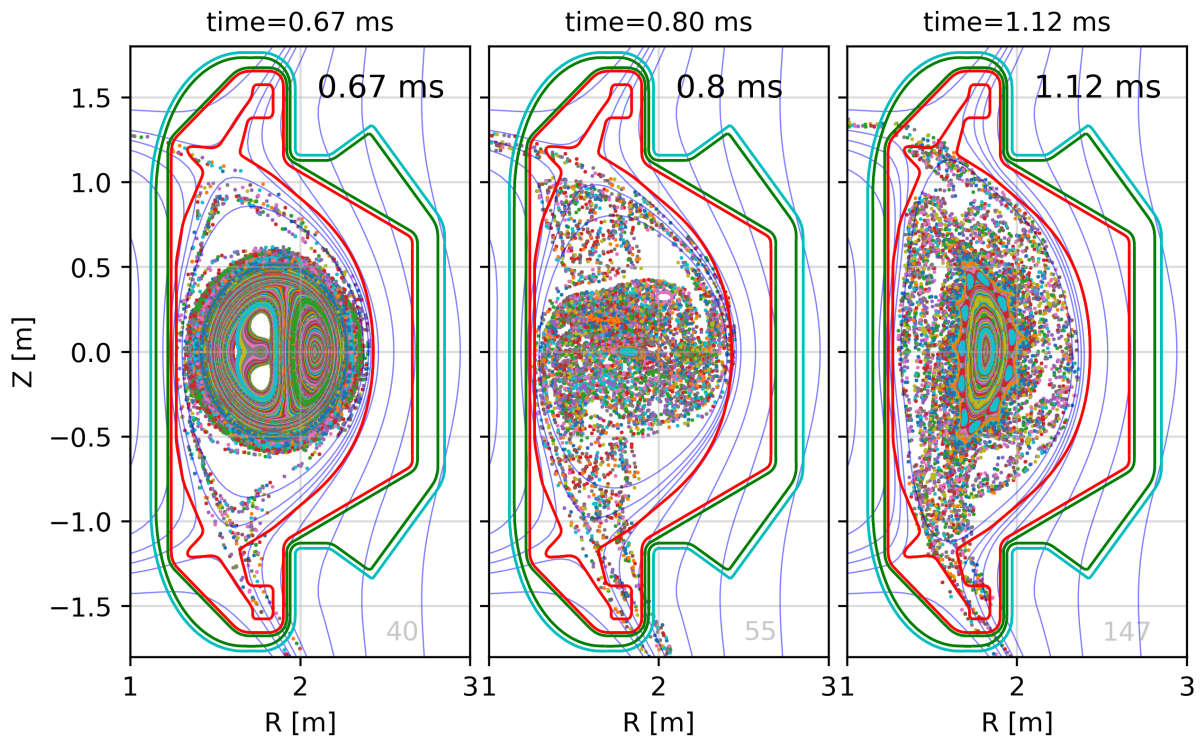
```

period = 6.28319
nperiods = 1
Reading adjacency data.
Max neighbors = 5

```

[DONE]

deleting simulation object and closing 0 fields



RE Strike Locations vs. Time

```

In [ ]: tids = np.array([52,55,63,100,147])

fig, axs = plt.subplots(1,tids.shape[0],dpi=400,figsize=(8.5,4),sharey=True)

#### Load the limiter ###
data = np.loadtxt('sparc_v3_first_wall_202303-smooth.dat',skiprows=1)
Rlim, Zlim = data[:,0], data[:,1]
# axs.plot(Rlim, Zlim,'--g')

from shapely.geometry import Polygon

# 1. Create a Polygon object from your Rlim, Zlim arrays
contour_coords = np.column_stack((Rlim, Zlim))
poly = Polygon(contour_coords)

# 2. Shrink the polygon by a specific distance (e.g., 0.02 meters)
# Passing a negative value to .buffer() shrinks it inwards
shrunk_poly = poly.buffer(-0.01)

# 3. Extract the new R and Z arrays to use in your interpolation/plotting
Rlim = np.array(shrunk_poly.exterior.coords.xy[0])
Zlim = np.array(shrunk_poly.exterior.coords.xy[1])

#####

```

```

for ii in range(tids.shape[0]):

    # Load
    tid = tids[ii]
    R, Z, jre = load_jre(tid)

    im = axs[ii].imshow(jre/1e6, cmap='inferno', origin='lower', extent=[R[0],
                                vmin=5e-7,
                                vmax=5e-4,
                                norm='log'
                                ])

    axs[ii].set_xlabel('R [m]')
    axs[ii].set_ylim([-1.8, 1.8])
    axs[ii].set_xlim([1.0, 3.0])
    axs[ii].set_facecolor('k')

    # LCFS
    lcfs = np.loadtxt(inDir + sim_id + '/idl_output/' + 'lcfs-' + str(tid) +
    axs[ii].plot(lcfs[:,0], lcfs[:,1], 'w', linewidth=0.33, alpha=1)

    # Bound
    m3dc1.plot_mesh(mesh_ob, boundary=True, ax=axs[ii], meshcol='C1', filename=j

    if (ii == tids.shape[0]-1):
        fig.colorbar(im, ax=axs, fraction=0.015, pad=0.04, label='$j_{RE}$ I

    tval = np.round(get_time_of_slice(int(tid), filename=inDir+sim_id+'C1.h5')
    axs[ii].set_title('t = ' + str(tval) + ' ms')
    # axs[ii].grid(color='w', alpha=0.25)

    # Strike locations
    # axs[ii].plot(Rlim, Zlim, '--g')
    intersect_R, intersect_Z, strike_jre = find_contour_intersections(jre.T/
    axs[ii].plot(intersect_R, intersect_Z, 's', color='lime', markersize=0.2

    axs[ii].text(2.4, -1.7, str(tids[ii]), color='w', alpha=0.33, zorder=11, fonts

axs[0].set_ylabel('Z [m]')

# fig.tight_layout(pad=0.2)

```

```

deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields

```

```
Out[ ]: Text(0, 0.5, 'Z [m]')
```

```

In [136... tids = np.array([55,147])

fig, axs = plt.subplots(1,tids.shape[0],dpi=400,figsize=(4.5,4),sharey=True)

#### Load the limiter ###
data = np.loadtxt('sparc_v3_first_wall_202303-smooth.dat',skiprows=1)
Rlim, Zlim = data[:,0], data[:,1]
# axs.plot(Rlim, Zlim,'--g')

from shapely.geometry import Polygon

# 1. Create a Polygon object from your Rlim, Zlim arrays
contour_coords = np.column_stack((Rlim, Zlim))
poly = Polygon(contour_coords)

# 2. Shrink the polygon by a specific distance (e.g., 0.02 meters)
# Passing a negative value to .buffer() shrinks it inwards
shrunk_poly = poly.buffer(-0.01)

# 3. Extract the new R and Z arrays to use in your interpolation/plotting
Rlim = np.array(shrunk_poly.exterior.coords.xy[0])
Zlim = np.array(shrunk_poly.exterior.coords.xy[1])

#####

for ii in range(tids.shape[0]):

    # Load
    tid = tids[ii]
    R, Z, jre = load_jre(tid)

    im = axs[ii].imshow(jre/1e6,cmap='inferno',origin='lower', extent=[R[0],
                                vmin=5e-7,
                                vmax=1,
                                norm='log'
                                ])

    axs[ii].set_xlabel('R [m]')

```

```

    axs[ii].set_ylim([-1.8,1.8])
    axs[ii].set_xlim([1.0,3.0])
    axs[ii].set_facecolor('k')

    # LCFS
    lcfs = np.loadtxt(inDir + sim_id + '/idl_output/' + 'lcfs-' + str(tid) +
    axs[ii].plot(lcfs[:,0],lcfs[:,1], 'w', linewidth=0.33, alpha=1)

    # Bound
    m3dc1.plot_mesh(mesh_ob, boundary=True, ax=axs[ii], meshcol='C1', filename=i

    # if (ii == tids.shape[0]-1):
        # fig.colorbar(im, ax=axs, fraction=0.04, pad=0.1)

    tval = np.round(get_time_of_slice(int(tid), filename=inDir+sim_id+'C1.h5')
    # axs[ii].set_title('t = ' + str(tval) + ' ms')
    # axs[ii].grid(color='w', alpha=0.25)

    # Strike locations
    # axs[ii].plot(Rlim, Zlim, '--g')
    intersect_R, intersect_Z, strike_jre = find_contour_intersections(jre.T/
    axs[ii].plot(intersect_R, intersect_Z, 's', color='lime', markersize=0.5

    axs[ii].text(2.4,-1.7,str(tids[ii]),color='w',alpha=0.33,zorder=11,fontsize=10)
    axs[ii].text(2.1,1.5,str(tval) + ' ms',color='w',alpha=1,zorder=11,fontsize=10)

axs[0].set_ylabel('Z [m]')

fig.subplots_adjust(wspace=0.055) # Adjust the float value to your liking

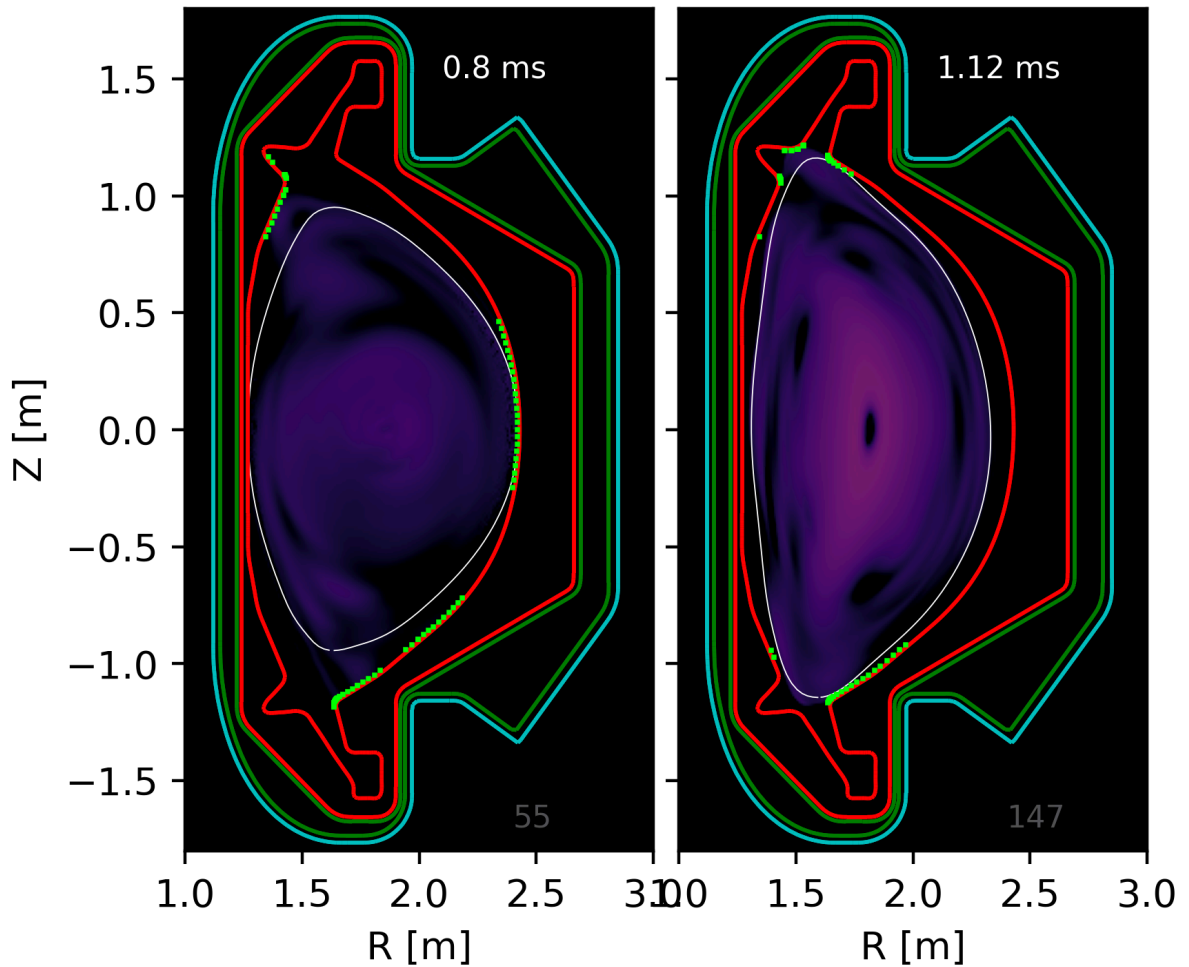
# fig.tight_layout(pad=0.2)

```

```

deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields

```



RE Strike Locations vs. Toroidal Position

```
In [394... tid = 55
angles = np.array([0,90,180,270])

fig, axs = plt.subplots(1,angles.shape[0],dpi=400,figsize=(8.5,4),sharey=True)

#### Load the limiter ###
data = np.loadtxt('sparc_v3_first_wall_202303-smooth.dat',skiprows=1)
Rlim, Zlim = data[:,0], data[:,1]
# axs.plot(Rlim, Zlim,'--g')

from shapely.geometry import Polygon

# 1. Create a Polygon object from your Rlim, Zlim arrays
contour_coords = np.column_stack((Rlim, Zlim))
poly = Polygon(contour_coords)

# 2. Shrink the polygon by a specific distance (e.g., 0.02 meters)
# Passing a negative value to .buffer() shrinks it inwards
shrunk_poly = poly.buffer(-0.01)

# 3. Extract the new R and Z arrays to use in your interpolation/plotting
Rlim = np.array(shrunk_poly.exterior.coords.xy[0])
Zlim = np.array(shrunk_poly.exterior.coords.xy[1])
```

```
#####

for ii in range(angles.shape[0]):
    R, Z, jre = load_jre(tid, angles[ii])
    im = axs[ii].imshow(jre/1e6, cmap='inferno', origin='lower', extent=[R[0],
                                vmin=5e-7,
                                vmax=5e-4,
                                norm='log'])

    axs[ii].set_xlabel('R [m]')
    axs[ii].set_ylim([-1.8, 1.8])
    axs[ii].set_xlim([1.0, 3.0])
    axs[ii].set_facecolor('k')

    # LCFS
    lcfs = np.loadtxt(inDir + sim_id + '/idl_output/' + 'lcfs-' + str(tid) +
    axs[ii].plot(lcfs[:,0], lcfs[:,1], 'w', linewidth=0.33, alpha=1)

    # Bound
    m3dc1.plot_mesh(mesh_ob, boundary=True, ax=axs[ii], meshcol='C1', filename=i

    if (ii == tids.shape[0]-1):
        fig.colorbar(im, ax=axs, fraction=0.015, pad=0.04, label='$j_{RE}$ I

    tval = np.round(get_time_of_slice(int(tid), filename=inDir+sim_id+'C1.h5')
    axs[ii].set_title('$\phi$ = ' + str(angles[ii]) + '$\degree$')

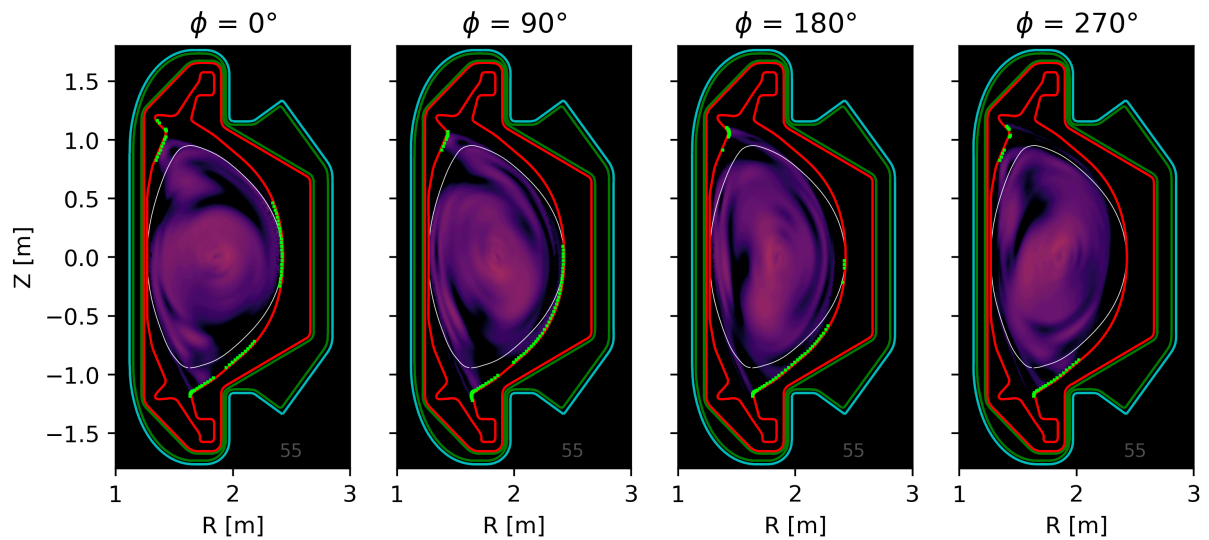
    # Strike locations
    intersect_R, intersect_Z, strike_jre = find_contour_intersections(jre.T/
    axs[ii].plot(intersect_R, intersect_Z, 's', color='lime', markersize=0.5

    axs[ii].text(2.4, -1.7, str(tid), color='w', alpha=0.33, zorder=11, fontsize=8

axs[0].set_ylabel('Z [m]')
```

```
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
```

```
Out[394]: Text(0, 0.5, 'Z [m]')
```

Scrape Off

E-field and RE current density vs time

```
In [238... # Target time
tid_target = 242
t_target = m3dc1.get_time_of_slice(int(tid_target), filename=inDir+sim_id+'C1
```

deleting simulation object and closing 0 fields

```
In [239... jre, E, t = [], [], []
tid = range(228, 250, 1)
nmax=5
for ii in tid:
    try:
        data = np.loadtxt(inDir + sim_id + '/q_profiles/nre-' + str(ii) + '.
        jre.append(-1 * m3dc1_units.unit['j0'] * np.mean(data[1:nmax, 1])) #

        data = np.loadtxt(inDir + sim_id + '/q_profiles/epar-' + str(ii) + '.
        E.append(np.mean(data[1:nmax, 1])) # V/m

        t.append(m3dc1.get_time_of_slice(int(ii), filename=inDir+sim_id+'C1.f

    except:
        continue

# Extract values
jre_out = np.interp(t_target, np.array(t), np.array(jre)) # A/m2
E_out = np.interp(t_target, np.array(t), np.array(E)) # V/m

plt.rcParams.update({'font.size': 8})
fig, axs = plt.subplots(3, 1, dpi=400, figsize=(2, 4), sharex=True)

axs[0].plot(t, np.array(jre)/1e6, color='tab:blue')
axs[0].set_ylabel('$j_{RE}$ (r = 0) [MA m-2]$')
```

```

axs[-1].set_xlabel('t [ms]')
axs[0].set_ylim(ymin=0)
axs[0].grid(alpha=0.33)
axs[0].set_xlim(np.min(t),2.45)
axs[0].set_xticks(np.arange(2.1,2.45,0.1))

# Plot RE current
re = get_h5_dataset('scalars/runaways',filename=inDir + sim_id + 'C1.h5')
time = get_h5_dataset('scalars/time',filename=inDir + sim_id + 'C1.h5')
re = re * m3dc1_units.unit["j0"] # A
time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2) # s

# Extract values
ire_out = np.interp(t_target,time*1e3,re) # A

ire_dot = np.gradient(re,time) # A/s
ire_dot_out = np.interp(t_target,time*1e3,ire_dot) # A/s

ax2 = plt.twinx(axs[0])
ax2.plot(time*1e3,re/1e3,color='tab:orange',alpha=0.7)
ax2.set_ylabel('$I_{RE}$ [kA]')
ax2.set_ylim(ymin=0)

axs[1].plot(t,np.array(E)/1e3)
axs[1].set_ylabel('$E_{\parallel}$ (r = 0) [kV/m]$')
axs[1].set_ylim(ymin=0)
axs[1].grid(alpha=0.33)

# Ap vs time

zmag = get_h5_dataset('scalars/zmag',filename=inDir + sim_id + 'C1.h5') # m
Ap = get_h5_dataset('scalars/area_p',filename=inDir + sim_id + 'C1.h5') # m2
time = get_h5_dataset('scalars/time',filename=inDir + sim_id + 'C1.h5')

time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2) # s

Apdot = np.gradient(Ap,time) # m2/s

axs[2].plot(time*1e3,Ap)
axs[2].set_xlabel('t [ms]')
axs[2].set_ylabel(r'$A_p$ [m2]')
axs[2].set_ylim([0,1])
axs[2].grid(alpha=0.5)

ax2 = axs[2].twinx()
ax2.plot(time*1e3,Apdot/1e3,'-',color='tab:red')
ax2.set_ylabel(r'$\dot{A}_p$ [m2/ms]')
ax2.set_ylim(-4,0)

# Plot target time
for ii in range(axs.shape[0]):
    axs[ii].plot([t_target]*2,[0,5e3], '--k',alpha=0.33)

```

```

# Extract values
Ap_out = np.interp(t_target,time*1e3,Ap) # m2
Apdot_out = np.interp(t_target,time*1e3,Apdot) # m2/s

# Print
print("\n")
print("*****")
print("jre [MA/m2] = %1.3g"%(jre_out/1e6))
print("E_par [V/m] = %1.3g"%(E_out))
print("I_RE [kA] = %1.3g"%(ire_out/1e3))
print("Ap [m2] = %1.3g"%(Ap_out))
print("Ap_dot [m2/s] = %1.3g"%(Apdot_out))

IRE_loss_rate = Apdot_out * jre_out # A/s

print("RE Current Loss Rate [kA/ms] = %1.3g"%(IRE_loss_rate/1e3/1e3))
print("Actual RE Rate [kA/ms] = %1.3g"%(ire_dot_out/1e3/1e3))

print("Expected (Effective) Generation Rate [kA/ms] = %1.3g"%((ire_dot_out-I

# Avalanching rate
data = np.loadtxt(inDir + sim_id + '/q_profiles/ne-' + str(tid_target) + '.t
ne = np.mean(data[1:nmax,1]) # m-3
data = np.loadtxt(inDir + sim_id + '/q_profiles/den-' + str(tid_target) + '.t
ni = np.mean(data[1:nmax,1]) # m-3
data = np.loadtxt(inDir + sim_id + '/q_profiles/kprad_totden-' + str(tid_tar
nz = np.mean(data[1:nmax,1]) # m-3
data = np.loadtxt(inDir + sim_id + '/q_profiles/Te-' + str(tid_target) + '.t
Te = np.mean(data[1:nmax,1]) - 7.5 # eV

lnAc = 14.6 + 0.5 * np.log((Te + 1e-1) / (ne/1e20)) # RE-Thermal Coul. log
Ec = RE.getConnorHastieField(ne, lnAc)
E_star = E_out / Ec
Zion = (ne - ni) / nz

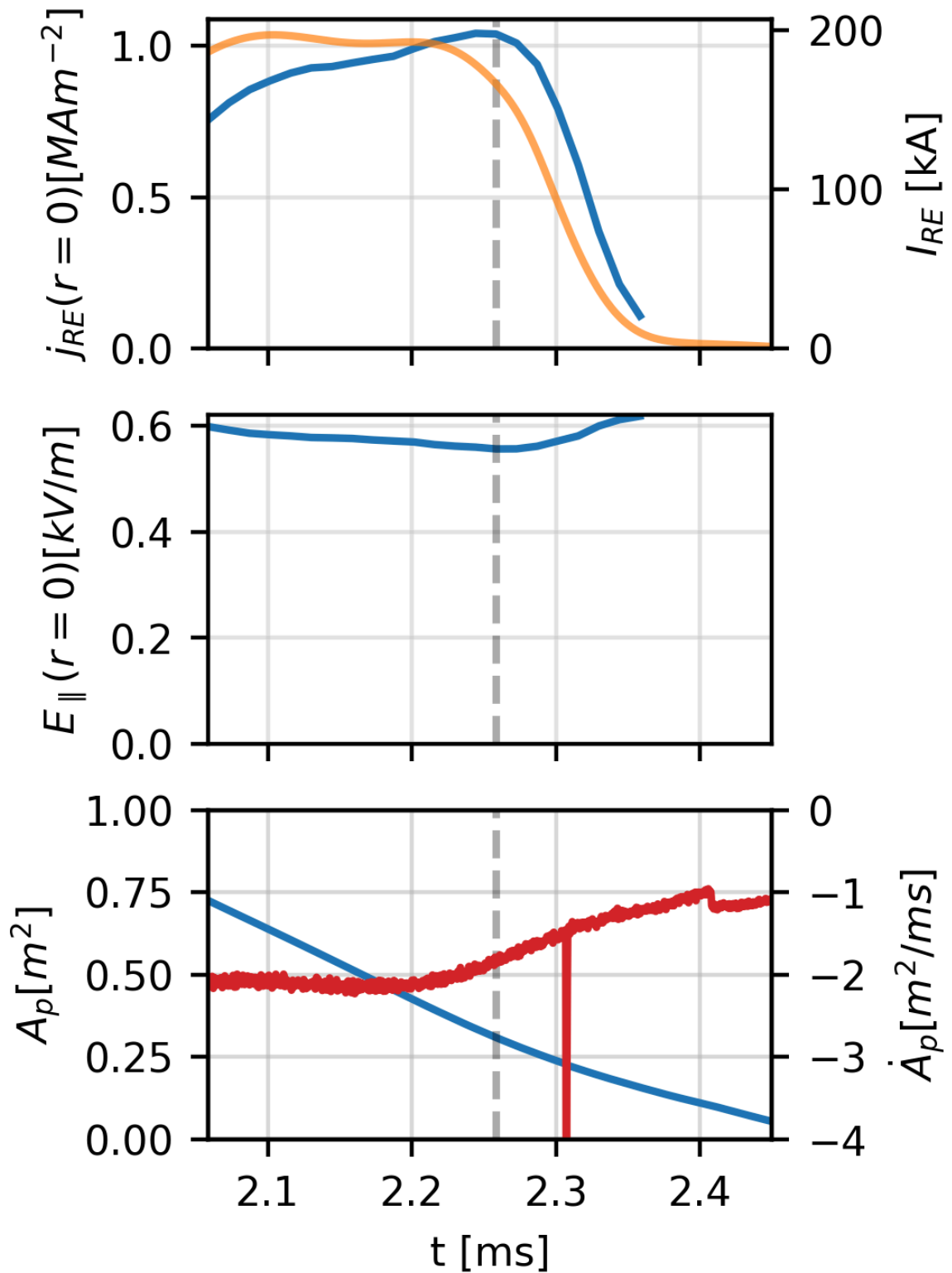
Zion_up = int(np.floor(Zion))
Yup = RE.getHesslowAvalanchingRate(E_star,
                                   np.array([1,10]),
                                   np.array([1,Zion_up]),
                                   np.array([ni,nz]),
                                   Te,
                                   'Ne',
                                   B=12.2) # s

ire_gen_rate = Yup * jre_out * Ap_out # A/s
print("Calculated Avalanche Rate [kA/ms] = %1.3g"%(ire_gen_rate/1e3/1e3))

```

```
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
```

```
jre [MA/m2] = 1.04
E_par [V/m] = 556
I_RE [kA] = 166
Ap [m2] = 0.309
Ap_dot [m2/s] = -1.83e+03
RE Current Loss Rate [kA/ms] = -1.9e+03
Actual RE Rate [kA/ms] = -995
Expected (Effective) Generation Rate [kA/ms] = 908
Calculated Avalanche Rate [kA/ms] = 5.23e+03
```



Avalanching rate

```
In [240... tid = tid_target # 228, 242
R, Z, ne, lcfs = load_data(tid, 'ne')
R, Z, ni, lcfs = load_data(tid, 'den')
R, Z, nz, lcfs = load_data(tid, 'kprad_totden')
```

```

R, Z, Te, lcfs = load_data(tid, 'te')
Te = Te - 7.5
R, Z, epar, lcfs = load_data(tid, 'E_par')
R, Z, nre, lcfs = load_data(tid, 'nre')
jre = -1 * m3dc1_units.unit['j0'] * nre # A/m2

```

```

In [241]: fields = [ne, ni, nz, Te, epar, jre]
labels = ['$n_e$', '$n_i$', '$n_z$', '$T_e$ [eV]', '$E_{||}$', '$j_{RE}$']
cmaps = ['viridis', 'viridis', 'viridis', 'hot', 'RdBu_r', 'magma']
minvals = [1e18, 1e18, 1e18, 0, -2e3, 0]
maxvals = [1e22, 1e21, 1e21, 50, 1e3, 1e6]

fig, axs = plt.subplots(1, len(fields), figsize=(18, 5), sharey=True, dpi=15)
extent = [R.min(), R.max(), Z.min(), Z.max()]

for i, ax in enumerate(axs):
    norm = 'log' if i < 3 else None

    im = ax.imshow(fields[i].reshape(len(R), len(Z)),
                    extent=extent, origin='lower',
                    cmap=cmaps[i], norm=norm, vmin=minvals[i], vmax=maxvals[i])

    # Add LCFS overlay
    ax.plot(lcfs[:, 0], lcfs[:, 1], 'w', linewidth=0.8, alpha=0.8)

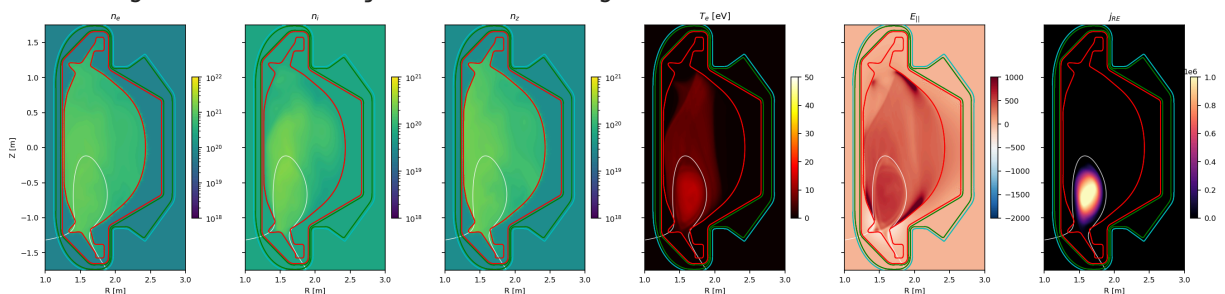
    # Bound
    m3dc1.plot_mesh(mesh_ob, boundary=True, ax=ax, meshcol='C1', filename=inDir)

    ax.set_title(labels[i])
    ax.set_xlabel('R [m]')
    ax.set_xlim([1.0, 3])
    ax.set_ylim([-1.75, 1.75])
    plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

axs[0].set_ylabel('Z [m]')
fig.tight_layout()
plt.show()

```

deleting simulation object and closing 0 fields
 deleting simulation object and closing 0 fields
 deleting simulation object and closing 0 fields
 deleting simulation object and closing 0 fields
 deleting simulation object and closing 0 fields
 deleting simulation object and closing 0 fields



In [242... *# Calculate Avalanching Rate*

```

Y = np.zeros_like(ne)

for jj in range(ne.shape[0]):
    for ii in range(ne.shape[1]):

        lnAc = 14.6 + 0.5 * np.log((Te[jj,ii] + 1e-1) / ((ne[jj,ii]+1e1)/1e20)) #
        Ec = RE.getConnorHastieField(ne[jj,ii], lnAc)
        E_star = epar[jj,ii] / Ec
        Zion = (ne[jj,ii] - ni[jj,ii]) / nz[jj,ii]

        Zion_up = int(np.ceil(Zion))
        Yup = RE.getHesslowAvalanchingRate(E_star,
                                           np.array([1,10]),
                                           np.array([1,Zion_up]),
                                           np.array([ni[jj,ii],nz[jj,ii]]),
                                           Te[jj,ii],
                                           'Ne',
                                           B=12.2) # s

        Zion_dn = int(np.floor(Zion))
        Ydn = RE.getHesslowAvalanchingRate(E_star,
                                           np.array([1,10]),
                                           np.array([1,Zion_dn]),
                                           np.array([ni[jj,ii],nz[jj,ii]]),
                                           Te[jj,ii],
                                           'Ne',
                                           B=12.2) # s

        w = 1- (Zion_up - Zion)
        Y[jj,ii] = w * Yup + (1-w) * Ydn

```

```

/tmp/ipykernel_270931/1057949171.py:8: RuntimeWarning: invalid value encountered in log
    lnAc = 14.6 + 0.5 * np.log((Te[jj,ii] + 1e-1) / ((ne[jj,ii]+1e1)/1e20)) #
RE-Thermal Coul. log

```

In [225... *source_ava = jre * Y # [A/m2/s]*

```

fig, ax = plt.subplots(1, 1, figsize=(2, 4), sharey=True, dpi=300)
im = ax.imshow(1e-3*1e-6*source_ava.reshape(len(R), len(Z)),
               extent=extent, origin='lower', vmin=0, vmax=20)

# Add LCFS overlay
ax.plot(lcfs[:, 0], lcfs[:, 1], 'w', linewidth=0.8, alpha=0.8)

# Bound
m3dc1.plot_mesh(mesh_ob, boundary=True, ax=ax, meshcol='C1', filename=inDir + si

plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04, label='$j_{RE}$ \\gamma_{av}$

ax.set_xlim([1.0,3])
ax.set_ylim([-1.75,1.75])
ax.set_xlabel('R [m]')

```

```

ax.set_ylabel('Z [m]')

# Integrate

dA = np.diff(R)[0] * np.diff(Z)[0]
ava_tot = np.sum(source_ava.flatten()*dA) # A/s
print('ava_tot [kA/ms] = %1.3g'%(ava_tot/1e3/1e3))

import scipy.integrate as integrate

# Integrate over Z (columns), then over R (rows)
ava_tot = integrate.trapz(integrate.trapz(source_ava, x=Z, axis=1), x=R, axis=0)

print('ava_tot [kA/ms] = %1.3g'%(ava_tot/1e3/1e3))

import scipy.integrate as integrate
from matplotlib.path import Path

lcfs_path = Path(lcfs)
R_grid, Z_grid = np.meshgrid(R, Z, indexing='ij')
points = np.column_stack((R_grid.flatten(), Z_grid.flatten()))
mask = lcfs_path.contains_points(points).reshape(source_ava.shape)
mask = mask.T
source_ava_lcfs = np.where(mask, source_ava, 0)

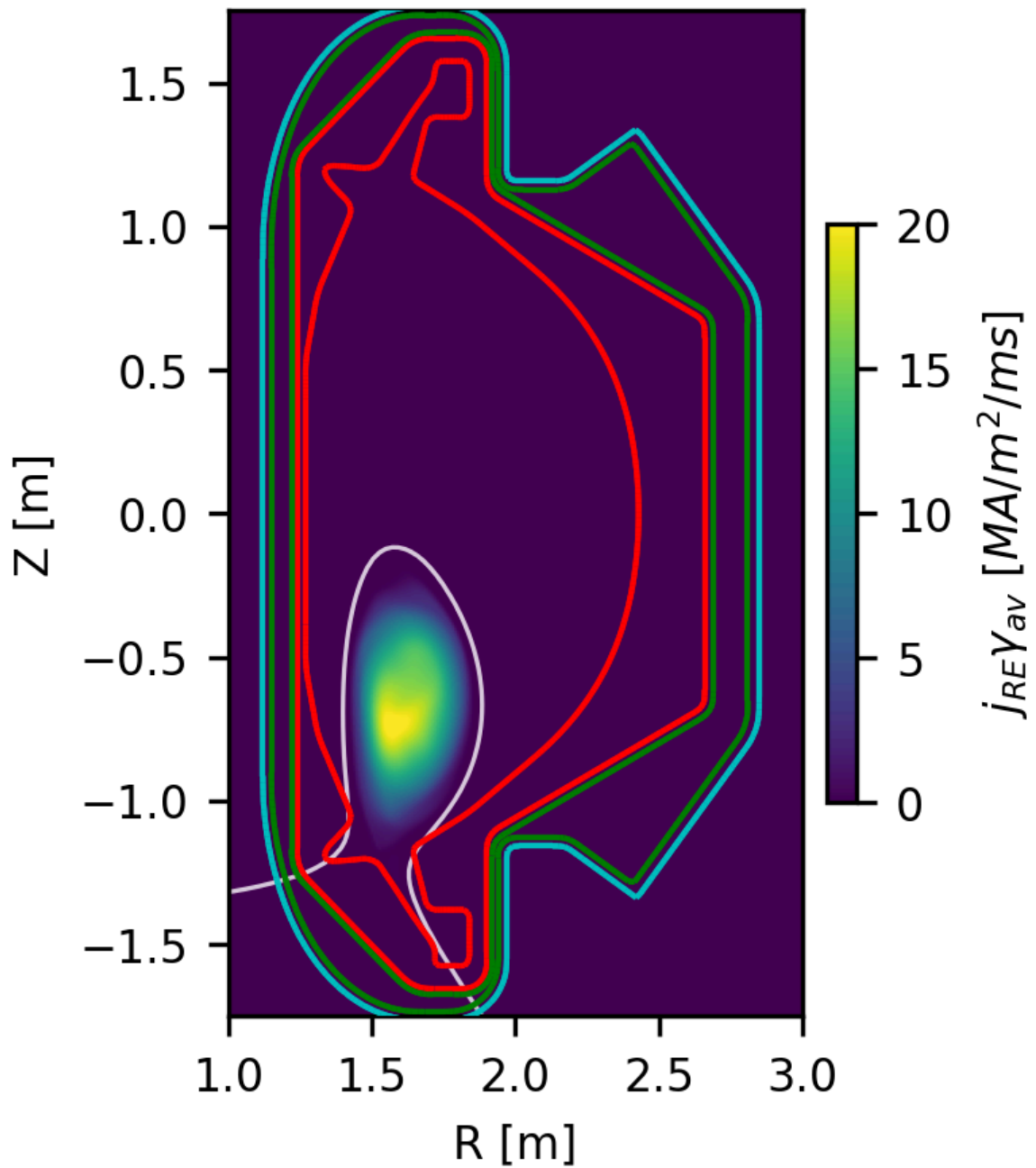
# 5. Integrate the masked array
ava_tot_lcfs = integrate.trapz(integrate.trapz(source_ava_lcfs, x=Z, axis=1), x=R, axis=0)

print("*****")
print('ava_tot inside LCFS [kA/ms] = %1.3g' % (ava_tot_lcfs/1e3/1e3))
print("*****")

print("Effective Generation Rate [kA/ms] = %1.3g"%((ire_dot_out-IRE_loss_rate)/1e3/1e3))

deleting simulation object and closing 0 fields
ava_tot [kA/ms] = 2.72e+03
ava_tot [kA/ms] = 2.72e+03
*****
ava_tot inside LCFS [kA/ms] = 2.72e+03
*****
Effective Generation Rate [kA/ms] = 908

```

```
In [134... fig, ax = plt.subplots(1, 1, figsize=(2, 4), sharey=True, dpi=300)
im = ax.imshow(mask.reshape(len(R), len(Z)),
                extent=extent, origin='lower', vmin=0, vmax=1)

# Add LCFS overlay
ax.plot(lcfs[:, 0], lcfs[:, 1], 'w', linewidth=0.8, alpha=0.8)

# Bound
m3dc1.plot_mesh(mesh_ob, boundary=True, ax=ax, meshcol='C1', filename=inDir + si

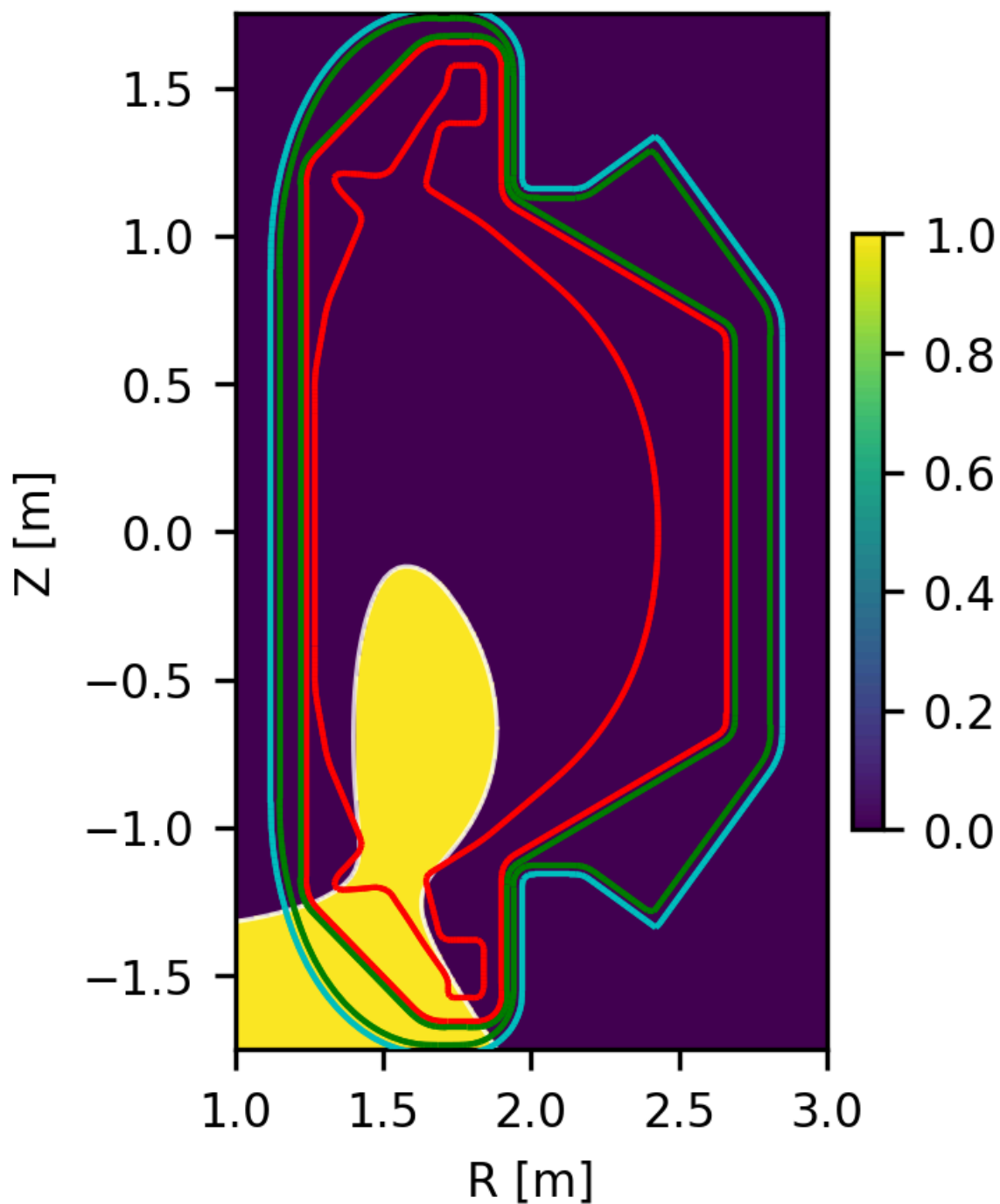
plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

ax.set_xlim([1.0, 3])
ax.set_ylim([-1.75, 1.75])
```

```
ax.set_xlabel('R [m]')  
ax.set_ylabel('Z [m]')
```

deleting simulation object and closing 0 fields

```
Out[134... Text(0, 0.5, 'Z [m]')
```



In []:

Flux across boundary

```

In [226... # Plasma boundary contour and velocity
levels=[0.32] # Psi level to extract
tid_target_prime = tid_target + 2

fig, axs = plt.subplots(1,3,dpi=250,sharey=True)

# At t = tid_target
sim, time, mesh_ob, R, phi_list, Z, R_mesh, Z_mesh, R_ave, Z_ave, psi = m3dc

axs[0].contourf(R_ave,Z_ave,np.squeeze(psi),200)
CS = axs[0].contour(R_ave,Z_ave,np.squeeze(psi), levels=levels, colors='w',
axs[0].set_title('tid =' + str(tid_target))

psi0 = psi

# Extract contour
s = CS.allsegs[0][0]
RS, ZS = s[:,0], s[:,1]

# at t = tid_target + 1

sim, time, mesh_ob, R, phi_list, Z, R_mesh, Z_mesh, R_ave, Z_ave, psi = m3dc

axs[1].contourf(R_ave,Z_ave,np.squeeze(psi),200)
CS = axs[1].contour(R_ave,Z_ave,np.squeeze(psi), levels=levels, colors='w',
axs[1].set_title('tid =' + str(tid_target_prime))

# Extract contour
s_prime = CS.allsegs[0][0]
RS_prime, ZS_prime = s_prime[:,0], s_prime[:,1]

```

```

# Poincare plot
tid = tid_target
poin_plot(sim=sim,time=tid,ax=axis[2],inDir=inDir,sim_id=sim_id) # Poincare p
m3dc1.plot_mesh(mesh_ob,boundary=True,ax=axis,meshcol='C1',filename=inDir + s
axis[2].contour(R_ave,Z_ave,np.squeeze(psi0), levels=levels, colors='blue', l

# format
for ii in range(axis.shape[0]):
    axis[ii].set_xlim([1.2,2])
    axis[ii].set_ylim([-1.25,0])
    axis[ii].set_aspect('equal')
    m3dc1.plot_mesh(mesh_ob,boundary=True,ax=axis[ii],meshcol='C1',filename=i
    axis[ii].set_xlabel('R [m]')

axis[0].set_ylabel('Z [m]')

```

Output version: 45

Mesh shape: (260337, 10)

Evaluating field 'psi'...

period = 6.28319

nperiods = 1

Reading adjacency data.

Max neighbors = 5

[DONE]

deleting simulation object and closing 0 fields

Output version: 45

Mesh shape: (260337, 10)

Evaluating field 'psi'...

/tmp/ipykernel_270931/735933949.py:23: UserWarning: The following kwargs were not used by contour: 'linewidth'

```
CS = axis[0].contour(R_ave,Z_ave,np.squeeze(psi), levels=levels, colors
='w', linestyle='--',alpha=0.7,linewidth=0.5)
```

period = 6.28319

nperiods = 1

Reading adjacency data.

Max neighbors = 5

[DONE]

deleting simulation object and closing 0 fields

Reading /pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-3D-01/TRACE/TS242

/tmp/ipykernel_270931/735933949.py:48: UserWarning: The following kwargs were not used by contour: 'linewidth'

```
CS = axis[1].contour(R_ave,Z_ave,np.squeeze(psi), levels=levels, colors
='w', linestyle='--',alpha=0.7,linewidth=0.5)
```

deleting simulation object and closing 0 fields

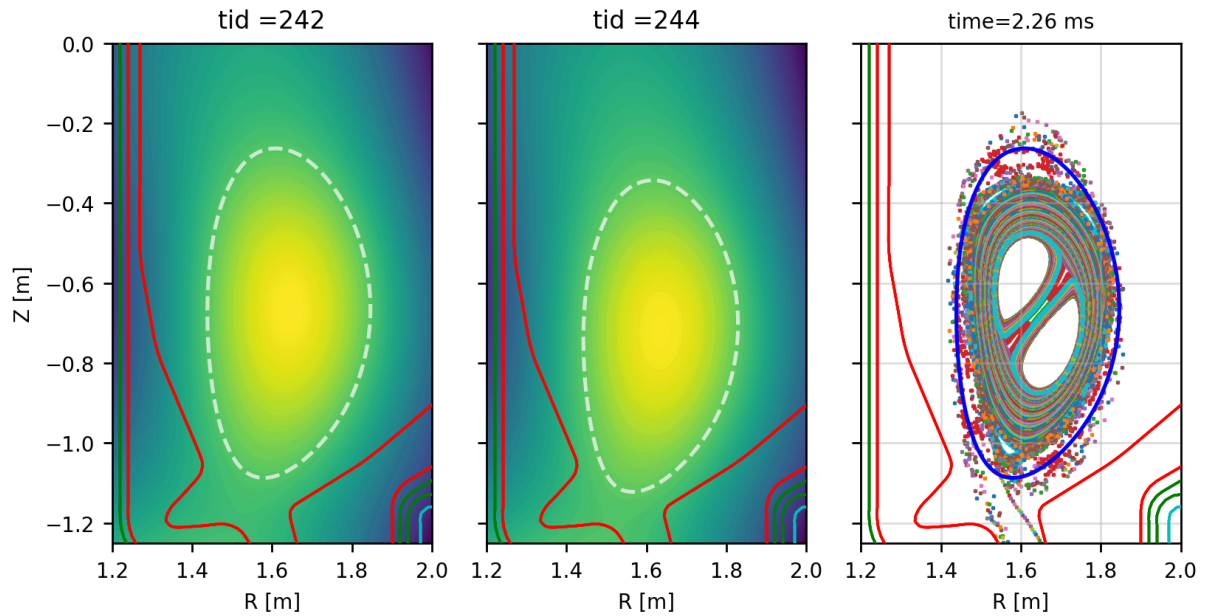
deleting simulation object and closing 0 fields

deleting simulation object and closing 0 fields

deleting simulation object and closing 0 fields

deleting simulation object and closing 0 fields

Out[226... Text(94.43055555555557, 0.5, 'Z [m]')



In [227... *# Plot jre*

```

R, Z, nre, lcfs = load_data(tid_target, 'nre')
jre = -1 * m3dc1_units.unit['j0'] * nre # A/m2

t0 = get_time_of_slice(int(tid_target), filename=inDir+sim_id+'C1.h5', units='
t_prime = get_time_of_slice(int(tid_target_prime), filename=inDir+sim_id+'C1.

#####
# Plot the contour
#####
fig, axs = plt.subplots(1,2,dpi=250)

im = axs[0].imshow(jre.reshape(len(R), len(Z)),
                    extent=[R.min(), R.max(), Z.min(), Z.max()], origin='lower',
                    cmap='inferno', vmin = 0, vmax = 1e6)

axs[0].plot(RS,ZS,color='tab:blue',
            label='s (t = ' + str(np.round(t0,2)) + ' ms)')
axs[0].plot(RS_prime,ZS_prime,'--',color='tab:red',
            label=r'$s^{\prime}$ (t = ' + str(np.round(t_prime,2)) + ' ms)')
axs[0].set_xlim([1.2,2])
axs[0].set_ylim([-1.25,0])
m3dc1.plot_mesh(mesh_ob,boundary=True,ax=axs[0],meshcol='C1',filename=inDir
axs[0].set_title('Lab Frame')

#####

#####
# Magnetic axis
#####
zmag = get_h5_dataset('scalars/zmag',filename=inDir + sim_id + 'C1.h5') # m
rmag = get_h5_dataset('scalars/xmag',filename=inDir + sim_id + 'C1.h5') # m
time = get_h5_dataset('scalars/time',filename=inDir + sim_id + 'C1.h5') # s

time = time * m3dc1_units.unit["tau_A"] * np.sqrt(2) # s

```

```

Rm, Zm = np.interp(t0*1e-3, time, rmag), np.interp(t0*1e-3, time, zmag) # m
Rm_prime, Zm_prime = np.interp(t_prime*1e-3, time, rmag), np.interp(t_prime*

axs[0].plot(Rm,Zm,'x',color='tab:blue')
axs[0].plot(Rm_prime,Zm_prime,'x',color='tab:red')

#####
# Plasma frame
#####

RS1, ZS1 = RS - Rm, ZS - Zm
RS_prime1, ZS_prime1 = RS_prime - Rm_prime, ZS_prime - Zm_prime

### Re-sample ###

thq = np.linspace(-np.pi, np.pi, 500)

th = np.arctan2(ZS1, RS1)
sort_idx = np.argsort(th)
th_sorted = th[sort_idx]
RS1_sorted = RS1[sort_idx]
ZS1_sorted = ZS1[sort_idx]
ZS1 = np.interp(thq, th_sorted, ZS1_sorted)
RS1 = np.interp(thq, th_sorted, RS1_sorted)

# Normal to contour
rrp = np.gradient(RS1,thq)
zzp = np.gradient(ZS1,thq)
# Rotate
nr = -1*zzp
nz = rrp
# outward normal vector
nr = -1*nr
nz = -1*nz

# Normalize
mag = np.sqrt(nr**2 + nz**2)
eps = 1e-6
nr, nz = (nr+eps) / (mag+eps), (nz+eps) / (mag+eps)

th_prime = np.arctan2(ZS_prime1, RS_prime1)
sort_idx_prime = np.argsort(th_prime)
th_prime_sorted = th_prime[sort_idx_prime]
RS_prime1_sorted = RS_prime1[sort_idx_prime]
ZS_prime1_sorted = ZS_prime1[sort_idx_prime]
ZS_prime1 = np.interp(thq, th_prime_sorted, ZS_prime1_sorted)
RS_prime1 = np.interp(thq, th_prime_sorted, RS_prime1_sorted)

Rnew, Znew = R - Rm, Z - Zm
im = axs[1].imshow(jre.reshape(len(Rnew), len(Znew)),
                    extent=[Rnew.min(), Rnew.max(), Znew.min(), Znew.max()], orig
                    cmap='inferno', vmin = 0, vmax = 1e6)

```

```

axs[1].plot(RS1,ZS1,color='tab:blue',label='s')
axs[1].plot(RS_prime1,ZS_prime1,'--',color='tab:red',label=r'$s^{\prime}$')

axs[1].set_xlim([1.2-Rm,2-Rm])
axs[1].set_ylim([-1.25-Zm,0-Zm])
axs[1].plot(0,0,'kx')
axs[1].set_title('Plasma Frame')

#####

#####
# Boundary Velocity
#####

del_t = (t_prime - t0) * 1e-3 # s
vb_R = (RS_prime1 - RS1) / del_t # m/s
vb_Z = (ZS_prime1 - ZS1) / del_t # m/s

step = 10

# normal vector
axs[1].quiver(
    RS1[:,step], ZS1[:,step],
    nr[:,step], nz[:,step],
    color='blue',
    label=r'$\vec{n}$',
    angles='xy',
    # scale=100,
    zorder=10
)

# boundary velocity
axs[1].quiver(
    RS1[:,step], ZS1[:,step],
    vb_R[:,step], vb_Z[:,step],
    color='tab:green',
    label=r'$\vec{v}_b$',
    angles='xy',
    # scale=100,
    zorder=11
)

# Check
axs[1].plot(RS1+vb_R*del_t,ZS1+vb_Z*del_t,':',color='tab:pink')

#####
#### RE Velocity at Boundary ####
#####

B = m3dc1.eval_field('B', RS1+Rm, 0*RS1, ZS1+Zm,
                    coord='vector',
                    sim=sim,
                    filename=inDir + sim_id + 'C1.h5',

```

```

        time=tid_target,
        quiet=True) # Tesla

B = B.T

E = m3dc1.eval_field('E', RS1+Rm, 0*RS1, ZS1+Zm,
                    coord='vector',
                    sim=sim,
                    filename=inDir + sim_id + 'C1.h5',
                    time=tid_target,
                    quiet=True) # V/m

E = E.T

# ExB
ExB = np.zeros_like(B)
for ii in range(E.shape[0]):
    ExB[ii,:] = np.cross(E[ii,:],B[ii,:])

# Calculate the vector
Bmag = np.sqrt(B[:,0]**2 + B[:,1]**2 + B[:,2]**2)
Bmag = np.vstack((Bmag,Bmag,Bmag))
b = B / Bmag.T # normalize

# RE Velocity
cre = 0.4 * const.c # m/s
vRE = cre * b + ExB / Bmag.T**2 # m/s
vRE_R = vRE[:,0]
vRE_Z = vRE[:,2]

# RE velocity
axs[1].quiver(
    RS1[:,::step], ZS1[:,::step],
    vRE_R[:,::step], vRE_Z[:,::step],
    color='tab:pink',
    label=r'$\vec{v}_{RE}$',
    angles='xy',
    # scale=100,
    zorder=12
)

#####
# RE Flux
#####

dV_R, dV_Z = vRE_R - vb_R, vRE_Z - vb_Z # Relative velocity
Vn = dV_R * nr + dV_Z * nz # Normal velocity [m/s]
dR = np.diff(RS1)
dZ = np.diff(ZS1)
ds = np.sqrt(dR**2 + dZ**2) # [m]

from scipy.interpolate import RegularGridInterpolator
interp = RegularGridInterpolator((R, Z), jre.T, bounds_error=False, fill_val=0)
contour_points = np.column_stack((RS1+Rm, ZS1+Zm))
jre_along_contour = interp(contour_points) # A/m2

```



```

# interp = RegularGridInterpolator((Rnew, Znew), jre.T, bounds_error=False,
# contour_points = np.column_stack((RS1, ZS1))
# jre_along_contour = interp(contour_points) # A/m2

# F = np.sum(jre_along_contour[1:] * Vn[1:] * ds) # A/s

jre_mid = 0.5 * (jre_along_contour[1:] + jre_along_contour[:-1])
Vn_mid = 0.5 * (Vn[1:] + Vn[:-1])
F = np.sum(jre_mid * Vn_mid * ds) # A/s

print('*****')
print('RE Flux [kA/ms] = %1.3g'%(F/1e3/1e3))
print('*****')

# Check
idx = np.argmax(jre_along_contour)
axs[1].plot(RS1[idx], ZS1[idx], 'cX', zorder=20)

#####
#####
axs[0].set_xlabel('R [m]')
axs[0].set_ylabel('Z [m]')

axs[1].set_xlabel(r'\mathcal{R}$ [m]')
axs[1].set_ylabel(r'\mathcal{Z}$ [m]')

for ii in range(axs.shape[0]):
    axs[ii].set_aspect('equal')
    axs[ii].legend()

fig.tight_layout(pad=0.1)

```

deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields

```

period = 6.28319
nperiods = 1
Reading adjacency data.
Max neighbors = 5
period =

```

```

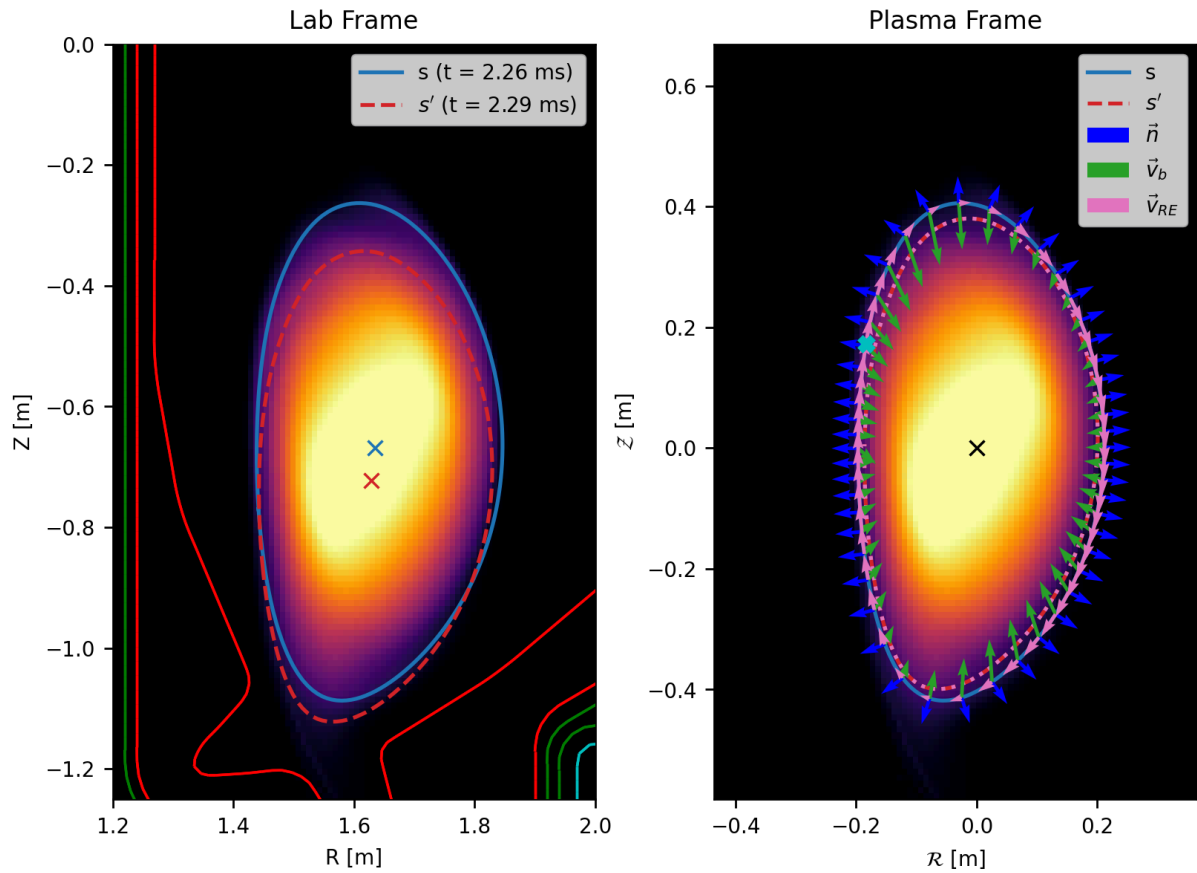
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
*****
RE Flux [kA/ms] = -546
*****

```

```

6.28319
nperiods = 1
Reading adjacency data.
Max neighbors = 5

```



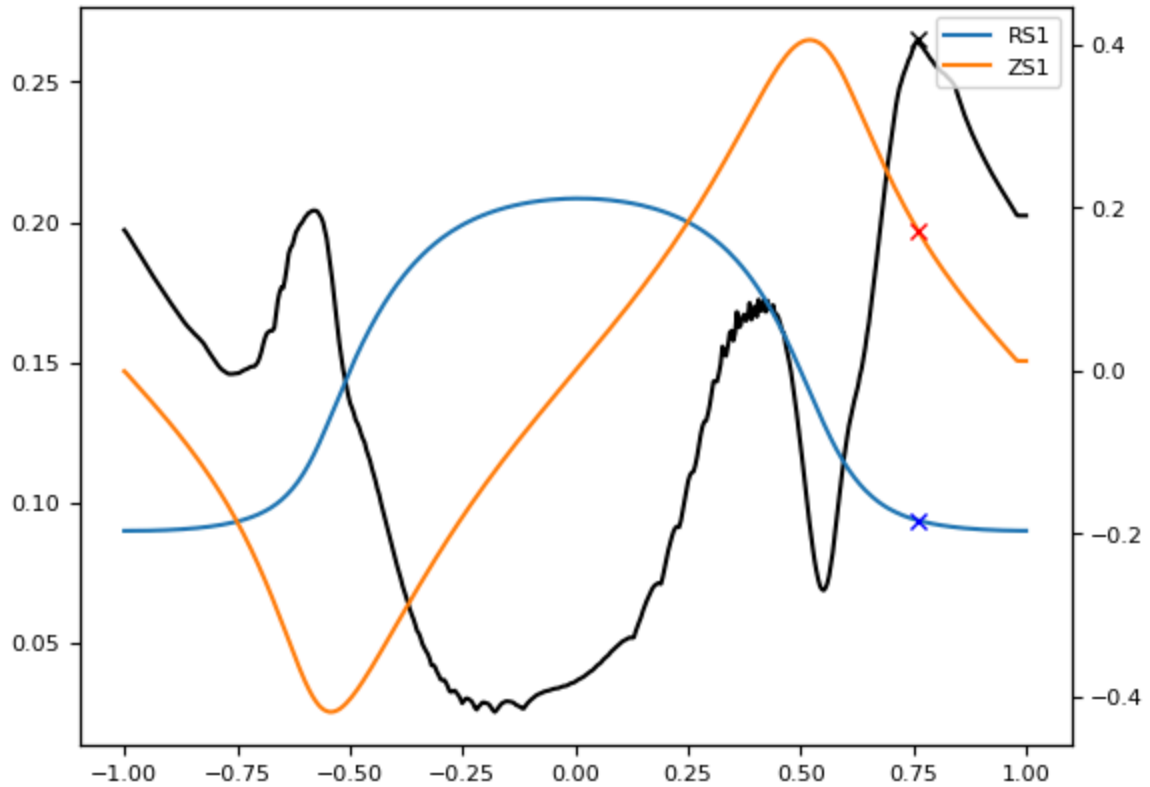
```
In [228... plt.plot(thq/np.pi,jre_along_contour/1e6,'-k')
plt.plot(thq[idx]/np.pi,jre_along_contour[idx]/1e6,'kx')
ax2 = plt.twinx()
ax2.plot(thq/np.pi,RS1,label='RS1')
ax2.plot(thq/np.pi,ZS1,label='ZS1')

ax2.plot(thq[idx]/np.pi,RS1[idx],'bx')
ax2.plot(thq[idx]/np.pi,ZS1[idx],'rx')

ax2.legend()

print('Rmax, Zmax =', RS1[idx], ZS1[idx])
```

Rmax, Zmax = -0.18325913535276864 0.17268182836024432



```
In [235... # Compare
phi=90
# --- First Dataset ---
R, Z, nre, lcfs = load_data(tid_target, 'nre', phi)
jre = -1 * m3dc1_units.unit['j0'] * nre # A/m2
R, Z = R - Rm, Z - Zm

# --- Second Dataset (Use distinct variables) ---
R2, Z2, nre2, lcfs2 = load_data(tid_target_prime, 'nre', phi)
jre_prime = -1 * m3dc1_units.unit['j0'] * nre2 # A/m2
R_prime, Z_prime = R2 - Rm_prime, Z2 - Zm_prime

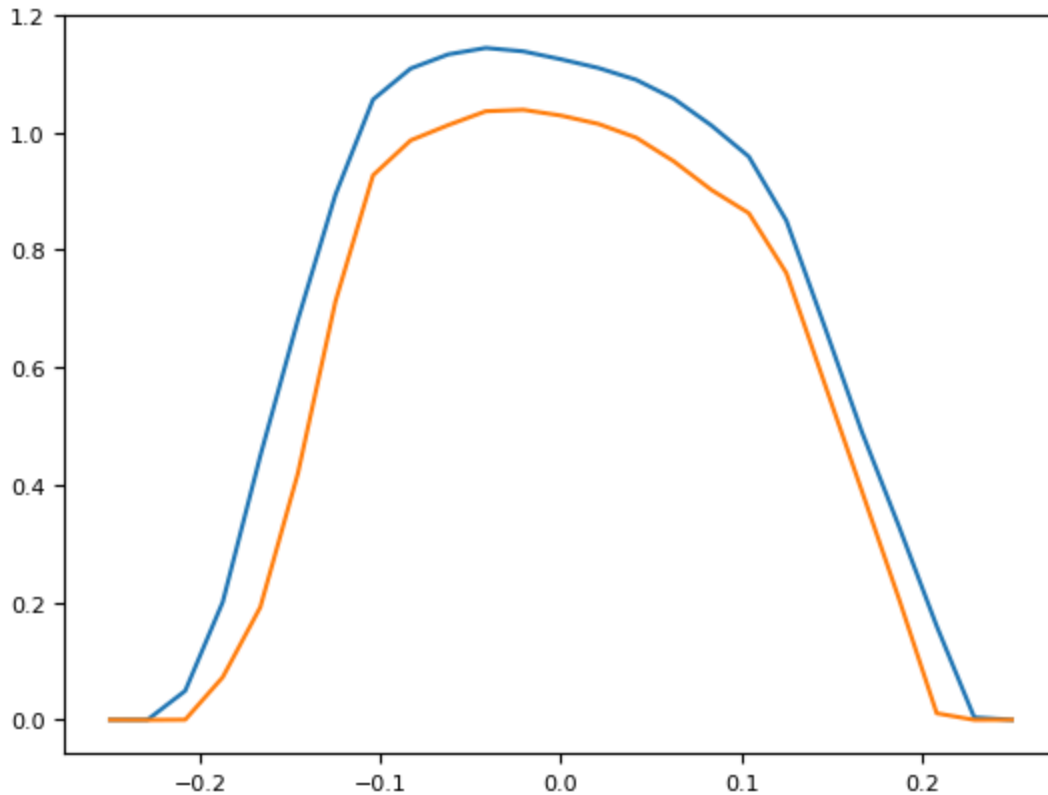
# --- Interpolation ---
Rq = np.linspace(-0.25, 0.25, 25)
Zq = 0 * Rq
contour_points = np.column_stack((Rq, Zq))

interp = RegularGridInterpolator((R, Z), jre.T, bounds_error=False, fill_val=0)
interp_prime = RegularGridInterpolator((R_prime, Z_prime), jre_prime.T, bounds_error=False, fill_val=0)

jre_out = interp(contour_points) # A/m2
jre_out_prime = interp_prime(contour_points) # A/m2

plt.plot(Rq, jre_out/1e6)
plt.plot(Rq, jre_out_prime/1e6)
```

```
Out[235... [<matplotlib.lines.Line2D at 0x7f951da69d10>]
```



External field

```
In [414]: # Create a 2x2 grid of subplots
fig, axs = plt.subplots(2, 2, dpi=200, figsize=(3, 4), sharex=True, sharey=True)

base_dir = inDir + sim_id + '/idl_output/'

# Helper function to load and reshape the CSV data
def load_b_component(phi, comp):
    base_file = f"{base_dir}Bext_160_phi_{phi}"

    # Load coordinates
    r = np.loadtxt(f"{base_file}_r.csv", delimiter=',')
    z = np.loadtxt(f"{base_file}_z.csv", delimiter=',')

    # Load and reshape the requested component ('br' or 'bz')
    with open(f"{base_file}_{comp}.csv") as f:
        lines = f.read().strip().splitlines()

    val = np.array([list(map(float, line.split(','))) for line in lines])
    val = val.reshape(z.shape[0], r.shape[0]).T

    return r, z, val

# Load all 4 datasets
r, z, br_90 = load_b_component('90', 'br')
_, _, bz_90 = load_b_component('90', 'bz')
_, _, br_270 = load_b_component('270', 'br')
_, _, bz_270 = load_b_component('270', 'bz')
```

```

# Plot styling configuration
plot_kwargs = {
    'shading': 'auto',
    'vmin': -0.5,
    'vmax': 0.5,
    'norm': 'linear',
    'cmap': 'bwr'
}

# --- Column 1: B_R ---
# Row 1: B_R at 90°
im = axs[0, 0].pcolormesh(r, z, br_90, **plot_kwargs)
axs[0, 0].set_title(r'$B_R$ at $\phi = 90^\circ$', fontsize=11)
axs[0, 0].set_ylabel('Z [m]')

# Row 2: B_R at 270°
axs[1, 0].pcolormesh(r, z, br_270, **plot_kwargs)
axs[1, 0].set_title(r'$B_R$ at $\phi = 270^\circ$', fontsize=11)
axs[1, 0].set_ylabel('Z [m]')
axs[1, 0].set_xlabel('R [m]')

# --- Column 2: B_Z ---
# Row 1: B_Z at 90°
axs[0, 1].pcolormesh(r, z, bz_90, **plot_kwargs)
axs[0, 1].set_title(r'$B_Z$ at $\phi = 90^\circ$', fontsize=11)

# Row 2: B_Z at 270°
axs[1, 1].pcolormesh(r, z, bz_270, **plot_kwargs)
axs[1, 1].set_title(r'$B_Z$ at $\phi = 270^\circ$', fontsize=11)
axs[1, 1].set_xlabel('R [m]')

# Apply mesh overlay, limits, and aspect ratios to all subplots
for ax in axs.flat:
    m3dc1.plot_mesh(mesh_ob, boundary=True, ax=ax, meshcol='C1', filename=ir)
    ax.set_aspect('equal', adjustable='box')
    ax.set_xlim([1, 3])
    ax.set_ylim([-1.8, 1.8])

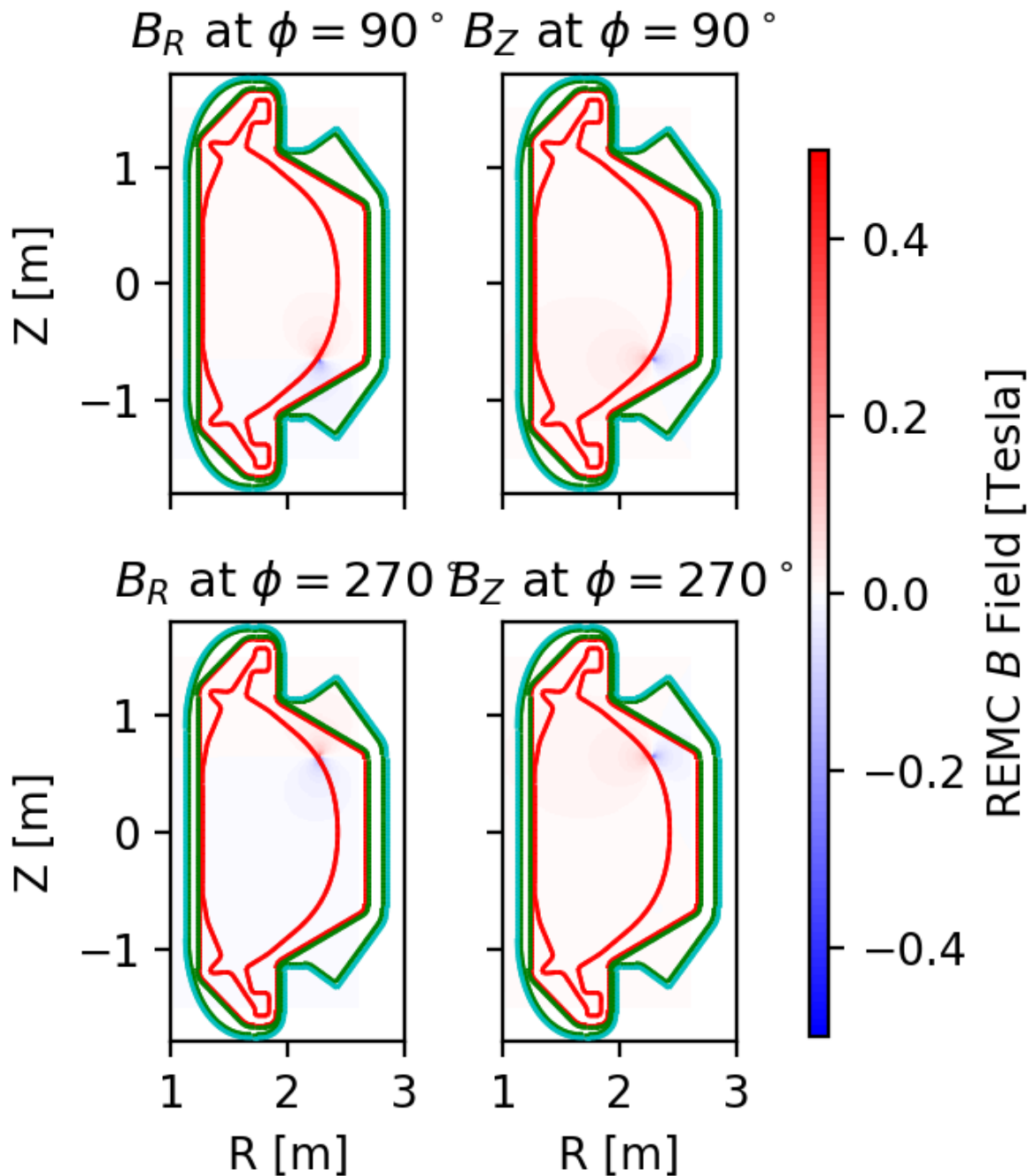
# Clean layout and add shared colorbar
fig.tight_layout()
fig.subplots_adjust(right=0.88)

# Add a dedicated axis for the colorbar so it doesn't squish the right panel
cbar_ax = fig.add_axes([0.90, 0.15, 0.02, 0.7])
fig.colorbar(im, cax=cbar_ax, label=r'REMC $B$ Field [Tesla]')

plt.show()

```

deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields
deleting simulation object and closing 0 fields



```
In [413... # Create a 2x2 grid of subplots
fig, axs = plt.subplots(2, 2, dpi=200, figsize=(3, 4), sharex=True, sharey=True)

base_dir = inDir + sim_id + '/idl_output/'

# Helper function to load and reshape the CSV data
def load_b_component(phi, comp):
    base_file = f"{base_dir}Bext_210_phi_{phi}"

    # Load coordinates
    r = np.loadtxt(f"{base_file}_r.csv", delimiter=',')
    z = np.loadtxt(f"{base_file}_z.csv", delimiter=',')

    # Load and reshape the requested component ('br' or 'bz')
```

```

with open(f'{base_file}_{comp}.csv') as f:
    lines = f.read().strip().splitlines()

val = np.array([list(map(float, line.split(','))) for line in lines])
val = val.reshape(z.shape[0], r.shape[0]).T

return r, z, val

# Load all 4 datasets
r, z, br_90 = load_b_component('90', 'br')
_, _, bz_90 = load_b_component('90', 'bz')
_, _, br_270 = load_b_component('270', 'br')
_, _, bz_270 = load_b_component('270', 'bz')

# Plot styling configuration
plot_kwargs = {
    'shading': 'auto',
    'vmin': -0.5,
    'vmax': 0.5,
    'norm': 'linear',
    'cmap': 'bwr'
}

# --- Column 1: B_R ---
# Row 1: B_R at 90°
im = axs[0, 0].pcolormesh(r, z, br_90, **plot_kwargs)
axs[0, 0].set_title(r'$B_R$ at $\phi = 90^\circ$', fontsize=11)
axs[0, 0].set_ylabel('Z [m]')

# Row 2: B_R at 270°
axs[1, 0].pcolormesh(r, z, br_270, **plot_kwargs)
axs[1, 0].set_title(r'$B_R$ at $\phi = 270^\circ$', fontsize=11)
axs[1, 0].set_ylabel('Z [m]')
axs[1, 0].set_xlabel('R [m]')

# --- Column 2: B_Z ---
# Row 1: B_Z at 90°
axs[0, 1].pcolormesh(r, z, bz_90, **plot_kwargs)
axs[0, 1].set_title(r'$B_Z$ at $\phi = 90^\circ$', fontsize=11)

# Row 2: B_Z at 270°
axs[1, 1].pcolormesh(r, z, bz_270, **plot_kwargs)
axs[1, 1].set_title(r'$B_Z$ at $\phi = 270^\circ$', fontsize=11)
axs[1, 1].set_xlabel('R [m]')

# Apply mesh overlay, limits, and aspect ratios to all subplots
for ax in axs.flat:
    m3dc1.plot_mesh(mesh_ob, boundary=True, ax=ax, meshcol='C1', filename=ir)
    ax.set_aspect('equal', adjustable='box')
    ax.set_xlim([1, 3])
    ax.set_ylim([-1.8, 1.8])

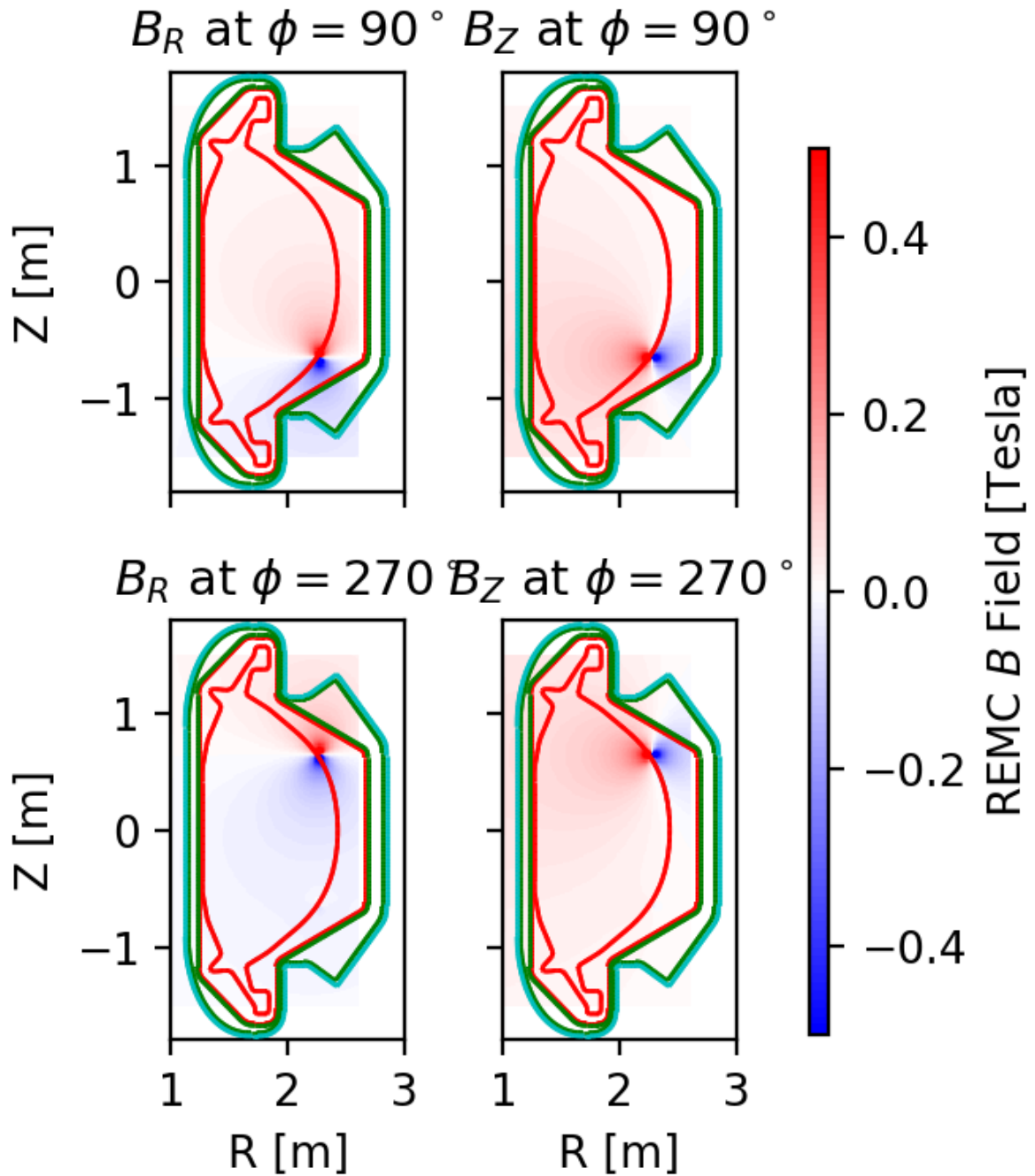
# Clean layout and add shared colorbar
fig.tight_layout()
fig.subplots_adjust(right=0.88)

```

```
# Add a dedicated axis for the colorbar so it doesn't squish the right panel
cbar_ax = fig.add_axes([0.90, 0.15, 0.02, 0.7])
fig.colorbar(im, cax=cbar_ax, label=r'REMC $B$ Field [Tesla]')

plt.show()
```

deleting simulation object and closing 0 fields
 deleting simulation object and closing 0 fields
 deleting simulation object and closing 0 fields
 deleting simulation object and closing 0 fields



Mesh


```
In [24]: fname= '/pscratch/sd/r/rdatta/REMC/TQ/SPRC-CQ-m8-Ne-REMC-2D-01/coil.dat'
import pandas as pd

data = pd.read_csv(fname,delim_whitespace=True,header=None)

fig, axs = plt.subplots(1,1,dpi=400)
plt.plot(data.iloc[:,3],data.iloc[:,4], 'o')

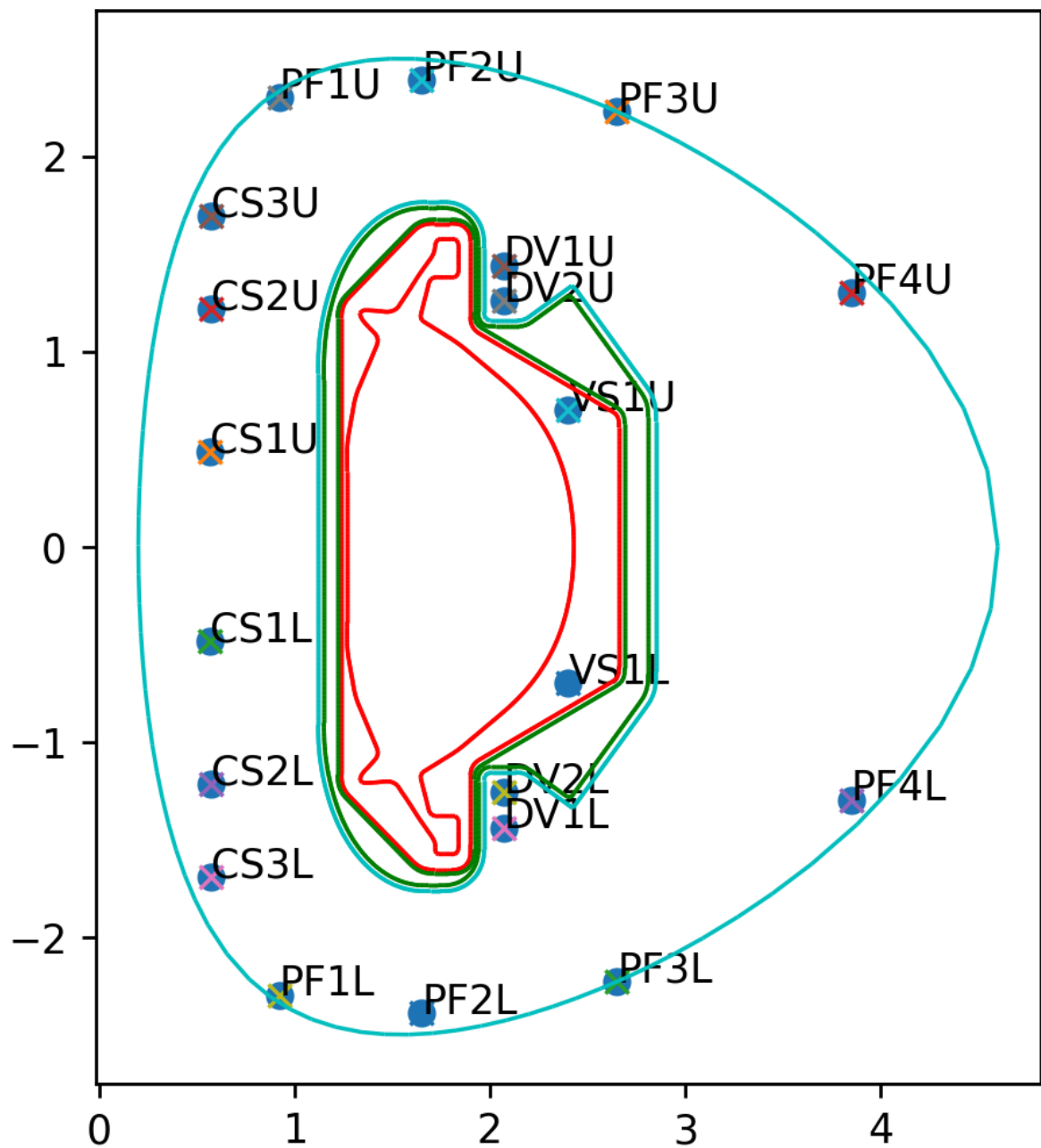
m3dc1.plot_mesh(mesh_ob,boundary=True,ax=axs,meshcol='C1',filename=inDir + s
axs.set_aspect('equal',adjustable='box')

names = data.iloc[:,0]
Rc = data.iloc[:,3]
Zc = data.iloc[:,4]

for ii in range(names.shape[0]):
    axs.plot(Rc[ii],Zc[ii], 'x')
    axs.text(Rc[ii],Zc[ii],names[ii])
```

/tmp/ipykernel_1676050/2439403312.py:4: FutureWarning: The 'delim_whitespace' keyword in pd.read_csv is deprecated and will be removed in a future version. Use ``sep='\s+'`` instead

```
data = pd.read_csv(fname,delim_whitespace=True,header=None)
deleting simulation object and closing 0 fields
```



In []: